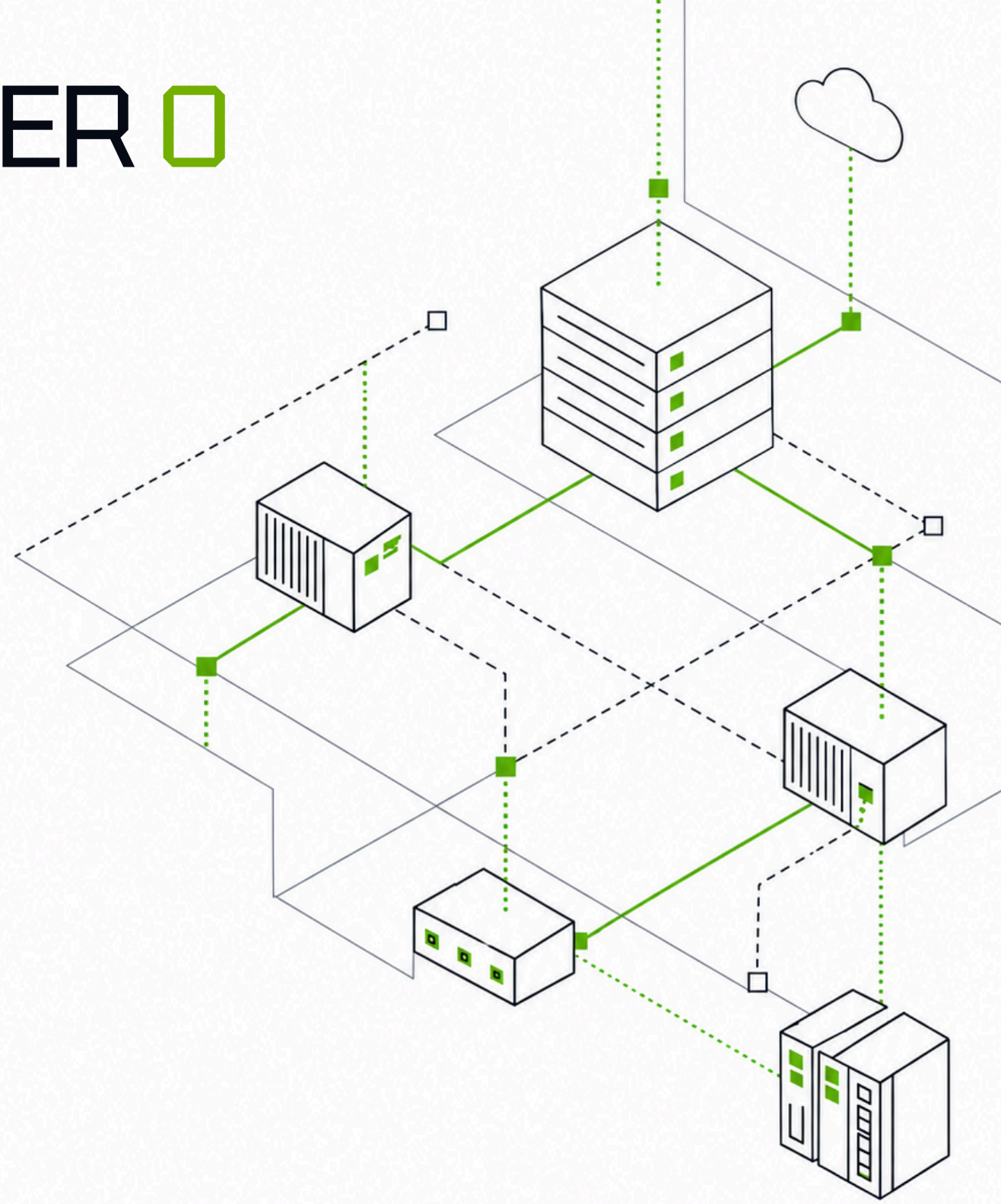
TIER 0



Tier0 企业版

用户手册

目录

概览

- Tier0 是什么?
- 功能列表
- 用户旅程

入门指导

- 部署 Tier0
- 高可用部署方案
- 标准私有化部署方案
- 登录

操作指南

连接数据

- 建立数据模型
- 通过数据流连接数据
- 常规数据源
- 连接 OPC UA
- 连接 OPC DA
- 连接 Modbus
- 连接 MQTT
- 连接文件
- 连接 RestAPI
- 连接 MQTT 客户端

处理数据

- 处理数据
- 常规数据处理
- 整合多源数据
- 数据过滤
- 更改数据
- 拆分数据
- 数据可视化
- 使用 UNS Agent

高阶使用

同步数据
- 数据同步
- 从 Tier0 获取数据

使用数据
- 工业应用
- 配置 3D 模型
- 构建 3D 场景

分析数据
- 分析数据
- 视觉分析

系统设置

术语表

概览

Tier0 是什么?

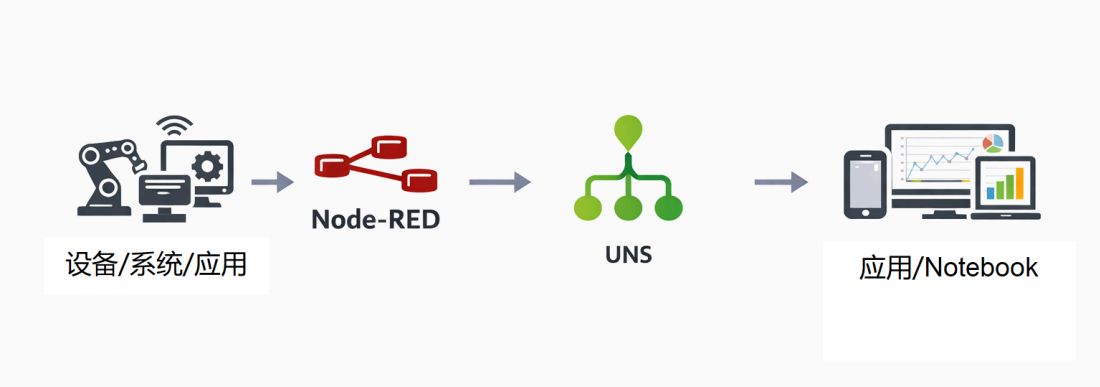
Tier0 是一个基于统一命名空间（UNS）的工业数据平台。

它通过 Node-RED 连接数据，将其组织为统一结构，并提供给上层应用和业务分析使用。

它如何工作?

Tier0 在一个统一的流程中对数据建模，处理数据：

- 基于统一命名空间概念，建立结构清晰的数据模型
- Node-RED 从设备和系统中连接并处理数据
- 数据被发布到统一命名空间（UNS）的数据模型并存储到平台数据库
- 数据被仪表板、应用程序和分析工具消费

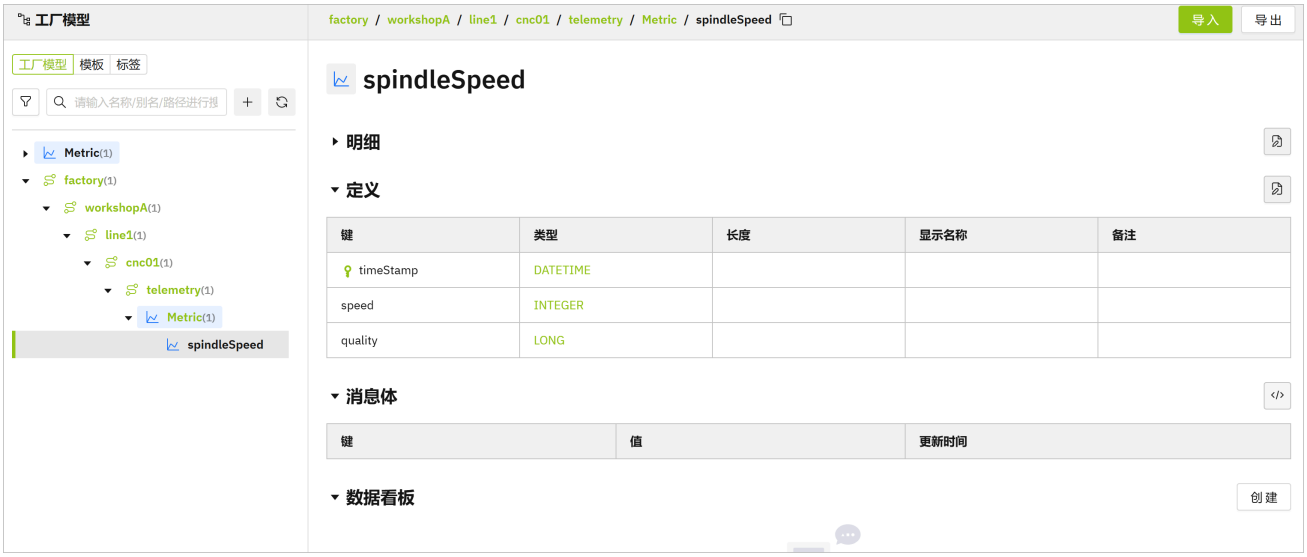


核心组件

统一命名空间（UNS）

为所有工业数据定义分层结构。

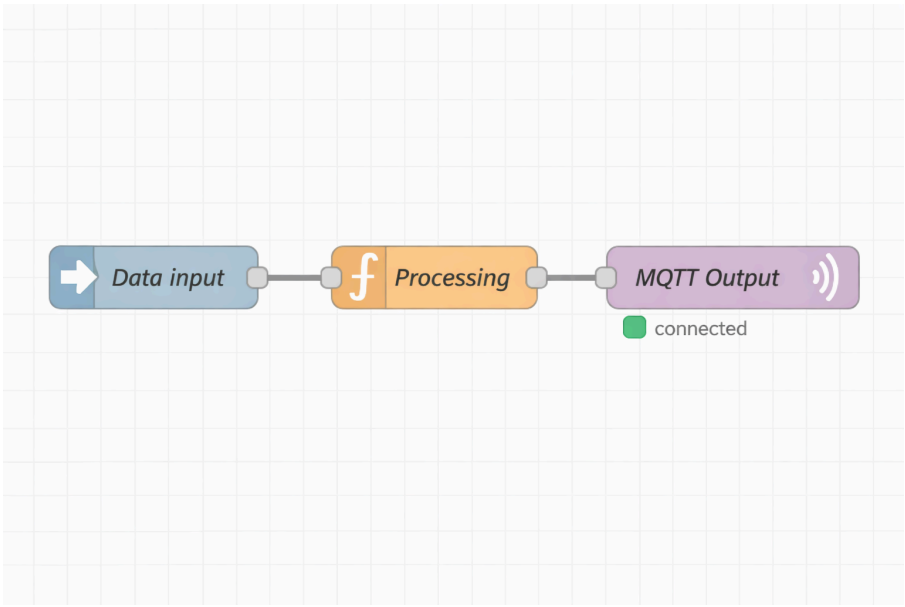
`factory/workshopA/line1/cnc01/telemetry/spindleSpeed`



Node-RED 集成

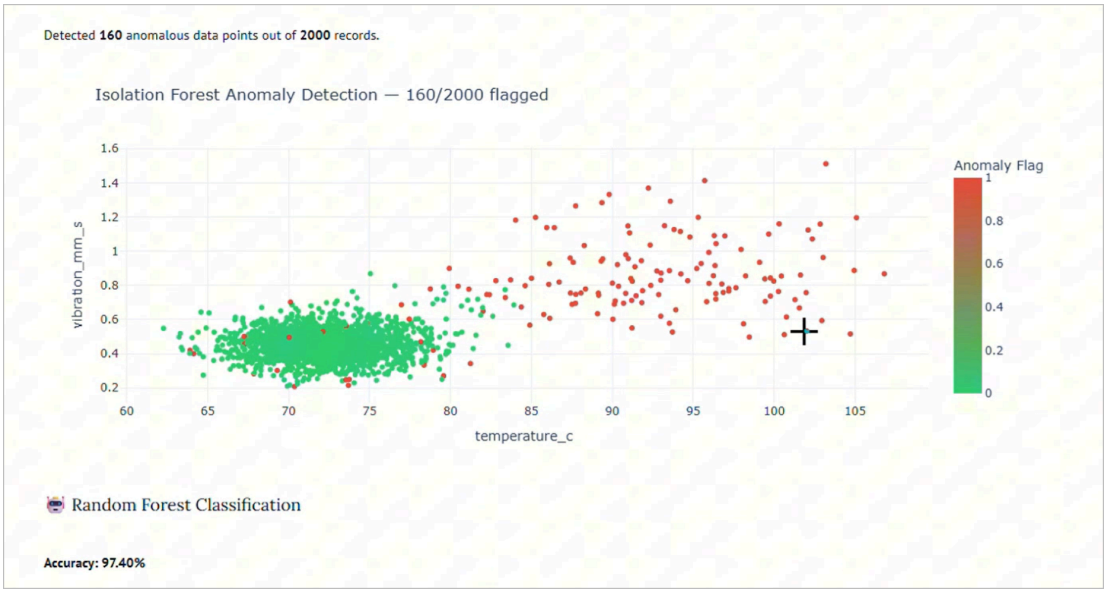
用于数据连接与处理的核心引擎。

- 连接设备、应用和系统
- 将数据格式化并发布到命名空间中



Python Notebook

使用 Python 在交互式 Notebook 中灵活分析数据。

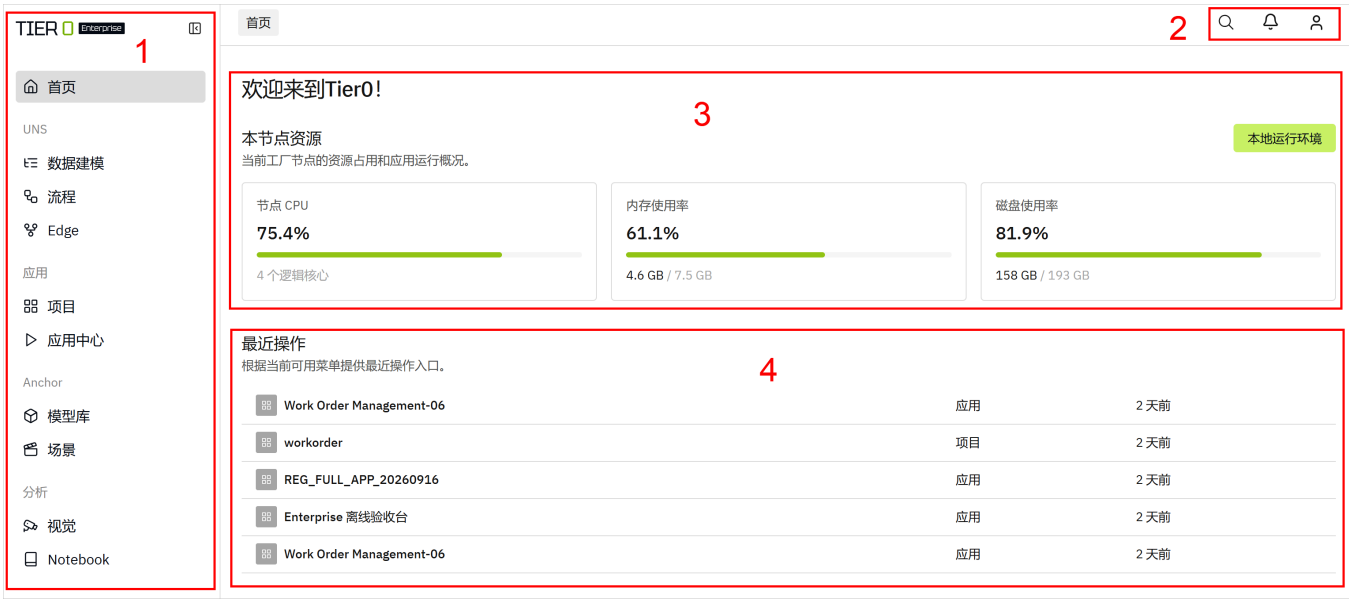


Agentic 应用构建器

使用自然语言在云端创建工业应用，无需消耗任何本地资源。应用创建完成可打包本地安装使用。



功能列表



首页

序号	名称	说明
1	菜单栏	列出平台功能菜单，便于快速进入各个模块。
2	工具栏	提供搜索、通知和系统设置等常用操作。
3	本节点资源	显示当前 Tier0 节点的资源。**本地运行环境**显示所有已安装应用的运行状态。
4	最近操作	列出近期操作，便于快速继续工作。

功能模块说明

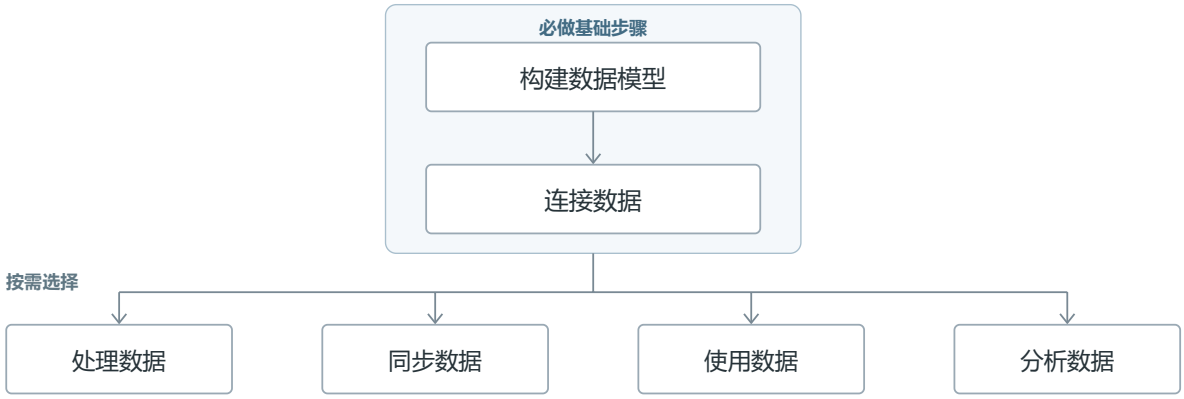
模块	名称	说明
UNS	数据建模	以文件夹和文件的层级结构组织工业数据。
	流程	使用 Node-RED 为已创建的数据模型配置数据源。
	Edge	连接 MQTT 客户端或Tier0 边缘节点，管理数据。
应用	项目	对从 Tier0 云服务版导入的应用进行分类和管理。
	应用中心	展示当前用户有权访问的应用。
Anchor	模型库	将数字孪生模型与 UNS 数据关联。
	场景	可视化工厂设备和生产场景。

模块	名称	说明
分析	视觉	对实时摄像头画面进行视觉分析。
	Notebook	使用 Marimo Notebook 编写 Python 脚本，例如训练模型、执行 SQL 查询和生成交互式数据报告。

> **信息**
>
> - 私有化部署的要求和步骤参见部署 Tier0。
> - 使用自然语言在 **App Builder** 中构建的应用在云端生成，可导出安装包并导入本地。

用户旅程

先完成图中的两个必做步骤：



- **必做**
  i. **构建数据模型**，组织资产并确定数据的归属位置。
  ii. **连接数据**，将数据源中的数据接入模型。
- **可选**
  - **处理数据**，通过**事件流程**转换和准备数据。
  - **同步数据**，与其他 Tier0 同步数据，或通过 OpenAPI 和 MQTT 对外提供数据。
  - **使用数据**，将数据用于应用和业务场景。
  - **分析数据**，探索并可视化数据，从中获得洞察。

入门指导

部署 Tier0

环境要求

- 单机部署
  - 最低：4C / 8G / 100G
  - 推荐：8C / 16G / 500G
- 标准部署
  - 最少：4 台虚拟机
  - 各 VM 类型的基线配置：
    - App：8C / 32G / 100G
    - Database：4C / 8G / 500G
    - Witness：2C / 4G / 40G
- 高可用部署
  - 最少：6 台虚拟机
  - 各 VM 类型的基线配置：
    - App (Swarm)：8C / 32G / 100G x 3
    - Database：4C / 8G / 500G x 2
    - Witness：2C / 4G / 40G

> **硬件要求可能会变化**
>
> Enterprise 的硬件要求会随部署场景变化。

> **备注**
>
> 详细部署信息见标准部署方案和高可用部署方案。

操作步骤

1. 从供应商处获取安装包，并完成校验。

```
sha256sum -c tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz.sha256
```

2. 解压安装包。

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. 根据部署信息编辑 `install.env`，然后开始部署。

```
vi install.env
bash install.sh
```

常用维护操作

- 在不安装 Enterprise 的情况下检查部署环境。

```
./install.sh --check-only
```

- 部署失败后清理所有节点。

```
./full-cleanup.sh --env install.env --yes
```

> ⚠ **影响说明**
>
> 此命令会清除全部 Tier0 运行时数据和数据库备份，但会保留虚拟机、操作系统以及已安装的 Docker 软件。

- 安装完成后会生成部署报告。

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> ⚠ **报告中出现 FAIL 时应该怎么做**
>
> 如果报告中包含任何 **FAIL** 项，请不要交付项目，请先联系支持团队。

高可用部署方案

方案目标

在离线或私有化环境中部署一套 Tier0 企业版，总计使用6台虚拟机：

- 3台：应用节点。组成 `Docker Swarm` 集群基座，承载 Tier0 企业版单副本服务和 **EMQX** 有状态服务
- 2台：数据库节点。组成 `PostgreSQL + TimescaleDB` 主从集群
- 1台：`Witness` 节点。参与数据库监督、仲裁和故障切换

> ⓘ **可选**
>
> - 引入外部 `NFS/NAS` 共享存储，供三台 Swarm 节点挂载，用于 Tier0 企业版跨节点共享文件。
> - 该共享存储是外部依赖，不计入本方案6台业务虚拟机。

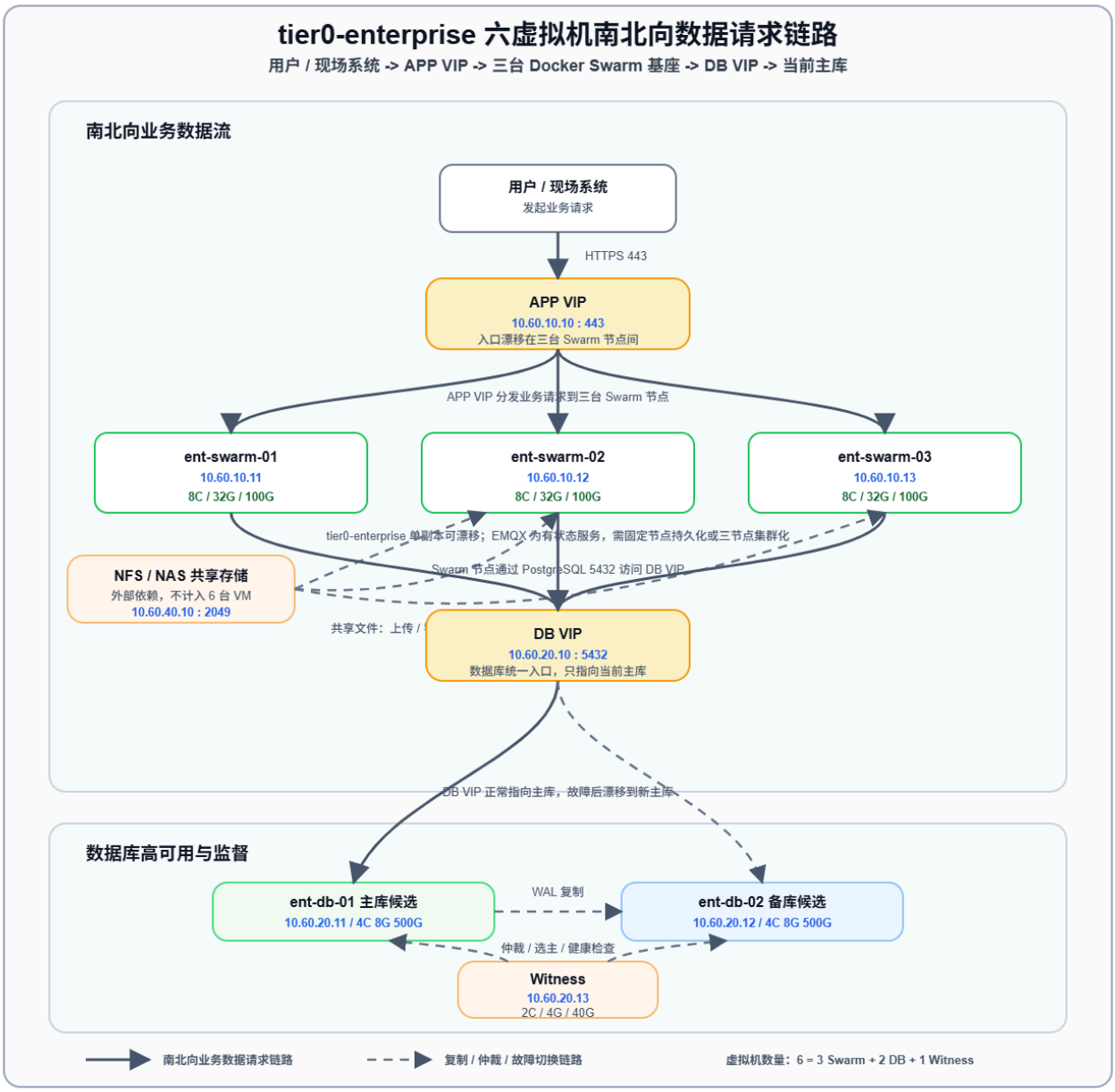
> **信息**
>
> 本方案假设每台数据库虚拟机规格为 `4C / 8G / 500G`。若用户实际要求为数据库集群合计 `4C / 8G / 500G`，则需要在实施前重新核算性能和容量。

规划 IP

> **信息**
>
> 以下 IP 为规划占位，实施前必须替换为现场实际网段。

类型	名称	规划 IP	说明
逻辑入口	APP VIP	10.60.10.10	业务统一入口，漂移在 3 台 Swarm 节点之间
外部依赖	NFS/NAS	10.60.40.10	可选共享存储，不计入 6 台 VM
应用节点	ent-swarm-01	10.60.10.11	Docker Swarm manager/worker，承载可漂移服务；可作为 EMQX 固定节点或 EMQX 集群节点
应用节点	ent-swarm-02	10.60.10.12	Docker Swarm manager/worker，承载可漂移服务；可作为 EMQX 集群节点
应用节点	ent-swarm-03	10.60.10.13	Docker Swarm manager/worker，承载可漂移服务；可作为 EMQX 集群节点
逻辑入口	DB VIP	10.60.20.10	数据库统一入口，只指向当前主库
数据库节点	ent-db-01	10.60.20.11	PostgreSQL + TimescaleDB 主库候选
数据库节点	ent-db-02	10.60.20.12	PostgreSQL + TimescaleDB 备库候选
仲裁节点	ent-db-witness-01	10.60.20.13	repmgr witness / 仲裁监督节点

总体拓扑



> **信息**
>
> 图中只表达核心链路。

- 业务访问路径：用户 / 现场系统 -> `APP VIP` -> 三台 `Docker Swarm` 节点 -> `DB VIP` -> 当前主库
- 图面按南北向从上到下排列，表现请求进入、应用处理、数据库访问和高可用监督关系
- 拓扑图不展示集群内部端口、SSH 运维链路和监控链路，统一放到连接方式清单和端口明细清单中
- `Witness` 只参与数据库仲裁、选主和健康检查，不在业务访问路径中
- `APP VIP` 直接分发到三台 Swarm 节点，并统一通过 `DB VIP` 访问数据库
- `APP VIP` 漂移在三台 Swarm 节点之间，`DB VIP` 随当前 PostgreSQL Primary 角色在两台数据库节点之间漂移
- Tier0 企业版按单副本服务运行，节点故障后由 Swarm 调度到其他健康节点重启
- `EMQX` 是有状态服务，不按普通无状态服务随机漂移；默认应固定节点持久化，若要求消息层高可用则部署三节点 EMQX 集群

- 可选 `NFS/NAS` 只作为 Swarm 节点共享文件存储，不进入主数据库请求链路

数据库监督与故障切换

数据库切换链路：

阶段	链路	说明
正常访问	Swarm 节点上的 Tier0 企业版副本 -> `DB VIP` -> 当前主库	应用始终只连接 `10.60.20.10:5432`
正常复制	当前主库 -> 当前备库	通过 PostgreSQL WAL 流复制同步数据
正常监督	`DB-01` / `DB-02` -> `Witness`	`repmgrd` 上报状态，`Witness` 参与仲裁
主库故障	备库 -> `Witness`	备库请求仲裁，避免脑裂
备库提升	备库 `promote`	`Witness` 确认后，备库提升为新主库
VIP 漂移	`DB VIP` -> 新主库	`10.60.20.10` 绑定到新主库
应用恢复	Swarm 节点上的 Tier0 企业版副本 -> `DB VIP` -> 新主库	应用连接池重连后恢复读写

关键边界：

- `Witness` 不承载业务数据，不参与业务读写，也不对应用开放数据库访问
- 业务只连接 `DB VIP:5432`，禁止直连 `ent-db-01/02`
- `DB VIP` 必须绑定当前 PostgreSQL Primary 节点，不能仅以数据库进程存活作为漂移依据
- `TimescaleDB` 作为 PostgreSQL 扩展部署在两个数据库节点上，不单独拆库
- 建议采用 `PostgreSQL streaming replication + repmgr + repmgr witness + keepalived DB VIP`
- 自动故障切换只处理主库不可用场景；原主库恢复后先作为备库重新加入，不自动抢回主库

节点配置清单

节点	角色	vCPU	内存	存储	系统盘建议	数据盘建议	主要组件
`ent-swarm-01`	Swarm manager/worker	8	32G	100G	单块系统盘 / 100G	不单独拆分	Docker Engine、Swarm、Keepalived、HAProxy/Nginx
`ent-swarm-02`	Swarm manager/worker	8	32G	100G	单块系统盘 / 100G	不单独拆分	Docker Engine、Swarm、Keepalived、HAProxy/Nginx
`ent-swarm-03`	Swarm manager/worker	8	32G	100G	单块系统盘 / 100G	不单独拆分	Docker Engine、Swarm、Keepalived、HAProxy/Nginx

节点	角色	vCPU	内存	存储	系统盘建议	数据盘建议	主要组件
`ent-db-01`	PostgreSQL/TSDB 主库候选	4	8G	500G	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-02`	PostgreSQL/TSDB 备库候选	4	8G	500G	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-witness-01`	Witness/仲裁监督	2	4G	40G	`/` 40G	不单独规划	repmgr witness、监控 agent、审计日志

配置汇总清单

类别	数量	单台规格	小计 vCPU	小计内存	小计存储
Docker Swarm 应用节点	3	8C / 32G / 100G	24C	96G	300G
PostgreSQL + TimescaleDB 节点	2	4C / 8G / 500G	8C	16G	1000G
Witness 节点	1	2C / 4G / 40G	2C	4G	40G
NFS/NAS 共享存储	1	现场已有或单独提供	不计入	不计入	按文件量规划
合计	6	-	34C	116G	1340G

换算后硬件需求为：

- 虚拟机总数：6 台
- vCPU 总量：34
- 总内存：116G
- 总存储：1.34T
- 可选外部共享存储：不计入 6 台 VM 汇总，容量按附件、报表、导入导出和日志归档保留周期单独规划

Swarm 服务运行口径

服务	运行方式	状态属性	故障处理	说明
Tier0 企业版	`replicas: 1`	无状态或弱状态	当前节点故障后由 Swarm 漂移重启到其他节点	数据必须落 PostgreSQL/TimescaleDB，不能依赖容器本地文件
EMQX 单节点模式	固定到指定 Swarm 节点	有状态	固定节点故障时需恢复节点或从备份恢复	适合规模较小但接受消息层单点的场景

服务	运行方式	状态属性	故障处理	说明
EMQX 三节点集群模式	三台 Swarm 节点各固定 1 个 EMQX 实例	有状态集群	单节点故障后其余 EMQX 节点继续服务	若要求消息层高可用，优先采用此模式
APP VIP / HAProxy / Nginx	三节点部署，VIP 漂移	运行态配置	健康节点接管 VIP	只作为入口，不保存业务数据
NFS/NAS 共享目录	三台 Swarm 节点统一挂载	文件状态	NFS 故障时依赖共享文件的功能受影响	仅用于上传、导入导出、报表等共享文件

关键口径：

- Tier0 企业版可以单副本运行，Swarm 只提供集群基座和故障漂移能力
- `EMQX` 不能简单套用 Tier0 企业版的单副本随机漂移策略，因为它有会话、订阅、保留消息、规则引擎状态和持久化目录
- 如果 `EMQX` 只部署单节点，应明确它是消息层单点，并固定到一台 Swarm 节点的持久化目录
- 如果业务不能接受 `EMQX` 单点，应在三台 Swarm 节点上部署三节点 EMQX 集群，并为每个节点配置固定主机名，节点名和持久化目录

NFS/NAS 共享存储口径

引入 NFS 的目的，是为了 Tier0 企业版单副本在 Swarm 节点之间漂移后仍能访问同一份业务文件。它不改变数据库主从和 EMQX 有状态服务边界。

适合放到 NFS 的内容：

- 用户上传附件
- 导入 / 导出文件
- 报表文件
- 离线包、模板、可再生成的中间文件
- 需要跨节点读取的业务静态文件

不建议放到 NFS 的内容：

- PostgreSQL / TimescaleDB 数据目录、WAL 目录
- EMQX 核心数据目录、Mnesia 数据、运行时状态
- Docker Engine 数据目录 `/var/lib/docker`
- 高频数据库型写入、锁敏感文件、强一致队列

建议挂载口径：

项	建议
NFS 地址	`10.60.40.10:/exports/tier0-enterprise`
Swarm 挂载点	`/mnt/tier0-shared`

项	建议
容器内挂载点	`/app/data/shared` 或按应用配置
协议版本	优先 NFSv4.1 或现场标准版本
挂载参数	`rw,hard,timeo=600,retrans=2,_netdev,noatime`，按现场 NAS 兼容性调整
权限	固定业务 UID/GID，避免容器节点漂移后权限不一致
备份	共享目录纳入 NAS 快照或独立备份

NFS 边界：

- NFS 是共享文件依赖，不是数据库高可用组件
- NFS 本身如果是单点，会成为文件功能单点；生产建议使用已有 HA NAS 或双控存储
- Tier0 企业版单副本漂移到其他 Swarm 节点前，目标节点必须已挂载并验证 NFS 可读写
- 如果 Tier0 企业版完全不写本地文件，可不引入 NFS

连接方式清单

运维访问

目标	方式	端口	说明
所有虚拟机	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	统一免密实施后使用；未实施前以现场账号为准
APP VIP	`https://10.60.10.10`	`443/TCP`	业务入口
APP VIP	`http://10.60.10.10`	`80/TCP`	可选，建议只用于跳转 HTTPS
DB VIP	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	只允许应用节点和运维白名单访问
NFS/NAS	`10.60.40.10:/exports/tier0-enterprise`	`2049/TCP`	可选共享文件存储

Swarm 集群通信

源	目标	端口	用途
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`2377/TCP`	Swarm manager 管理面
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`7946/TCP/UDP`	节点发现和 gossip
`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	`4789/UDP`	overlay 网络 VXLAN

源	目标	端口	用途
APP VIP	ent-swarm-01/02/03	80/443/TCP	业务入口转发

EMQX 访问与集群通信

源	目标	端口	用途
设备 / 现场系统	APP VIP 或 EMQX 入口	1883/TCP	MQTT 明文接入，按现场安全要求决定是否开放
设备 / 现场系统	APP VIP 或 EMQX 入口	8883/TCP	MQTT TLS 接入，建议优先使用
设备 / 现场系统	APP VIP 或 EMQX 入口	8083/TCP	MQTT WebSocket，可选
设备 / 现场系统	APP VIP 或 EMQX 入口	8084/TCP	MQTT WebSocket TLS，可选
运维网段	EMQX 管理入口	18083/TCP	EMQX Dashboard，仅允许运维白名单
ent-swarm-01/02/03	ent-swarm-01/02/03	4369/TCP	EMQX/Erlang EPMD，只有三节点 EMQX 集群模式需要
ent-swarm-01/02/03	ent-swarm-01/02/03	4370/TCP	EMQX 节点间通信，只有三节点 EMQX 集群模式需要
ent-swarm-01/02/03	ent-swarm-01/02/03	5370/TCP	EMQX 集群 RPC，按实际 EMQX 版本确认

数据库与监督通信

源	目标	端口	用途
Tier0 企业版容器	DB VIP	5432/TCP	业务数据库访问
ent-db-01	ent-db-02	5432/TCP	PostgreSQL WAL 流复制
ent-db-02	ent-db-01	5432/TCP	故障恢复后反向复制
ent-db-01/02	ent-db-witness-01	5432/TCP	repmgr metadata / witness 通信
ent-db-01	ent-db-02	VRRP	DB VIP 漂移，需允许协议 112
ent-swarm-01/02/03	ent-db-01/02/Witness	9100/TCP	可选 node exporter 监控

NFS/NAS 共享存储通信

源	目标	端口	用途
`ent-swarm-01/02/03`	`NFS/NAS 10.60.40.10`	`2049/TCP`	NFSv4 共享文件访问
运维网段	`NFS/NAS 10.60.40.10`	按现场管理端口	NAS 管理、快照、备份

服务部署建议

Docker Swarm

- 三台应用节点均作为 `manager + worker`
- Tier0 企业版以 `docker stack deploy` 发布
- Tier0 企业版默认 `replicas: 1`，Swarm 负责在节点故障后重新调度到其他健康节点
- Tier0 企业版容器不保存关键持久数据，业务数据统一进入 PostgreSQL/TimescaleDB
- Swarm 节点只有 `100G` 存储，按单块系统盘使用；Docker 数据目录保留在系统盘内，不单独规划数据盘
- 配置文件使用 `Docker config`
- 密码、数据库口令、证书私钥使用 `Docker secret`
- 镜像仓库建议使用现场内网 registry，避免部署依赖公网
- 发布时建议配置健康检查，避免入口转发到未就绪容器
- 如应用需要跨节点共享文件，将共享目录挂载到 NFS/NAS，禁止依赖容器本地临时目录保存关键文件
- 需要配置镜像、容器日志和构建缓存清理策略，避免 `/var/lib/docker` 占满系统盘

单副本部署示例口径：

```
deploy:
  replicas: 1
  restart_policy:
    condition: any
  update_config:
    order: stop-first
```

EMQX 有状态服务

`EMQX` 需要按有状态服务处理，不能简单作为普通 Swarm 无状态副本随机漂移。

模式	推荐场景	部署口径	风险
单节点固定模式	小规模、能接受消息层短时不可用	固定部署到 `ent-swarm-01`，使用本地持久化目录和备份	`ent-swarm-01` 故障时 EMQX 不自动无损漂移
三节点集群模式	正式生产、不能接受 EMQX 单点	三台 Swarm 节点各运行 1 个 EMQX 节点，固定 hostname、node name 和数据目录	需要额外配置集群端口、证书、节点发现和监控

单节点固定模式最低要求：

- 使用 Swarm placement constraint 固定节点，例如 `node.labels.emqx == true`
- 数据目录固定到宿主机，例如 `/data/emqx`
- 配置、证书、插件、规则引擎配置纳入备份
- 节点故障时按“恢复原节点优先，其次从备份恢复到新节点”处理，不做无状态式随机漂移

三节点集群模式最低要求：

- 每台 Swarm 节点固定一个 EMQX 实例
- 每个 EMQX 节点使用固定 node name，不随容器重建变化
- 集群通信端口只在三台 Swarm 节点之间开放
- Dashboard 只对运维网段开放
- MQTT 入口可通过 `APP VIP` / 独立 MQTT VIP / 四层负载均衡暴露，实施前需明确入口方式
- 不建议把 EMQX 核心数据目录放到 NFS；EMQX 单节点固定模式应优先使用固定节点本地盘并做好备份，三节点模式依赖 EMQX 自身集群机制

NFS/NAS 共享存储

- NFS/NAS 为可选外部依赖，不占用六台 VM 名额
- 三台 Swarm 节点统一挂载同一个共享目录
- 只承载 Tier0 企业版需要跨节点访问的共享文件
- 不承载 PostgreSQL/TimescaleDB 数据目录
- 不承载 EMQX 核心运行数据
- 需要配置启动前挂载检查，避免服务漂移到 NFS 未挂载节点后写入本地目录
- NFS 故障时，数据库读写不受影响，但上传、导入导出、报表等文件功能可能失败

APP VIP

- `APP VIP` 建议使用 `keepalived + HAProxy` 或 `keepalived + Nginx`，部署在三台 Swarm 节点
- VIP 只作为统一入口，不单独占用虚拟机
- 健康检查应以应用 `/health` 或 `/ready` 为准
- 任一 Swarm 节点故障时，VIP 漂移到其余健康节点

PostgreSQL + TimescaleDB

建议基线：

- PostgreSQL：`16` 或现场认证版本
- TimescaleDB：选择与 PostgreSQL 主版本匹配的稳定版本
- 主从复制：物理流复制
- 故障切换：`repmgrd + witness`
- 数据库入口：`DB VIP`
- 连接池：优先在应用侧或单独容器内使用 `PgBouncer`，避免数据库连接数过高

建议参数起点，实施时需按业务压力测试调整：

参数	建议值
`shared_buffers`	`2GB`

参数	建议值
`effective_cache_size`	`6GB`
`work_mem`	`16MB` 起步
`maintenance_work_mem`	`512MB`
`max_connections`	`200` 以内，优先配连接池
`wal_level`	`replica`
`max_wal_senders`	`10`
`max_replication_slots`	`10`
`hot_standby`	`on`
TimescaleDB 压缩	按时序表保留周期开启

Witness

- `Witness` 节点仅保存 `repmgr witness` 元数据，用于辅助数据库故障判断和仲裁
- `Witness` 不承载业务数据，不参与业务读写，也不对应用开放数据库访问
- `Witness` 需要稳定网络，不要求高存储性能
- `Witness` 故障不应中断当前业务访问，但会降低自动切换可信度
- `Witness` 恢复后必须验证其重新加入数据库集群状态

端口与防火墙基线

端口/协议	放行范围	用途
`22/TCP`	运维网段到 6 台 VM	SSH
`80/TCP`	用户网段到 `APP VIP`	HTTP 跳转或内网明文入口
`443/TCP`	用户网段到 `APP VIP`	HTTPS 业务入口
`2377/TCP`	3 台 Swarm 节点互通	Swarm 管理面
`7946/TCP/UDP`	3 台 Swarm 节点互通	Swarm gossip
`4789/UDP`	3 台 Swarm 节点互通	Swarm overlay
`1883/TCP`	设备 / 现场系统到 MQTT 入口	EMQX MQTT 明文，可选

端口/协议	放行范围	用途
`8883/TCP`	设备 / 现场系统到 MQTT 入口	EMQX MQTT TLS，建议
`8083/TCP`	设备 / 现场系统到 MQTT WebSocket 入口	EMQX WebSocket，可选
`8084/TCP`	设备 / 现场系统到 MQTT WebSocket TLS 入口	EMQX WebSocket TLS，可选
`18083/TCP`	运维网段到 EMQX 管理入口	EMQX Dashboard，仅白名单
`5432/TCP`	Swarm 节点到 `DB VIP`；DB/Witness 内部互通	PostgreSQL / repmgr
`2049/TCP`	Swarm 节点到 NFS/NAS	NFSv4 共享文件访问，可选
`112/VRRP`	Swarm 节点间、DB 节点间	VIP 漂移
`9100/TCP`	监控系统到 6 台 VM	node exporter，可选

端口明细清单

对外入口端口

端口/协议	监听位置	访问来源	用途	必须开放
`80/TCP`	`APP VIP 10.60.10.10`	用户网段 / 运维网段	HTTP 入口或跳转 HTTPS	可选
`443/TCP`	`APP VIP 10.60.10.10`	用户网段 / 运维网段	HTTPS 业务入口	是
`1883/TCP`	MQTT 入口	设备 / 现场系统	EMQX MQTT 明文	可选
`8883/TCP`	MQTT 入口	设备 / 现场系统	EMQX MQTT TLS	建议
`8083/TCP`	MQTT WebSocket 入口	设备 / 现场系统	EMQX WebSocket	可选
`8084/TCP`	MQTT WebSocket TLS 入口	设备 / 现场系统	EMQX WebSocket TLS	可选
`18083/TCP`	EMQX Dashboard	运维白名单	EMQX 管理控制台	可选，必须限源
`22/TCP`	6 台 VM 管理 IP	运维网段	SSH 运维登录	是
`5432/TCP`	`DB VIP 10.60.20.10`	3 台 Swarm 节点、运维白名单	PostgreSQL 业务访问	是
`2049/TCP`	`NFS/NAS 10.60.40.10`	3 台 Swarm 节点	NFSv4 共享文件访问	按是否引入 NFS

Docker Swarm 节点端口

端口/协议	监听节点	访问来源	用途	必须开放
`2377/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm 集群管理面	是
`7946/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm 节点发现 / gossip	是
`7946/UDP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm 节点发现 / gossip	是
`4789/UDP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	Swarm overlay VXLAN	是
`80/TCP`	`ent-swarm-01/02/03`	`APP VIP` / 内部 LB	应用 HTTP 后端	按部署方式
`443/TCP`	`ent-swarm-01/02/03`	`APP VIP` / 内部 LB	应用 HTTPS 后端	按部署方式
`4369/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX/Erlang EPMD，三节点集群模式	按 EMQX 模式
`4370/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX 节点间通信，三节点集群模式	按 EMQX 模式
`5370/TCP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	EMQX 集群 RPC，按版本确认	按 EMQX 模式

数据库集群端口

端口/协议	监听节点	访问来源	用途	必须开放
`5432/TCP`	`ent-db-01`	`ent-swarm-01/02/03`、`ent-db-02`、`ent-db-witness-01`	主库业务访问、复制、repmgr metadata	是
`5432/TCP`	`ent-db-02`	`ent-swarm-01/02/03`、`ent-db-01`、`ent-db-witness-01`	备库提升后业务访问、复制、repmgr metadata	是
`5432/TCP`	`ent-db-witness-01`	`ent-db-01/02`	repmgr witness 通信	是
`873/TCP`	`ent-db-01/02`	运维备份节点	rsync 备份通道	可选
`6432/TCP`	PgBouncer 容器或数据库节点	Tier0 企业版容器	PostgreSQL 连接池入口	可选

VIP 与监控端口

端口/协议	监听节点	访问来源	用途	必须开放
`112/VRRP`	`ent-swarm-01/02/03`	`ent-swarm-01/02/03`	APP VIP 漂移	是，若使用 Keepalived

端口/协议	监听节点	访问来源	用途	必须开放
`112/VRRP`	`ent-db-01/02`	`ent-db-01/02`	DB VIP 漂移	是，若使用 Keepalived
`9100/TCP`	6 台 VM	监控系统	node exporter	建议
`9187/TCP`	`ent-db-01/02`	监控系统	postgres exporter	建议
`9323/TCP`	`ent-swarm-01/02/03`	监控系统	Docker metrics，需显式开启	可选
EMQX metrics 端口	EMQX 节点	监控系统	EMQX 指标，按版本和配置确认	建议

默认不开放端口

端口/协议	说明
`2375/TCP`	Docker 明文远程 API，禁止开放
`2376/TCP`	Docker TLS 远程 API，默认不开放；确需使用时只允许运维网段
数据库节点 `80/443`	数据库节点不承载 Web 入口，默认关闭
Witness `5432` 对应用网段	Witness 不承载业务 SQL，禁止应用访问
EMQX Dashboard 对用户网段	`18083/TCP` 只能对运维白名单开放
NFS 对非 Swarm 网段	`2049/TCP` 只允许三台 Swarm 节点和备份/运维白名单访问

高可用能力与故障边界

故障场景	预期行为	业务影响
单台 Swarm 节点故障，影响 Tier0 企业版单副本	Swarm 将单副本重新调度到其他健康节点，APP VIP 继续指向健康入口	秒级到分钟级中断，取决于镜像拉取和健康检查
单节点固定模式下 EMQX 所在节点故障	优先恢复原节点；若无法恢复，从备份恢复到新节点	消息层中断，不能按无状态服务无损漂移
三节点 EMQX 集群中单节点故障	其余 EMQX 节点继续服务，故障节点恢复后重新加入	连接会重连，容量下降
NFS/NAS 故障	依赖共享文件的功能失败，数据库和已在内存中的业务流程不一定中断	上传、导入导出、报表等文件功能受影响
两台 Swarm 节点故障	Swarm manager 失去多数派，已运行容器可能继续，但无法可靠调度	高风险，需人工恢复

故障场景	预期行为	业务影响
当前数据库主库故障	备库经 Witness 仲裁后提升，DB VIP 漂移	秒级到分钟级写入中断，应用需重连
数据库备库故障	主库继续服务，复制中断告警	无直接业务中断，但失去数据库热备
Witness 故障	当前主库继续服务，自动切换能力下降	无直接业务中断，需优先恢复
DB VIP 漂移失败	数据库实际已切主但入口不可达	业务中断，需手动绑定 VIP

备份、监控与巡检

最低交付要求：

- 数据库每日全量备份，至少保留 `7` 天
- WAL 归档或增量备份，满足现场 RPO
- 每月做一次恢复演练，验证备份可用
- Swarm 服务健康检查、容器重启次数、节点磁盘、内存、CPU 纳入监控
- EMQX 连接数、订阅数、消息吞吐、队列堆积、规则引擎失败、持久化目录使用率纳入监控
- NFS 可用性、挂载状态、容量、inode、读写延迟纳入监控
- PostgreSQL 主从延迟、复制槽、WAL 堆积、连接数、慢 SQL、数据库膨胀纳入监控
- VIP 当前归属、repmgr 集群状态、Witness 状态纳入巡检

建议巡检命令：

```
docker node ls
docker service ls
docker stack ps tier0-enterprise
ssh ent-db-01 'repmgr cluster show'
ssh ent-db-01 'psql -h 10.60.20.10 -p 5432 -c "select now();"'
ssh ent-db-01 'ip addr show | grep 10.60.20.10'
```

上线前检查清单

检查项	通过标准
6 台 VM 主机名固定	`hostnamectl` 与规划一致
统一 SSH 可用	运维终端可免密访问 6 台 VM
时间同步	6 台 VM 时间偏差小于 `1s`
DNS/hosts 固定	节点名可稳定解析
防火墙放行	第 9 节端口按范围放行

检查项	通过标准
Docker Swarm 正常	`docker node ls` 三节点 `Ready`
Swarm 磁盘水位正常	三台 Swarm 节点 `/` 使用率低于 `70%`，Docker 日志和镜像缓存清理策略已配置
Tier0 企业版单副本正常	`docker service ls` 显示 `1/1`，并完成一次节点故障漂移验证
EMQX 状态确认	单节点固定模式确认持久化目录和备份；三节点集群模式确认集群健康
NFS 挂载确认	三台 Swarm 节点均可读写 `/mnt/tier0-shared`，重启后自动挂载
DB 主从正常	`repmgr cluster show` 主备和 witness 正常
DB VIP 正常	应用节点连接 `10.60.20.10:5432` 成功
APP VIP 正常	用户网段访问 `https://10.60.10.10` 成功
主库故障演练	备库可提升，DB VIP 可漂移
应用节点故障演练	APP VIP 和 Swarm 副本可恢复
备份恢复演练	可从备份恢复到测试库

实施顺序

1. 创建 6 台 VM，固定主机名、IP、时间同步和 SSH
2. 初始化系统基线、防火墙、磁盘挂载和内核参数
3. 如引入 NFS/NAS，先完成共享目录、权限、挂载和重启自恢复验证
4. 部署三节点 Docker Swarm
5. 部署 `APP VIP + HAProxy/Nginx`
6. 按单节点固定或三节点集群模式部署 EMQX
7. 部署 PostgreSQL + TimescaleDB 主备
8. 部署 repmgr witness、repmgrd 和 DB VIP 漂移脚本
9. 发布 Tier0 企业版单副本 stack
10. 执行连通性、NFS 读写、EMQX 恢复、数据库故障切换、备份恢复和巡检验证
11. 固化运行文档、账号凭据归档和日常巡检任务

风险与待确认项

- 现场真实网段、网关、DNS、NTP、证书和镜像仓库地址待确认
- Tier0 企业版是否需要本地文件持久化尚未确认；若需要，应补共享存储或对象存储方案
- NFS/NAS 是否由现场已有存储提供待确认；若 NFS 是单点，会成为共享文件功能单点
- Swarm 节点仅 `100G` 单系统盘，必须控制镜像数量、容器日志和临时文件；不适合承载长期业务数据
- `EMQX` 是有状态服务，需在实施前确认采用“单节点固定”还是“三节点集群”模式；单节点固定模式下消息层存在单点风险
- 数据库 RPO/RTO 未确认，当前方案按主从自动切换设计，不能承诺零数据丢失

- `DB VIP` 的漂移脚本需要在现场验证，特别是虚拟化平台是否允许 VRRP
- 如果现场网络不允许 VRRP，需要改为负载均衡器、DNS 切换或虚拟化平台浮动 IP

标准私有化部署方案

方案目标

简化版不使用 Docker Swarm，只部署单台 `enterprise` 应用节点，总计 `4` 台虚拟机：

- `1` 台 `enterprise` 节点承载 `tier0-enterprise + EMQX`
- `2` 台数据库节点组成 `PostgreSQL + TimescaleDB` 主从集群
- `1` 台 `Witness` 节点参与数据库监督、仲裁和故障切换

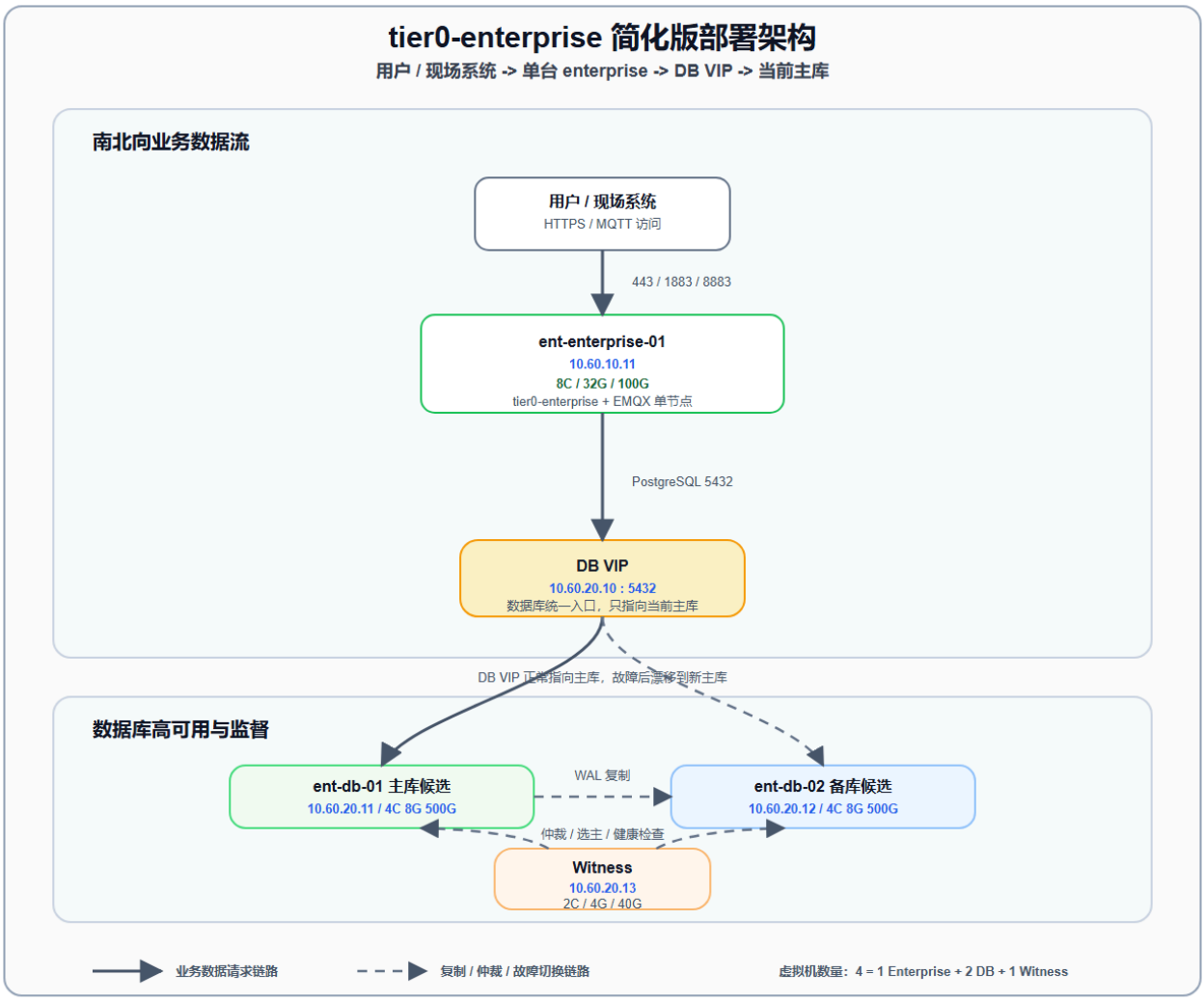
简化版目标是降低部署复杂度。代价是应用层和 EMQX 均为单点，只有数据库层保留 `DB VIP + Witness` 高可用。

规划 IP

以下 IP 为规划占位，实施前必须替换为现场真实网段。

类型	名称	规划 IP	说明
应用节点	`ent-enterprise-01`	`10.60.10.11`	单台 enterprise，承载 `tier0-enterprise + EMQX`
逻辑入口	`DB VIP`	`10.60.20.10`	数据库统一入口，只指向当前主库
数据库节点	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB 主库候选
数据库节点	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB 备库候选
仲裁节点	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / 仲裁监督节点

总体拓扑



核心链路：

- 业务访问路径：用户 / 现场系统 -> `ent-enterprise-01` -> `DB VIP` -> 当前主库
- `tier0-enterprise` 和 `EMQX` 均部署在 `ent-enterprise-01`
- 数据库仍通过 `DB VIP` 访问，不直连 `ent-db-01/02`
- `Witness` 只参与数据库仲裁、选主和健康检查，不进入业务访问链路

数据库监督与故障切换

阶段	链路	说明
正常访问	`ent-enterprise-01 -> DB VIP -> 当前主库`	应用始终只连接 `10.60.20.10:5432`
正常复制	`当前主库 -> 当前备库`	PostgreSQL WAL 流复制
正常监督	`DB-01/DB-02 -> Witness`	repmgrd 上报状态，Witness 参与仲裁

阶段	链路	说明
主库故障	`备库 -> Witness`	备库请求仲裁，避免脑裂
备库提升	`备库 promote`	Witness 确认后，备库提升为新主库
VIP 漂移	`DB VIP -> 新主库`	`10.60.20.10` 绑定到新主库
应用恢复	`ent-enterprise-01 -> DB VIP -> 新主库`	应用连接池重连后恢复读写

节点配置清单

节点	角色	vCPU	内存	存储	系统盘建议	数据盘建议	主要组件
`ent-enterprise-01`	单节点 enterprise	`8`	`32G`	`100G`	单块系统盘 `/` 100G	不单独拆分	tier0-enterprise、EMQX、Nginx/HAProxy 可选
`ent-db-01`	PostgreSQL/TSDB 主库候选	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-02`	PostgreSQL/TSDB 备库候选	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-witness-01`	Witness/仲裁监督	`2`	`4G`	`40G`	`/` 40G	不单独规划	repmgr witness、监控 agent、审计日志

配置汇总清单

类别	数量	单台规格	小计 vCPU	小计内存	小计存储
enterprise 应用节点	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB 节点	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness 节点	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
合计	`4`	-	`18C`	`52G`	`1140G`

换算：

- 总虚拟机：`4` 台
- 总 vCPU：`18`

- 总内存：`52G`
- 总存储：`1.14T`

连接方式清单

运维访问

目标	方式	端口	说明
所有虚拟机	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	统一免密实施后使用；未实施前以现场账号为准
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	业务入口
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	只允许 enterprise 节点和运维白名单访问

应用与 EMQX 端口

源	目标	端口	用途
用户 / 现场系统	`ent-enterprise-01`	`80/TCP`	HTTP 入口或跳转 HTTPS，可选
用户 / 现场系统	`ent-enterprise-01`	`443/TCP`	HTTPS 业务入口
设备 / 现场系统	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT 明文，可选
设备 / 现场系统	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS，建议
设备 / 现场系统	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket，可选
设备 / 现场系统	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS，可选
运维网段	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard，仅允许运维白名单
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL 业务访问

数据库与监督通信

源	目标	端口	用途
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL 流复制
`ent-db-02`	`ent-db-01`	`5432/TCP`	故障恢复后反向复制
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness 通信

源	目标	端口	用途
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP 漂移，需允许协议 `112`

服务部署建议

enterprise 应用节点

- 不部署 Docker Swarm
- `tier0-enterprise` 和 `EMQX` 部署在同一台 `ent-enterprise-01`
- `ent-enterprise-01` 是应用层单点，故障时需要人工恢复或从备份重建
- 节点只有 `100G` 单系统盘，不适合承载长期业务数据
- `tier0-enterprise` 业务数据必须进入 PostgreSQL/TimescaleDB
- 如有上传、导入导出、报表等文件，可保存在本机并纳入备份；如果后续要扩展为标准版，应迁移到 NFS/NAS 或对象存储

EMQX 单节点

- EMQX 在简化版中为单节点有状态服务
- 数据目录固定在 `ent-enterprise-01` 本地盘，例如 `/data/emqx`
- 配置、证书、插件、规则引擎配置需要纳入备份
- 不提供消息层高可用；如不能接受 EMQX 单点，应使用标准版三节点 EMQX 集群模式

PostgreSQL + TimescaleDB

- PostgreSQL：`16` 或现场认证版本
- TimescaleDB：选择与 PostgreSQL 主版本匹配的稳定版本
- 主从复制：物理流复制
- 故障切换：`repmgrd + witness`
- 数据库入口：`DB VIP`
- 连接池：可在 `ent-enterprise-01` 部署 PgBouncer，避免数据库连接数过高

Witness

- Witness 只放 `repmgr witness` 元数据和监督逻辑
- Witness 不承载业务 SQL
- Witness 故障不影响当前主库继续服务，但会降低自动切换可信度

端口与防火墙基线

端口/协议	放行范围	用途
`22/TCP`	运维网段到 4 台 VM	SSH
`80/TCP`	用户网段到 `ent-enterprise-01`	HTTP 跳转或内网明文入口
`443/TCP`	用户网段到 `ent-enterprise-01`	HTTPS 业务入口

端口/协议	放行范围	用途
`1883/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX MQTT 明文，可选
`8883/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX MQTT TLS，建议
`8083/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX WebSocket，可选
`8084/TCP`	设备 / 现场系统到 `ent-enterprise-01`	EMQX WebSocket TLS，可选
`18083/TCP`	运维网段到 `ent-enterprise-01`	EMQX Dashboard，仅白名单
`5432/TCP`	enterprise 到 `DB VIP`；DB/Witness 内部互通	PostgreSQL / repmgr
`112/VRRP`	DB 节点间	DB VIP 漂移
`9100/TCP`	监控系统到 4 台 VM	node exporter，可选

高可用能力与故障边界

故障场景	预期行为	业务影响
`ent-enterprise-01` 故障	应用和 EMQX 均中断，需要人工恢复或从备份重建	应用层整体中断
当前数据库主库故障	备库经 Witness 仲裁后提升，DB VIP 漂移	秒级到分钟级写入中断，应用需重连
数据库备库故障	主库继续服务，复制中断告警	无直接业务中断，但失去数据库热备
Witness 故障	当前主库继续服务，自动切换能力下降	无直接业务中断，需优先恢复
DB VIP 漂移失败	数据库实际已切主但入口不可达	业务中断，需手动绑定 VIP

上线前检查清单

检查项	通过标准
4 台 VM 主机名固定	`hostnamectl` 与规划一致
统一 SSH 可用	运维终端可免密访问 4 台 VM
时间同步	4 台 VM 时间偏差小于 `1s`
防火墙放行	第 9 节端口按范围放行

检查项	通过标准
enterprise 服务正常	`tier0-enterprise` 健康检查通过
EMQX 状态确认	EMQX 节点、监听端口和持久化目录正常
enterprise 磁盘水位正常	`/` 使用率低于 `70%`，日志和缓存清理策略已配置
DB 主从正常	`repmgr cluster show` 主备和 witness 正常
DB VIP 正常	enterprise 节点连接 `10.60.20.10:5432` 成功
主库故障演练	备库可提升，DB VIP 可漂移
备份恢复演练	可从备份恢复到测试库

实施顺序

1. 创建 4 台 VM，固定主机名、IP、时间同步和 SSH
2. 初始化系统基线、防火墙、磁盘挂载和内核参数
3. 在 `ent-enterprise-01` 部署 `tier0-enterprise`
4. 在 `ent-enterprise-01` 部署单节点 EMQX
5. 部署 PostgreSQL + TimescaleDB 主备
6. 部署 repmgr witness、repmgrd 和 DB VIP 漂移脚本
7. 执行连通性、EMQX 恢复、数据库故障切换、备份恢复和巡检验证
8. 固化运行文档、账号凭据归档和日常巡检任务

风险与待确认项

- 简化版应用层是单点，`ent-enterprise-01` 故障会导致业务整体不可用
- 简化版 EMQX 是单点，不能提供消息层高可用
- `ent-enterprise-01` 只有 `100G` 单系统盘，必须控制日志、缓存、附件和临时文件
- 如后续需要应用层或 EMQX 高可用，应切换到标准版
- 数据库 RPO/RTO 未确认，当前方案按主从自动切换设计，不能承诺零数据丢失
- `DB VIP` 漂移脚本需要在现场验证，特别是虚拟化平台是否允许 VRRP

登录

1. 在浏览器中使用部署好的域名和端口访问 Tier0。
2. 在登录页面，使用许可证激活 Tier0。

- **在线激活**：输入许可证密钥，然后单击 **立即激活。**



- **离线激活**：将设备标识发送给供应商以获取 JSON 文件，然后导入该文件。



3. 激活后，输入默认用户名和密码（tier0/tier0）。

4. 单击 **登录**。

操作指南

连接数据

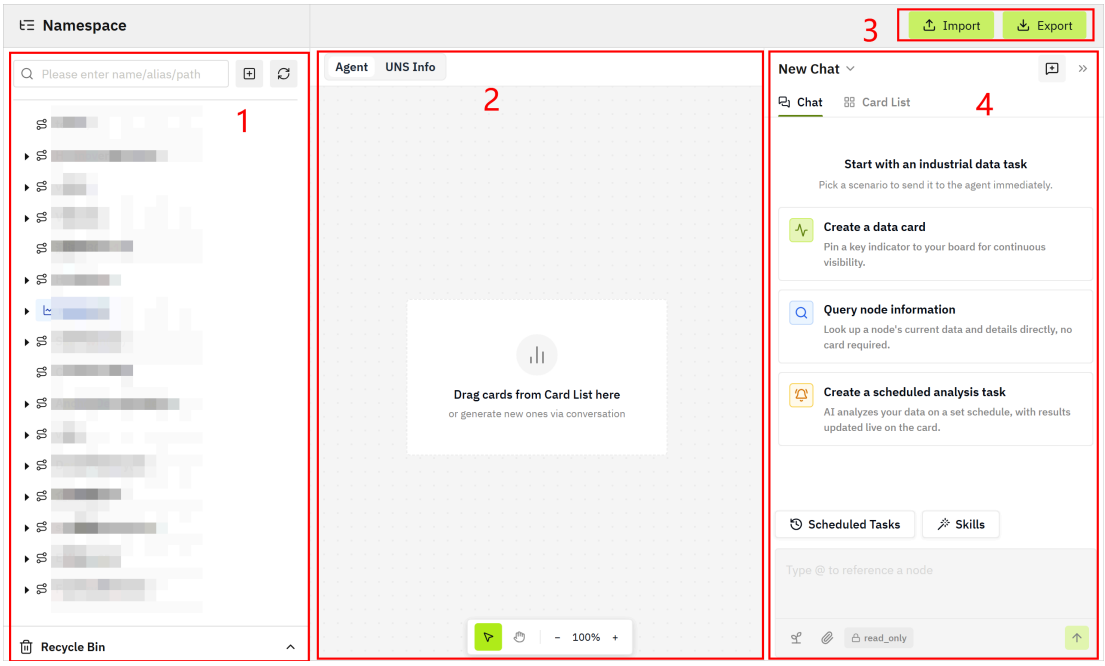
建立数据模型

Tier0 基于统一命名空间（UNS）将数据组织成清晰的树形结构。在这棵树中，每个主题都会自动生成对应的 MQTT 主题，便于获取实时数据。

开始前须知

页面概览

登录 Tier0 后，选择**统一命名空间 > UNS**，进入该模块的主页面。



序号	区域	说明
1	命名空间	显示数据模型列表，可新增模型、刷新列表，以及从回收站恢复模型。
2	画布	显示选中的 UNS 信息，或右侧对话生成的卡片。
3	导入/导出	通过 JSON 文本或文件导入、导出命名空间模型。
4	对话	使用自然语言与 UNS 主题交互，并将对话生成的卡片拖放到画布上展示。

补充参数说明

- **路径-更多设置**

项目	说明
扩展属性	为路径添加扩展属性，例如单位。

- **主题-主题类型**

类型	说明
状态	作为 API 接收系统状态数据。
操作	作为 API 执行增删改查（CRUD）命令。
指标	实时数据，例如设备温度。

- **主题-属性生成方式**

属性生成方式	说明
自定义	自定义属性。
自动解析	通过 JSON 文本生成相同结构的属性。

- **主题-数据质量**

数据质量	说明
数值范围检查	检查属性值是否处于指定范围内。
采集中断检查	根据设置的采集间隔和延迟时间，检查该主题的数据采集是否中断。

- **主题-可选行为**

项目	说明
模拟数据	在**数据连接**中为该主题自动创建数据流，用于发送生成的模拟数据。
启用历史数据	为该主题开启历史数据存储功能。

如何创建数据模型

基于简单的路径和主题结构，可将数据层级定义为树形视图。

手动创建模型

1. 登录 Tier0，选择**统一命名空间** > **UNS**。
2. 单击  > **新建路径**，添加路径（例如 `factory`）。



3. 填写路径信息，按需设置别名，然后单击**保存**。
4. 选中 **factory**，单击  > **新建主题**，在其下添加主题（例如 `CNC`）。

新建Topic

命名空间

* 名称

文件类型 状态 操作 指标

属性生成方式 自定义 自动解析

键

名称	类型	单位	
_quality	LONG	单位	
_timestamp	DATET...	单位	

+

更多设置

* 别名 __e07bc0a87163458fb6e9

显示名称

文件描述

扩展属性 +

数据质量

值范围检测

字段	下限	上限	单位
选择数值字段	0 或留空	0 或留空	可选

+ 添加字段

采集中断检测

预期采集间隔 输入秒数 秒 允许延迟 输入秒数 秒

预期采集间隔大于 0 时自动启用；清空或填 0 即停止检测

可选行为

模拟数据 启用历史数据

保存

5. 填写主题信息，然后单击**保存**。

导入模型

1. 登录 Tier0，进入 **UNS** 页面。
2. 单击右上角的**导入**。
3. 导入 JSON 创建模型：

- 在 **UNS** 选项卡中直接输入 JSON。

> **如何快速获取模型 JSON**
>
> 单击**复制给 Agent**，将复制的内容发送给智能体，再将其生成的结果粘贴回来。

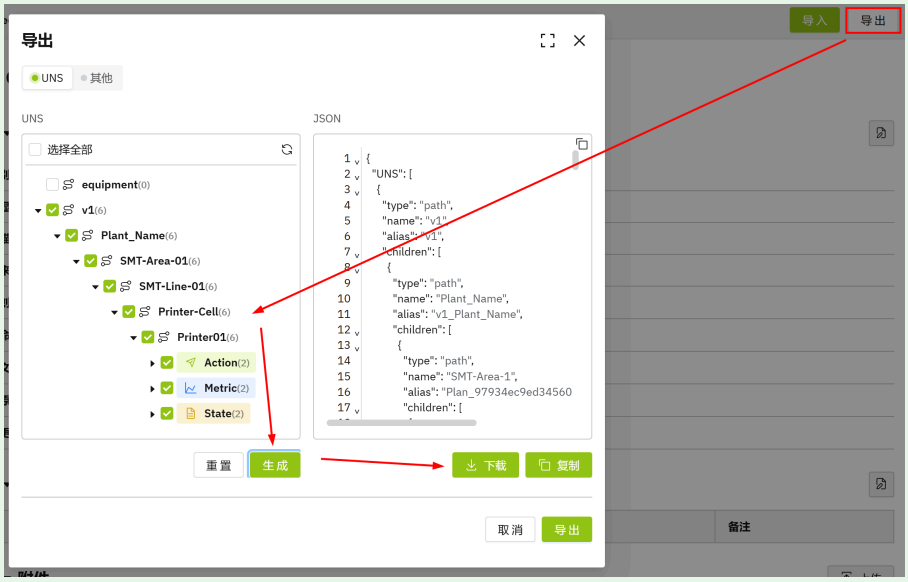


- 在 **UNS** 选项卡中单击**下载模板**，填写模型内容后，在**全部**选项卡上传文件。

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION_
  "Label": [{"name": "debug"},{"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
```


提示

也可以手动添加路径和主题，再将其导出，作为导入模型的示例。

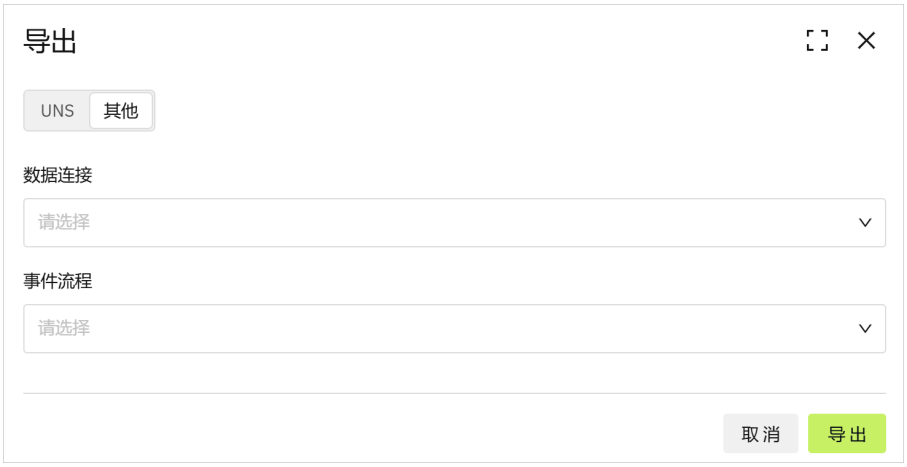


4. 单击**保存**。

导出数据

Tier0 支持导出 UNS 模型及其他数据，例如数据连接、事件流程和数据看板。

1. 在 **UNS** 页面，单击右上角的**导出**。
2. 单击**其他**选项卡，选择对应的数据类型。



3. 单击**导出**。

通过数据流连接数据

如何通过数据连接添加数据源

在 **UNS** 中构建数据模型后，需要将真实数据连接到模型，用于后续处理和使用。

手动添加数据流

1. 登录 Tier0，选择 **流程**。



编号	项目	说明
1	数据流列表	选择查看全部数据流，或仅查看某一分类下的流。
2	数据流信息	显示数据流基础信息。单击流名称可开始编辑数据流。
3	搜索/筛选	搜索或筛选已有数据流。
	新建文件夹	添加文件夹，用于对数据流进行分类。
	新建流程	添加新的数据流。

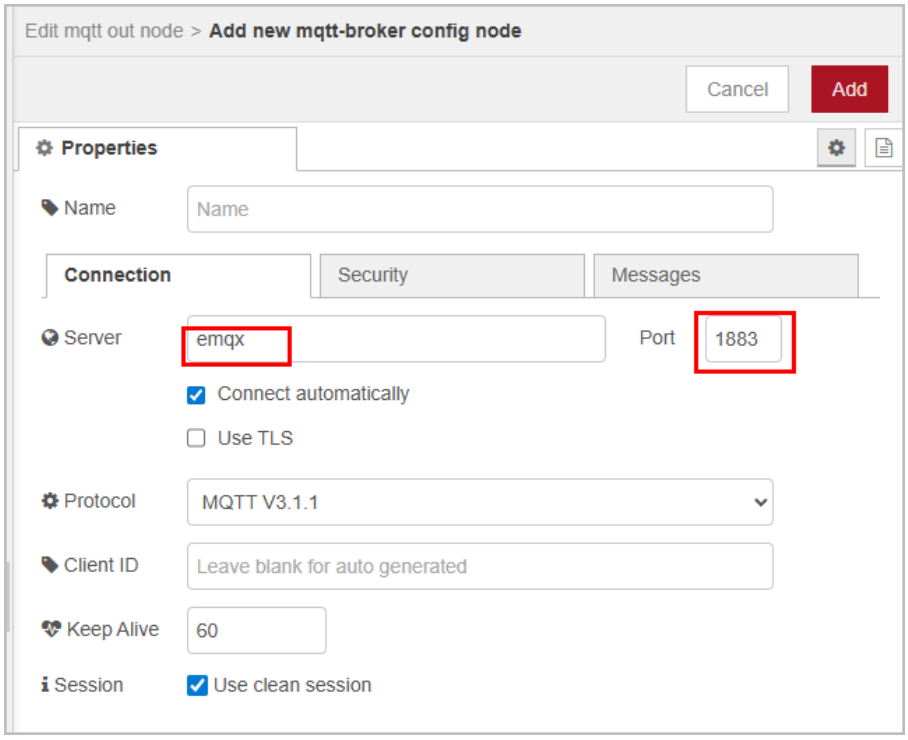
2. 单击 **新建流程**，在弹窗中输入名称和说明，将 **流程类型** 选择为 **数据连接**，然后单击 **保存**。

- **数据连接**：用于从数据源采集数据，并发送到 UNS。
- **事件流程**：用于处理来自 UNS 的数据。



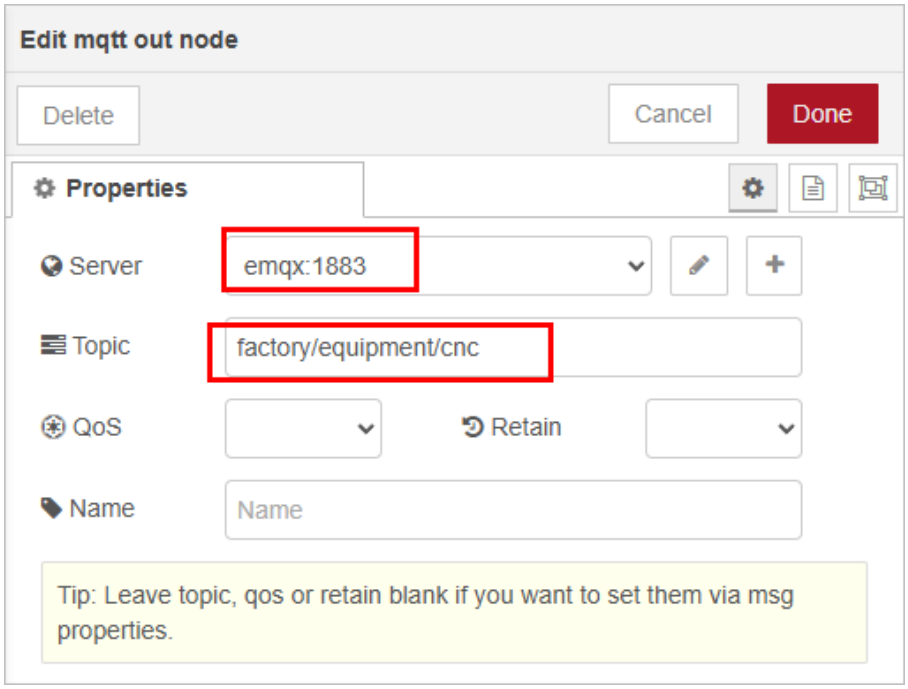
3. 单击刚添加的流程，进入配置页面，并根据数据源类型拖入对应节点，例如 `modbus`。
4. 配置数据源节点，然后拖入 `mqtt out` 节点，并完成相关配置。

   i. 添加内部 Broker：`emqx:1883`。



   ii. 将其订阅 Topic 设置为 **Namespace** 中的数据模型。

   例如，数据模型结构为 `factory/equipment/cnc`，则 Topic 为 `factory/equipment/cnc`。



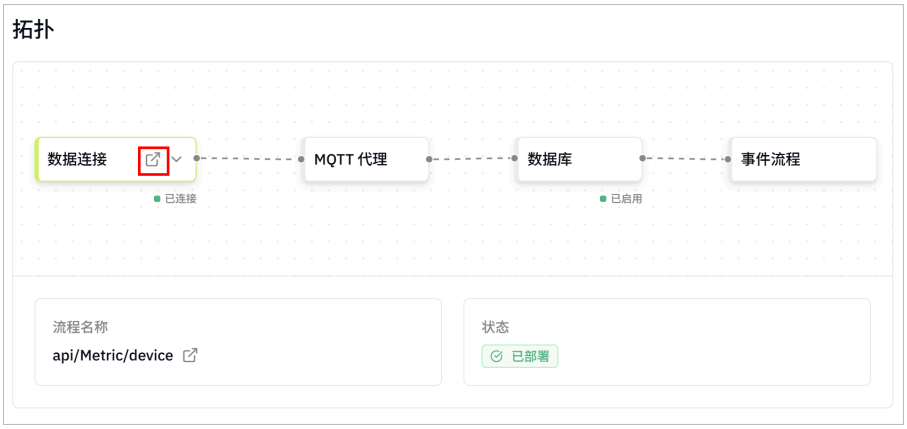
5. 单击右上角的 **Deploy**，触发流程后，数据会发送到 **命名空间** 下对应的模型中。

> **提示**
>
> 你可以单击 **Save Version** 将流程保存为不同版本，并在需要时切换到历史版本。

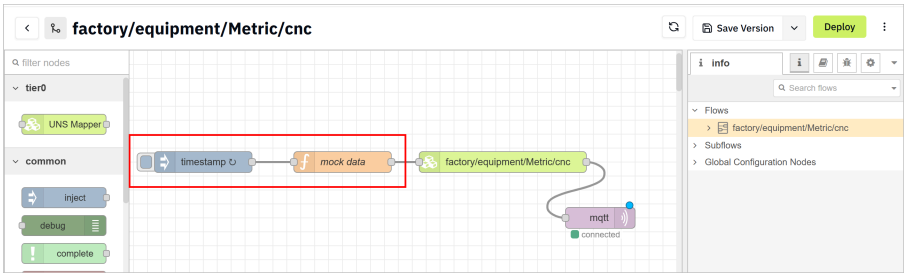
通过生成的流程配置数据源

> **备注**
>
> 只有在创建文件时启用了 **模拟数据**，系统才会自动生成数据流。

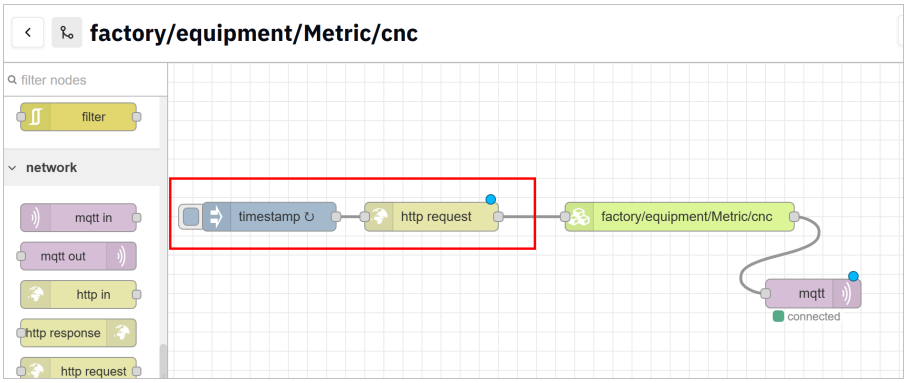
1. 登录 Tier0，进入 **UNS**。
2. 查看已创建文件的详情，向下滚动到 **拓扑**，然后单击 **数据连接** 图标进入生成的流程。



3. 删除流程中的模拟数据源。



4. 将真实数据源节点连接到流程，并完成配置。



5. 单击右上角的 **Deploy**，然后触发流程。
6. 返回 **UNS**，检查是否已接收到数据。

负载

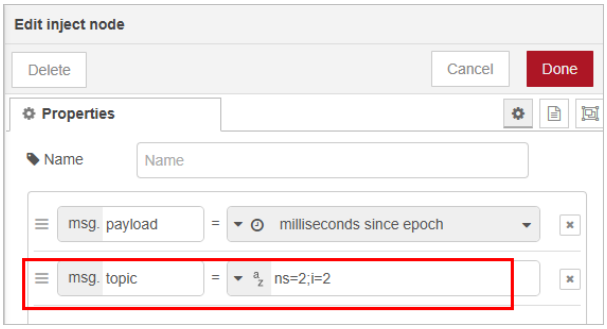
```
{
  "temp": 33,
  "humidity": 45
}
```

常规数据源

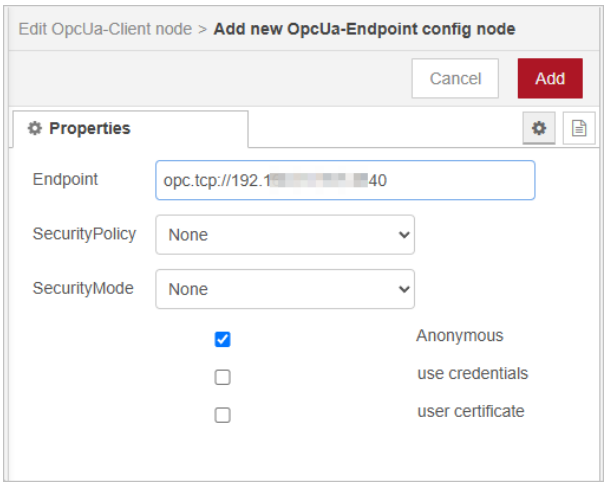
连接 OPC UA

从OPCUA服务器读取数据

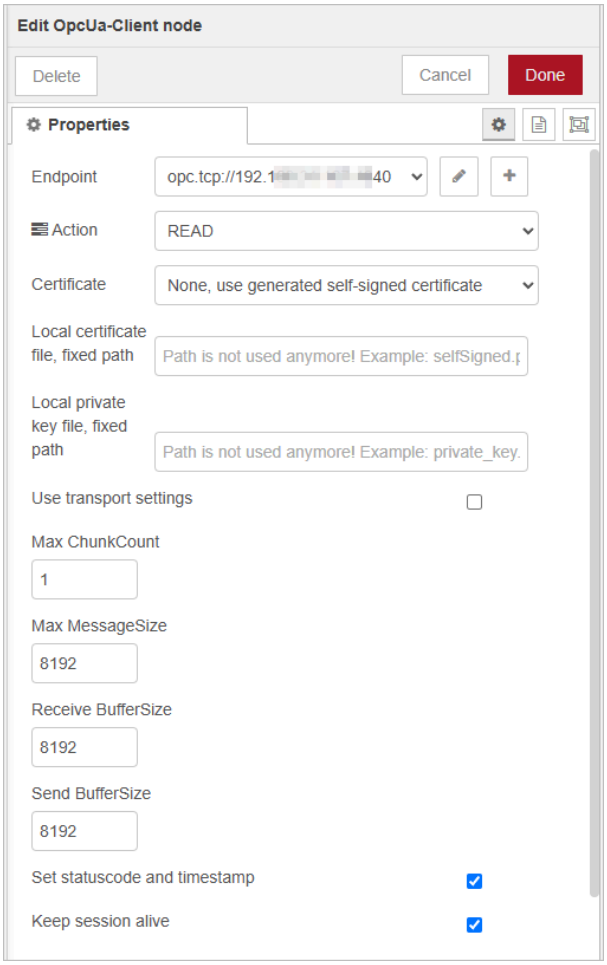
1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 双击该节点，输入 OPC UA 服务器中的 **节点ID**。



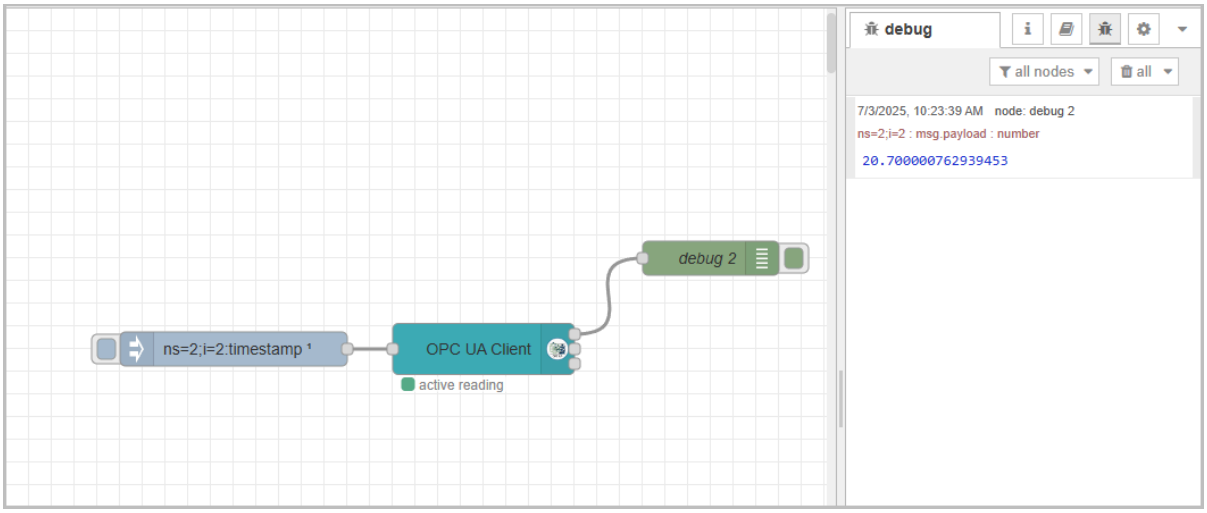
3. 拖拽一个 `OpcUa-Client` 节点，然后双击该节点。
4. 单击 **Endpoint** 右侧的 ➕ 图标，输入你的 OPC UA 服务器信息。



5. 单击 **Add**，将 **Action** 设置为 **READ**。
6. 其他参数保持默认配置，单击 **Done**。



7. 将 `inject` 节点与 `OpcUa-Client` 节点连接，最后在末端连接一个 `debug` 节点。
8. 单击右上角 **Deploy**，然后单击 `inject` 节点触发流程。



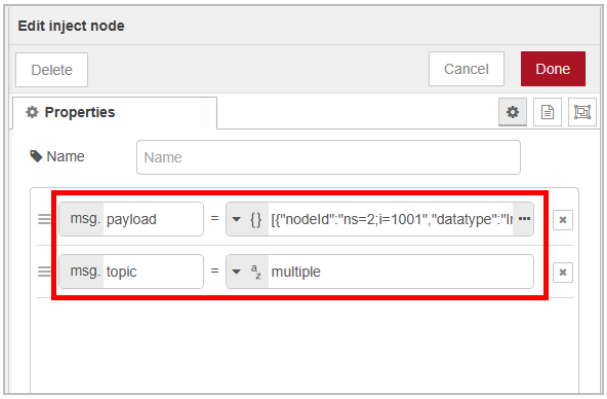
从OPCUA服务器订阅数据

1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 双击该节点，输入 OPC UA 服务器中的 **节点ID**。

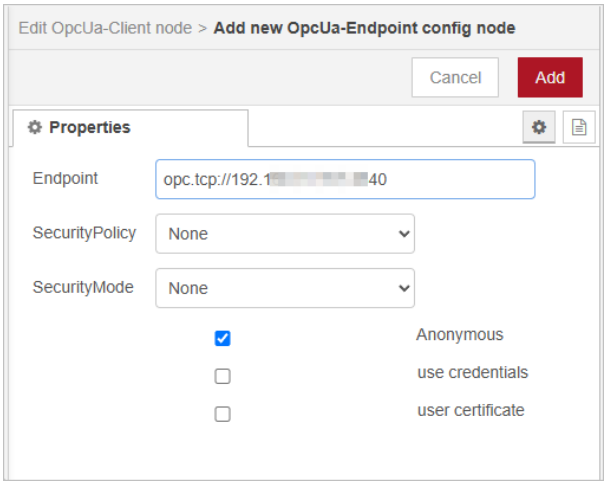
- msg.payload:

```
[
    {
        "nodeId": "ns=2;i=1001",
        "datatype": "Int32"
    },
    {
        "nodeId": "ns=2;i=1002",
        "datatype": "Int32"
    }
]
```

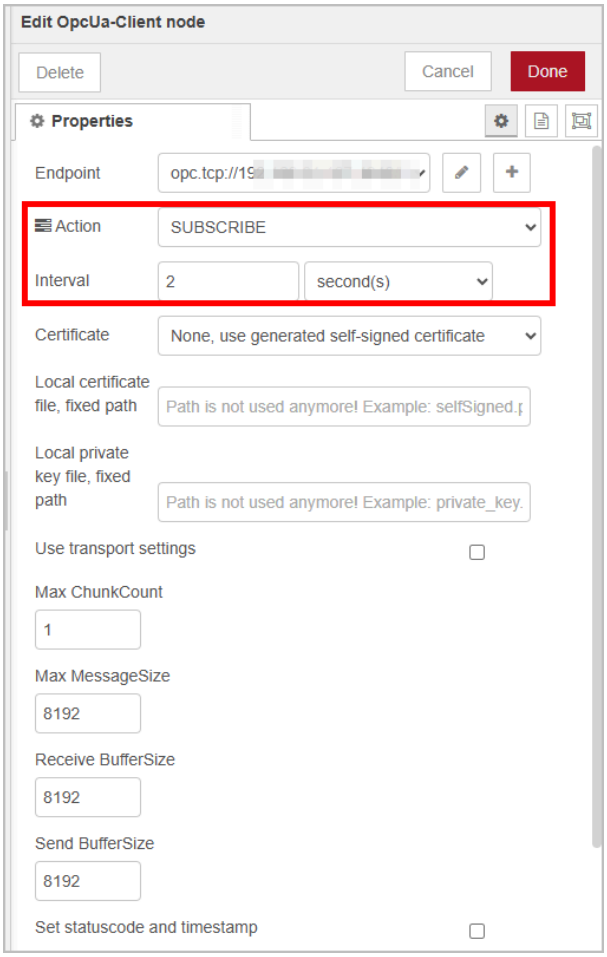
- msg.topic: multiple



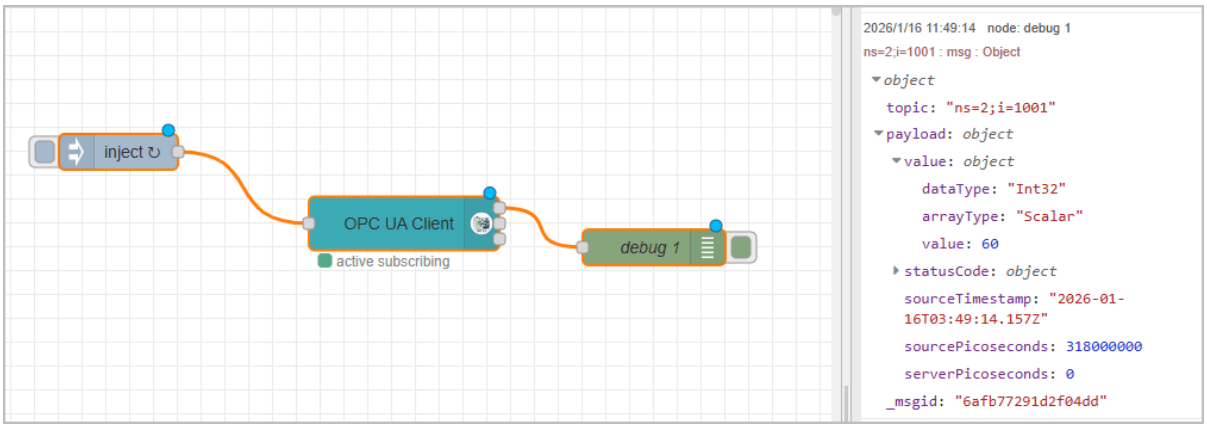
3. 拖拽一个 `OpcUa-Client` 节点，然后双击该节点。
4. 单击 **Endpoint** 右侧的 ✚ 图标，输入你的 OPC UA 服务器信息。



5. 单击 **Add**，将 **Action** 设置为 **SUBSCRIBE**。
6. 设置订阅间隔，其他参数保持默认配置，单击 **Done**。

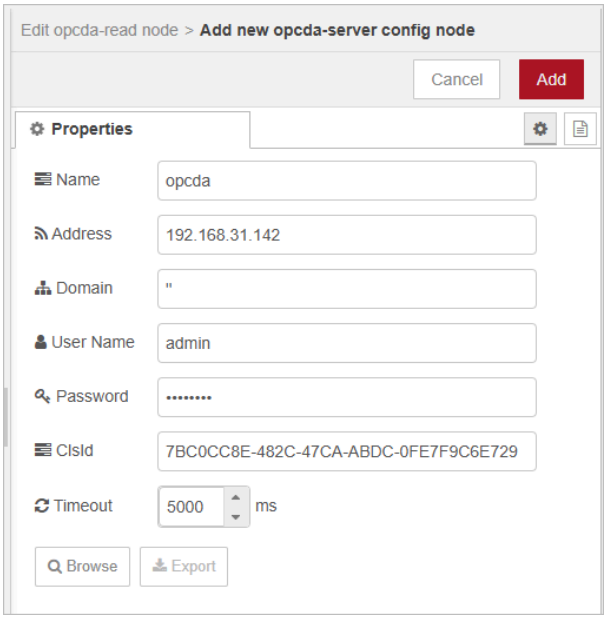


7. 将 `inject` 节点与 `OpcUa-Client` 节点连接，最后在末端连接一个 `debug` 节点。
8. 单击右上角 **Deploy**，然后单击 `inject` 节点触发流程。



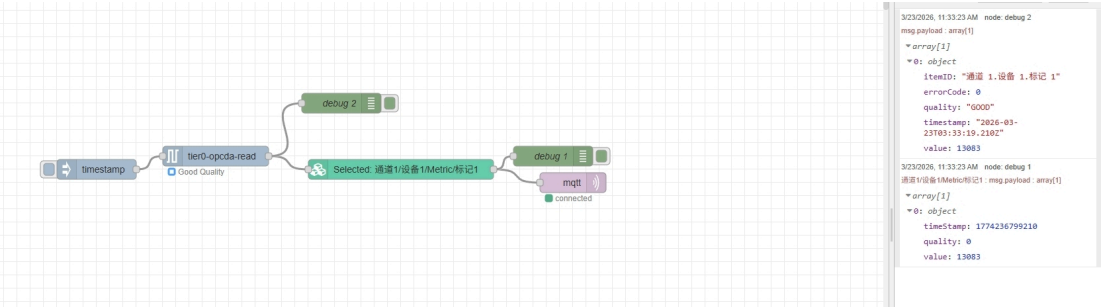
连接 OPC DA

1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `opcda-read` 节点，然后双击该节点。
3. 单击 **服务器** 右侧的 ➕ ，输入你的 OPC DA 服务器信息。



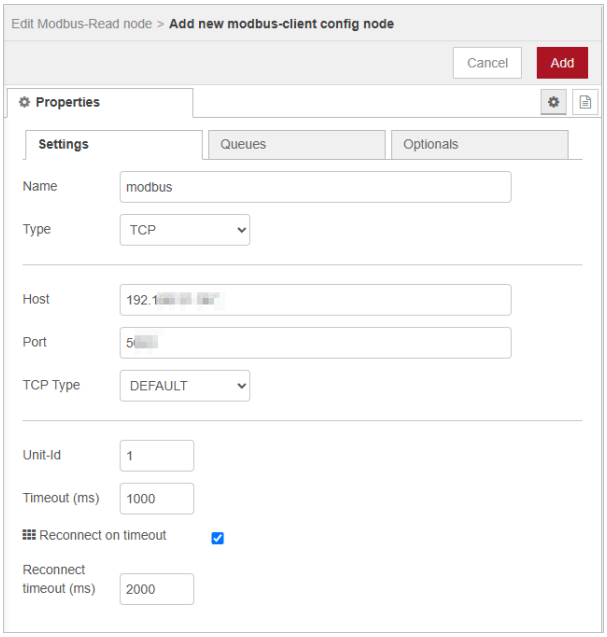
参数	说明
地址(Address)	OPC DA 服务器的 IP 地址。
域名(Domain)	OPC DA 服务器的域名。
用户名/密码(Username/Password)	登录 DA 服务器所用的用户名和密码。
应用标识(Clsid)	所使用的 DA 连接服务的应用 ID（例如 KepServer）。

5. 单击 **Browse** 查看已连接的通道。
6. 单击 **Add**，在 **Items** 下添加所需的项目。
7. 将 `inject` 节点与 `opcda-read` 节点连接，最后在末端连接一个 `debug` 节点。
8. 单击右上角 **Deploy**，然后单击 `inject` 节点触发流程。

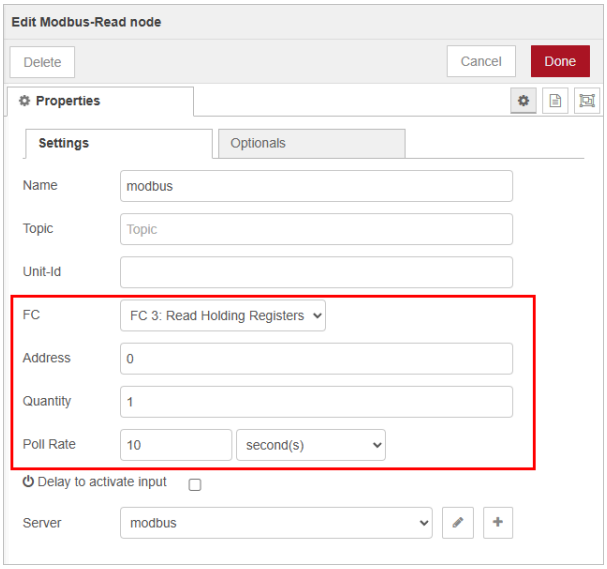


连接 Modbus

1. 在 **数据连接** 中，拖拽一个 `Modbus-Read` 节点。
2. 双击该节点，单击 **Server** 右侧的 ➕ ，输入你的 Modbus 服务器信息。



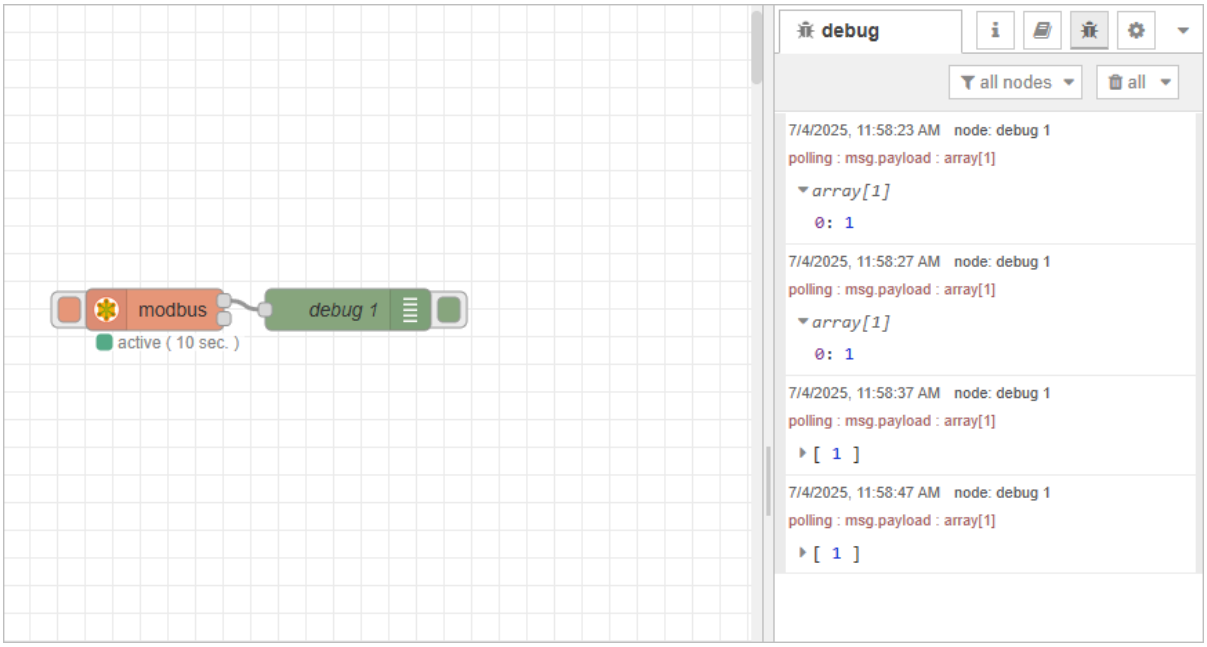
3. 单击 **Add**，输入待连接数据的相关信息。



参数	说明
名称（Name）	节点的显示名称（仅用于可视化识别）。
主题（Topic）	可选参数，将传递至输出消息的主题字符串。
从站地址（Unit-Id）	目标设备的 Modbus 从站 ID（通常为 1–247）。
功能码（FC）	指定 Modbus 操作的功能码（例如：功能码 3 = 读取保持寄存器）。
起始地址（Address）	读取的起始寄存器地址（通常为零基地址）。
读取数量（Quantity）	连续读取的寄存器个数。
轮询频率（Poll Rate）	节点轮询 Modbus 设备的间隔时间（例如：每 10 秒一次）。

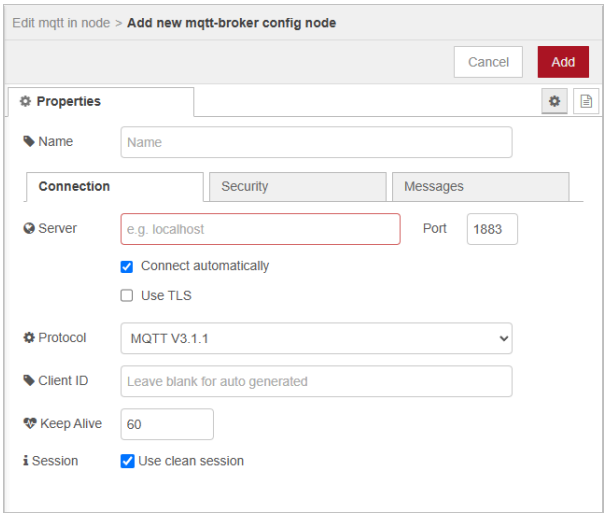
输入激活延迟（Delay to activate input）	可选参数，轮询输入信号激活前的延迟时间。
服务器（Server）	已配置的 Modbus 服务器引用（包含 IP、端口、协议等信息）。

4. 单击 **Done**。
5. 在末端连接一个 `debug` 节点。
6. 单击右上角 **Deploy**，然后单击 `Modbus-Read` 节点触发流程。



连接 MQTT

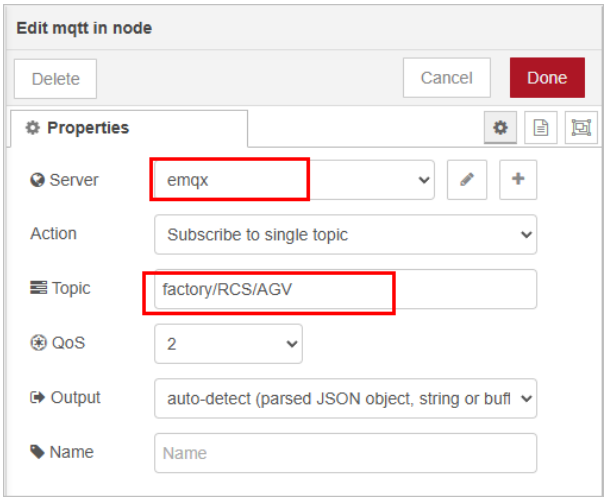
1. 在 **数据连接** 中，拖拽一个 `mqtt in` 节点。
2. 双击该节点，单击 **Server** 右侧的 ✚ 。



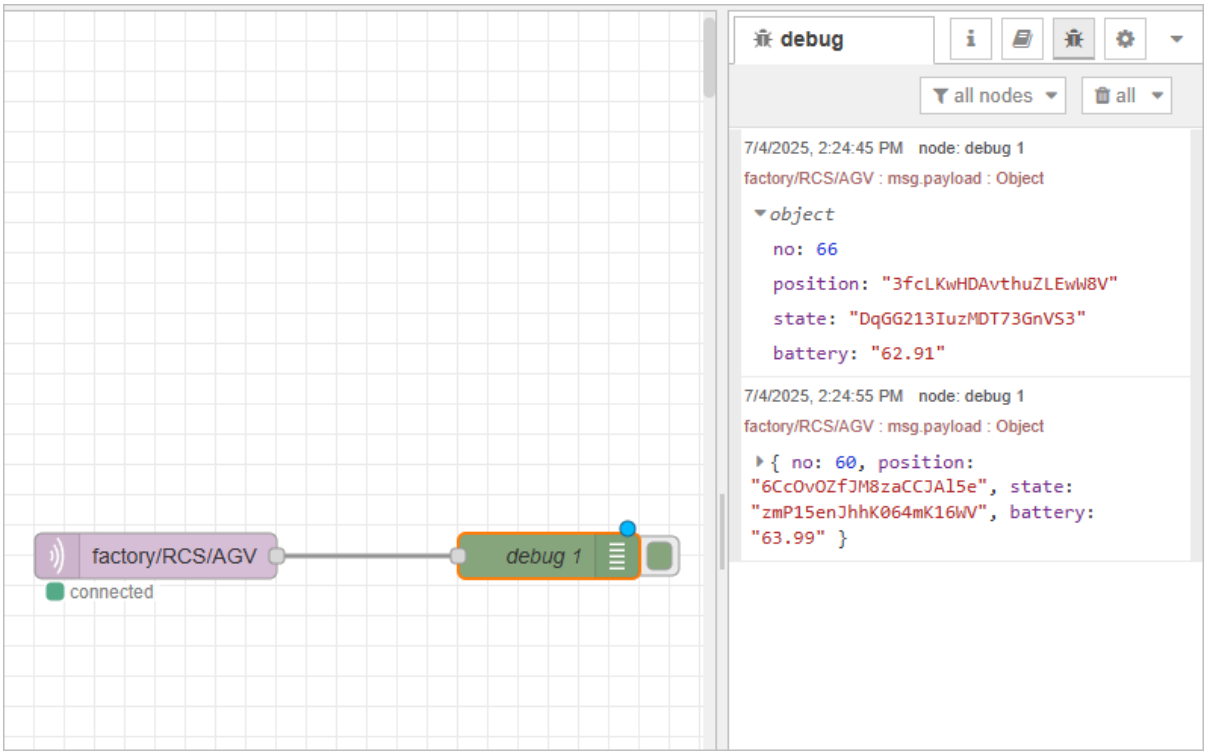
参数（Parameter）	说明（Description）

名称（Name）	MQTT 代理配置的可选名称（仅用于内部参考）。
服务器（Server）	MQTT 代理的主机名或 IP 地址（例如：`localhost` 或 `broker.hivemq.com`）。
端口（Port）	用于连接 MQTT 代理的端口（非 TLS 连接默认值为 `1883`，TLS 连接默认值为 `8883`）。
自动连接（Connect automatically）	启用后，Node-RED 会在流程启动时自动尝试连接代理。
使用 TLS（Use TLS）	若代理要求安全的 TLS/SSL 连接，需启用此选项。
协议（Protocol）	用于连接的 MQTT 协议版本（例如：MQTT v3.1.1 或 v5.0）。
客户端 ID（Client ID）	可选的客户端 ID。若留空，系统会自动生成唯一 ID，且每个客户端的 ID 必须唯一。
保活时间（Keep Alive）	与代理通信的最大间隔时间（单位：秒），用于检测断开的连接。
使用干净会话（Use clean session）	启用后，不会与代理保持持久会话；未启用时，会话状态（如订阅信息）会被保留。
用户名（Username）（安全标签页）	若需要，填写用于 MQTT 代理身份验证的用户名。
密码（Password）（安全标签页）	与用户名对应的代理身份验证密码，会被安全存储。

3. 单击 **Add**，输入需要订阅的主题。
4. 单击 **Done**。



5. 在末端连接一个 `debug` 节点。
6. 单击右上角 **部署（Deploy）**，当该主题接收到数据时，流程会被触发。



连接文件

> **信息**
>
> 在**数据连接**中处理文件前，需将文件放入对应的容器中。

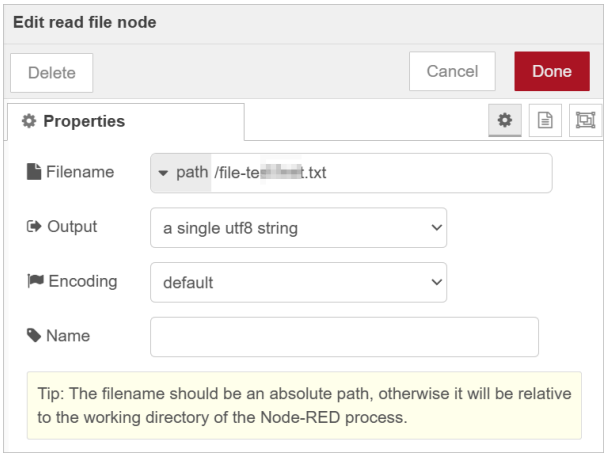
```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/
docker cp /root/config.json <容器名称或ID>:/app/config.json
```

> **常用DOCKER命令**

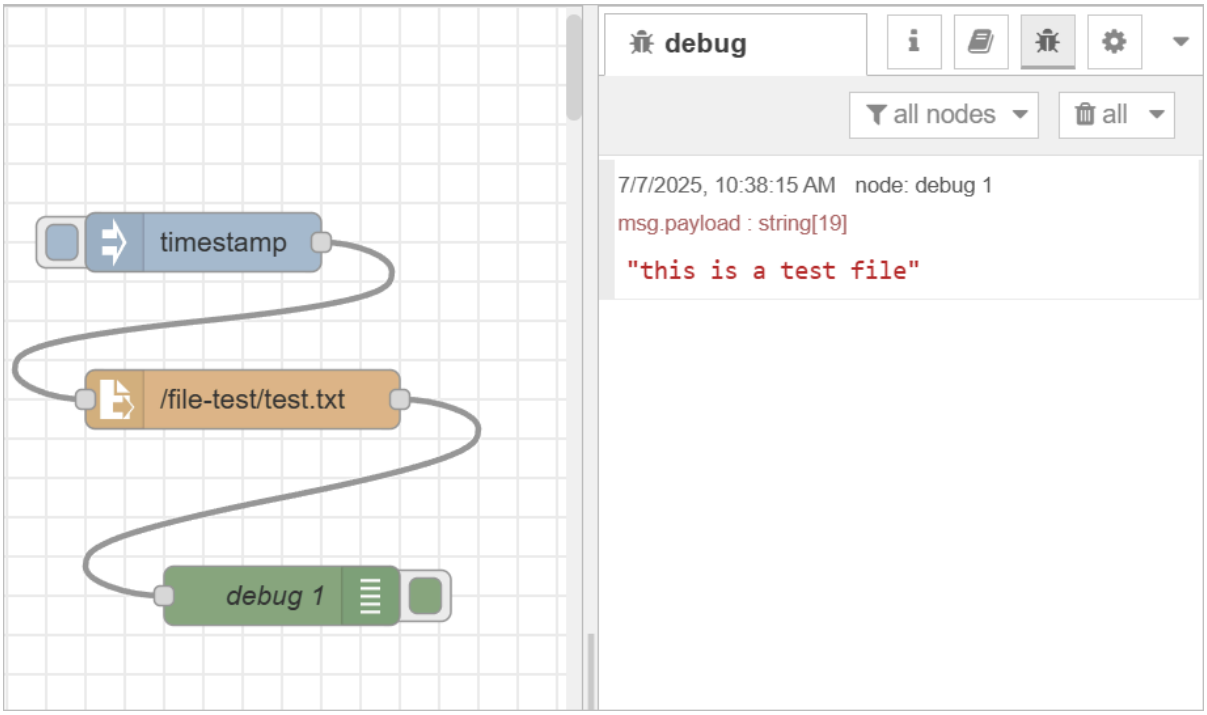
命令	说明
`docker ps`	列出所有正在运行的Docker容器。
`docker ps | grep [container]`	根据指定的容器信息筛选正在运行的容器。
`docker exec -it [container] /bin/sh`	通过 `/bin/sh` 在运行中的容器内启动一个shell会话。
`mkdir [directory]`	在容器内创建一个指定名称的新目录。

连接文本文件

1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `read file` 节点，然后双击该节点。
3. 输入文件路径和输出配置。

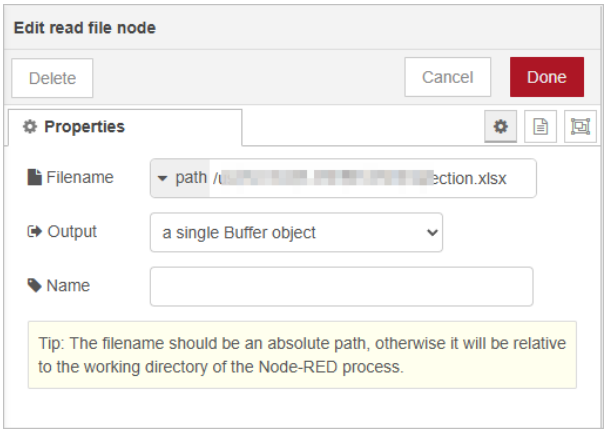


4. 单击 **完成**，然后在末端连接一个 `debug` 节点。
5. 单击右上角 **部署**，接着单击 `inject` 节点触发流程。



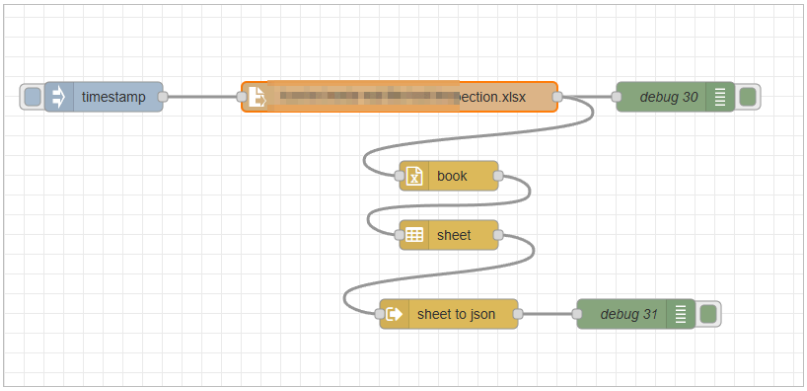
连接Excel文件

1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `read file` 节点，然后双击该节点。
3. 输入文件路径和输出配置。

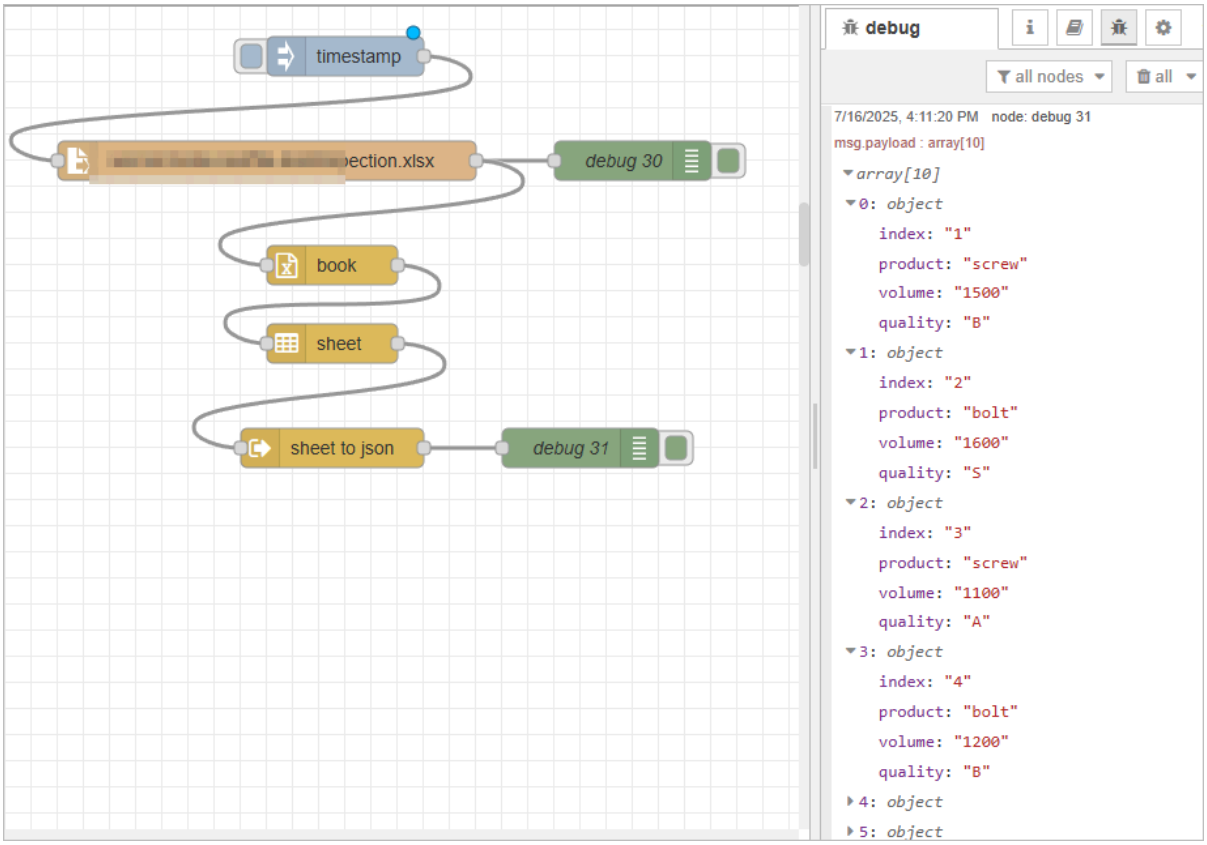


4. 单击 **Done**，然后按顺序连接 `book`、`sheet` 和 `sheet to json` 节点，将表格数据转换为JSON格式。

> **信息**
>
> 安装 **node-red-contrib-spreadsheet-in** 节点以获取转换所需的节点。



5. 单击右上角 **Deploy**，接着单击 `inject` 节点触发流程.



提示

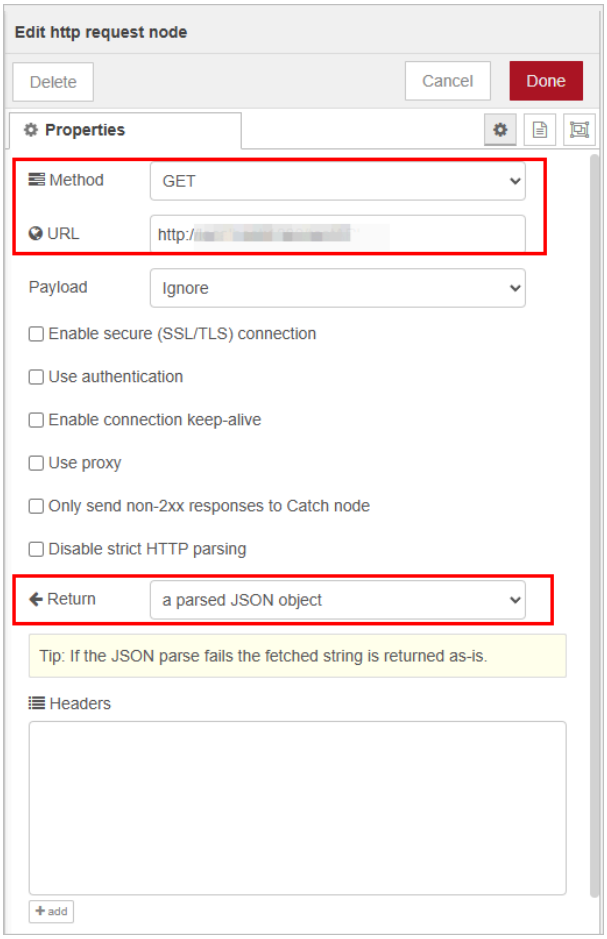
也可尝试其他节点处理文件数据，如node-red-contrib-xlsx-to-json.

连接 RestAPI

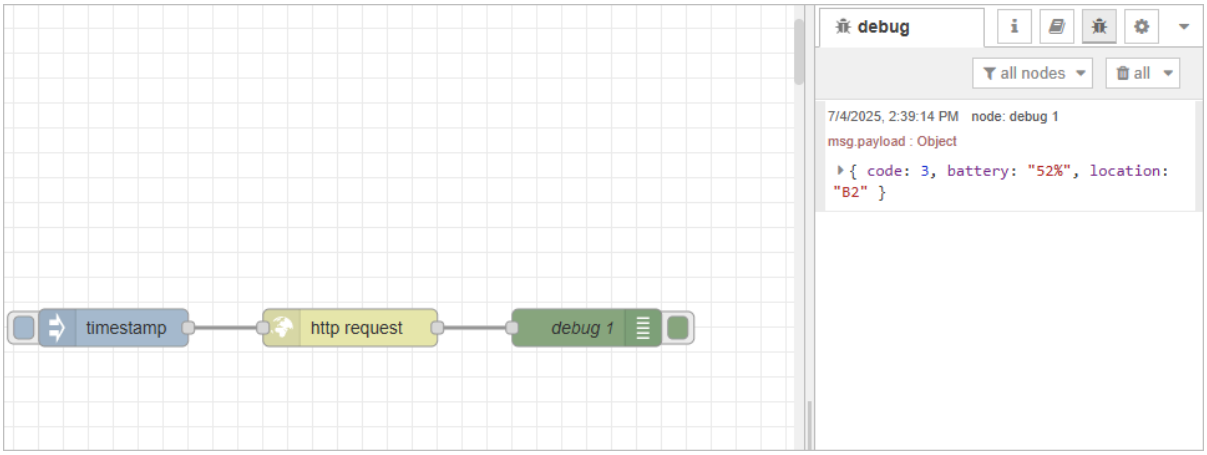
1. 在 **数据连接** 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `http request` 节点，然后双击该节点。
3. 输入API相关信息。

提示

需将 **Return** 设置为 **a parsed JSON object**，以便平台的数据建模模块解析。



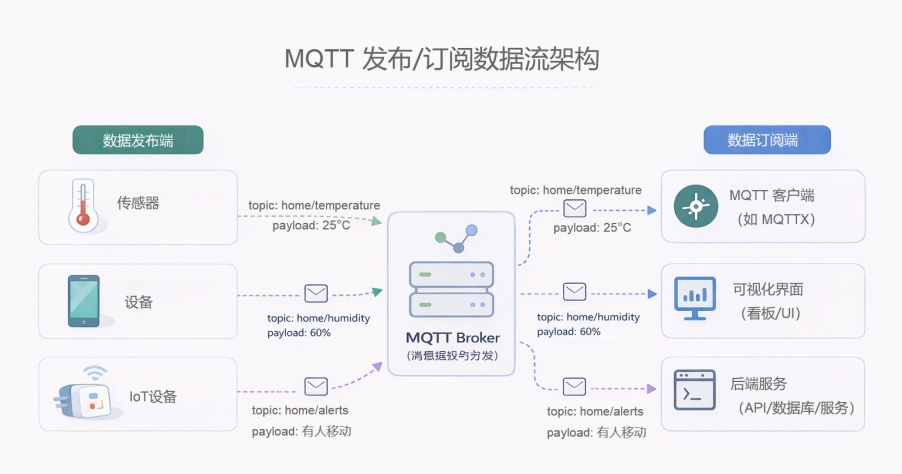
5. 单击 **完成**，然后在末端连接一个`debug`节点。
6. 单击右上角 **Deploy**，接着单击 `inject` 节点触发流程。



连接 MQTT 客户端

什么是 MQTT?

- MQTT 是一种轻量级实时消息协议，支持已连接的设备（客户端）向中央服务器（Broker）发送数据，并立即接收更新。
- 出于安全考虑，可以在中央服务器（Broker）上添加访问控制，因此设备（客户端）在连接前必须完成身份认证。
- 关于 MQTT 的更多信息，请参见 MQTT Protocol。



生成 MQTT 凭据

Tier0 提供内置 MQTT Broker，支持第三方客户端连接，并向 **UNS** 发布数据或订阅 **UNS** 中的数据。

1. 登录 Tier0，选择 **Edge** > **MQTT 接入**。
2. 单击 **生成凭据**，输入名称，然后单击 **保存**。



> **信息**
>
> - 开启**允许随机客户端 ID 后缀**后，生成的 **客户端 ID** 本身及附加不同后缀的 ID 均可与该用户名和密码匹配，用于建立连接。
> - 请记录生成的凭据，配置 MQTT 客户端时需要使用。

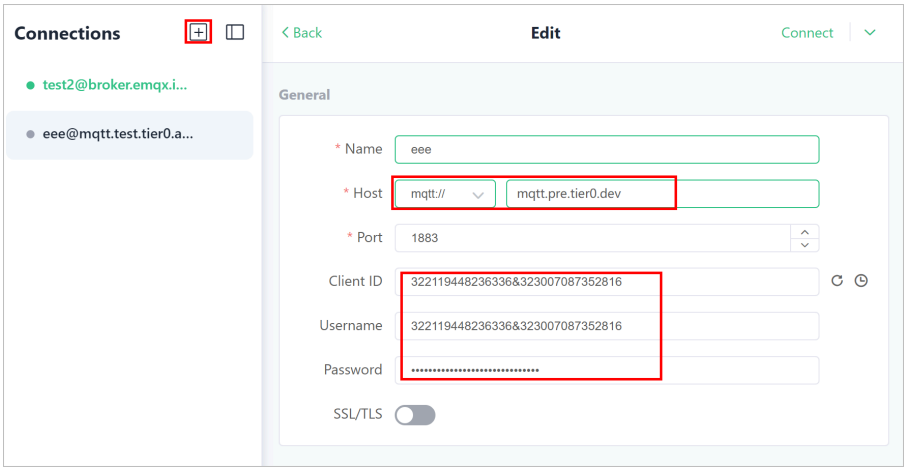
使用凭据连接 MQTT 客户端

> **备注**
>
> 以下客户端仅作为示例，用于说明如何使用凭据连接内部 MQTT Broker。

1. 安装 MQTT 客户端，例如 MQTTX、MQTT Explorer 等。
2. 使用 Tier0 中生成的凭据连接客户端。

- **MQTTX**

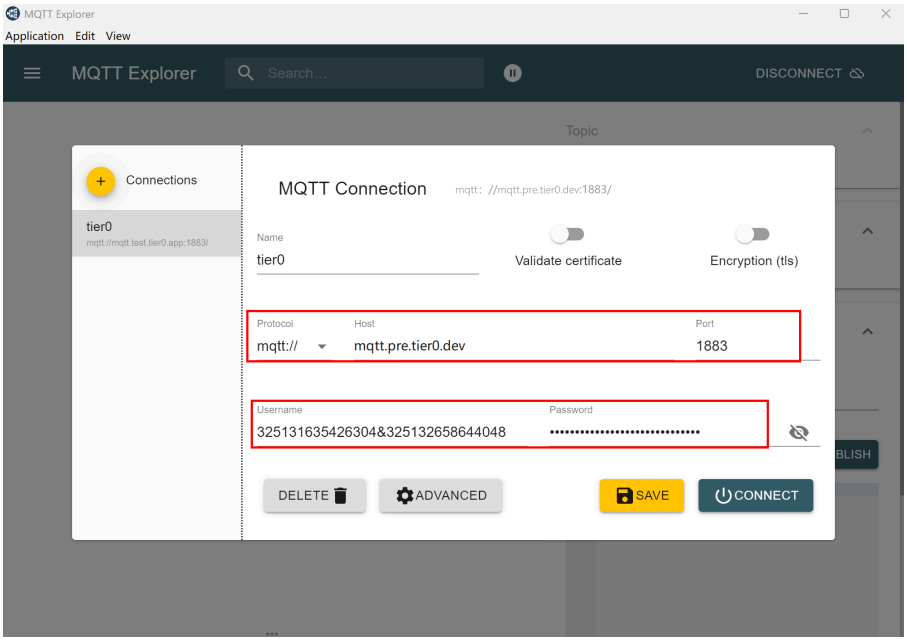
1. 在 MQTTX 中使用生成的认证凭据添加新连接。



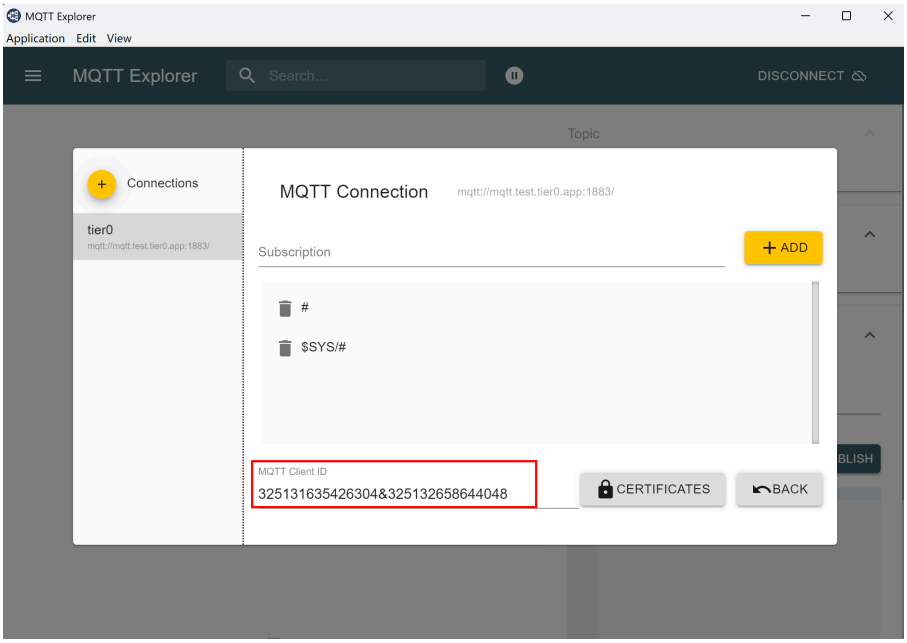
2. 单击 **Connect**。

- **MQTT Explorer**

1. 在 MQTT Explorer 中使用生成的认证凭据添加新连接。



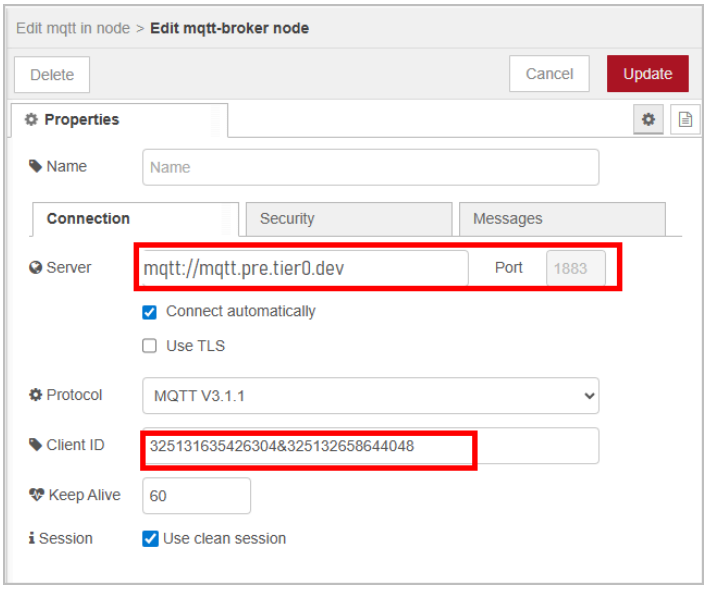
2. 单击 **ADVANCED**，修改 Client ID。

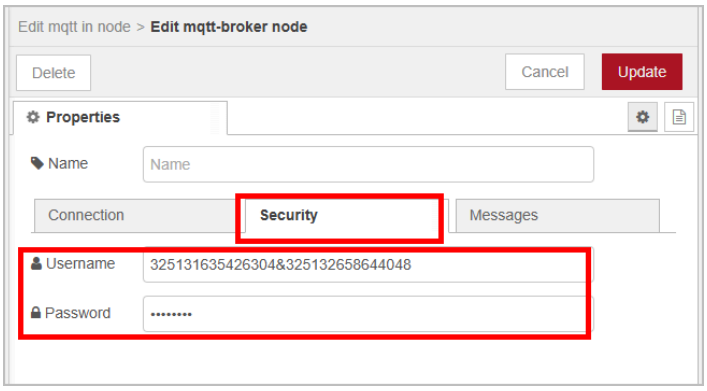


3. 单击 **BACK**，然后单击 **CONNECT**。

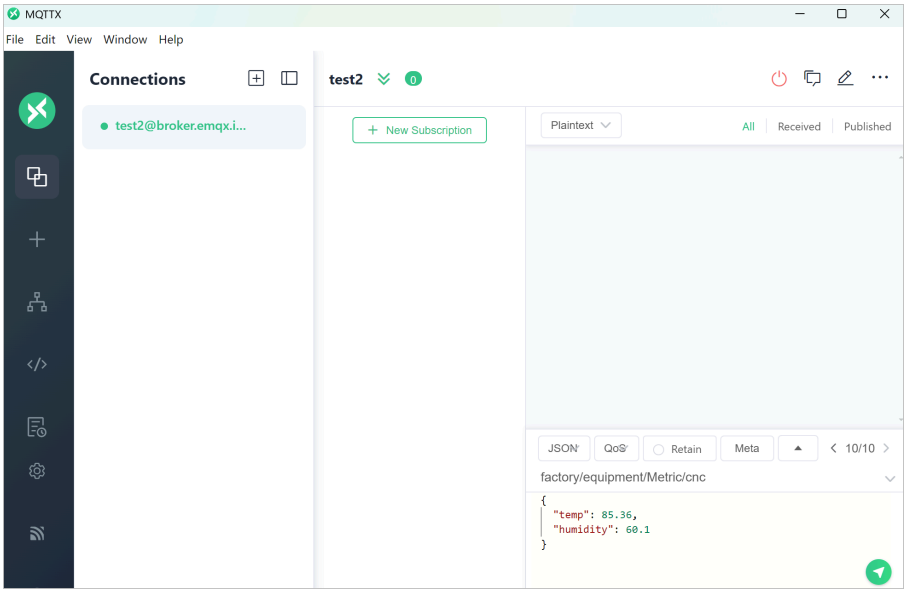
- **Node-RED**

1. 打开 Node-RED 实例，拖拽一个 `mqtt in` 节点并双击配置。
2. 使用 Tier0 中生成的认证凭据添加新的 MQTT Broker。





3. 返回 Tier0 的 **Edge** 页面，单击 **客户端** 页签，检查客户端是否已连接。
4. 复制 **UNS** 下数据模型的 Topic，并通过客户端向该 Topic 发布消息。

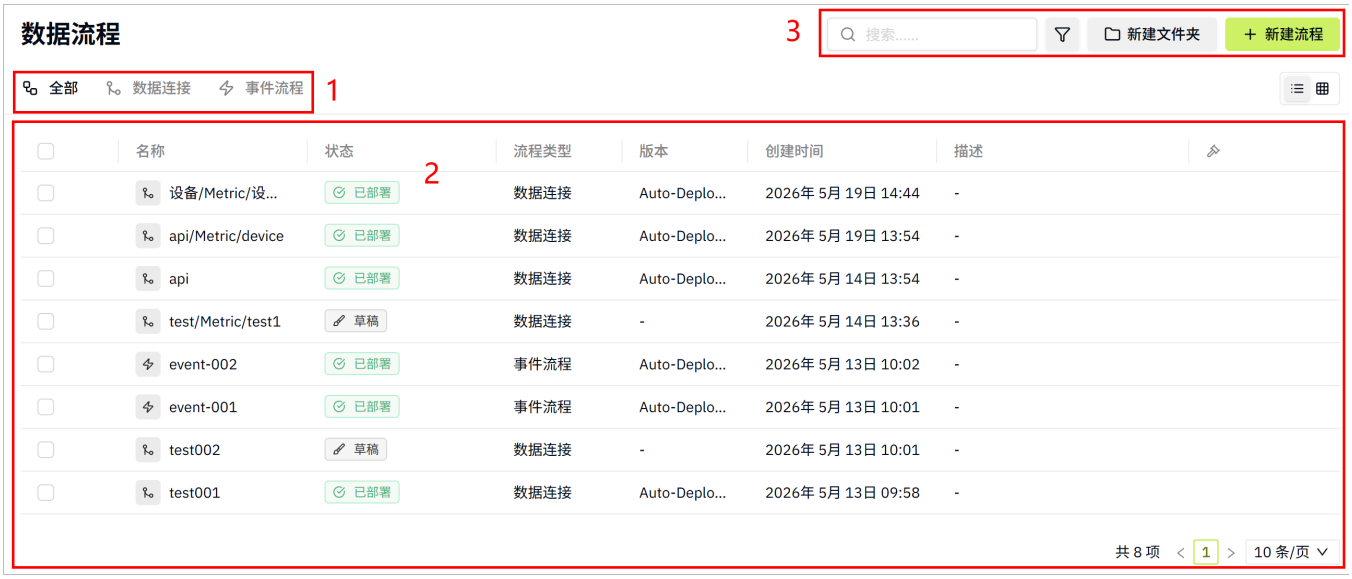


5. 返回 **UNS**，检查目标 Topic 是否已接收到数据。

处理数据

处理数据

1. 登录 Tier0，然后选择 **流程。**



编号	项目	说明
1	数据流列表	选择查看全部数据流，或仅查看某一分类下的流。
2	数据流信息	显示数据流基础信息。单击流名称可开始编辑数据流。
3	搜索/筛选	搜索或筛选已有数据流。
	新建文件夹	添加文件夹，用于对数据流进行分类。
	新建流程	添加新的数据流。

2. 单击 **新建流程**，输入名称和说明，将 **流程类型** 选择为 **事件流程**，然后单击 **保存。**



3. 单击刚添加的流程，构建数据事件以处理数据。

常规数据处理

整合多源数据

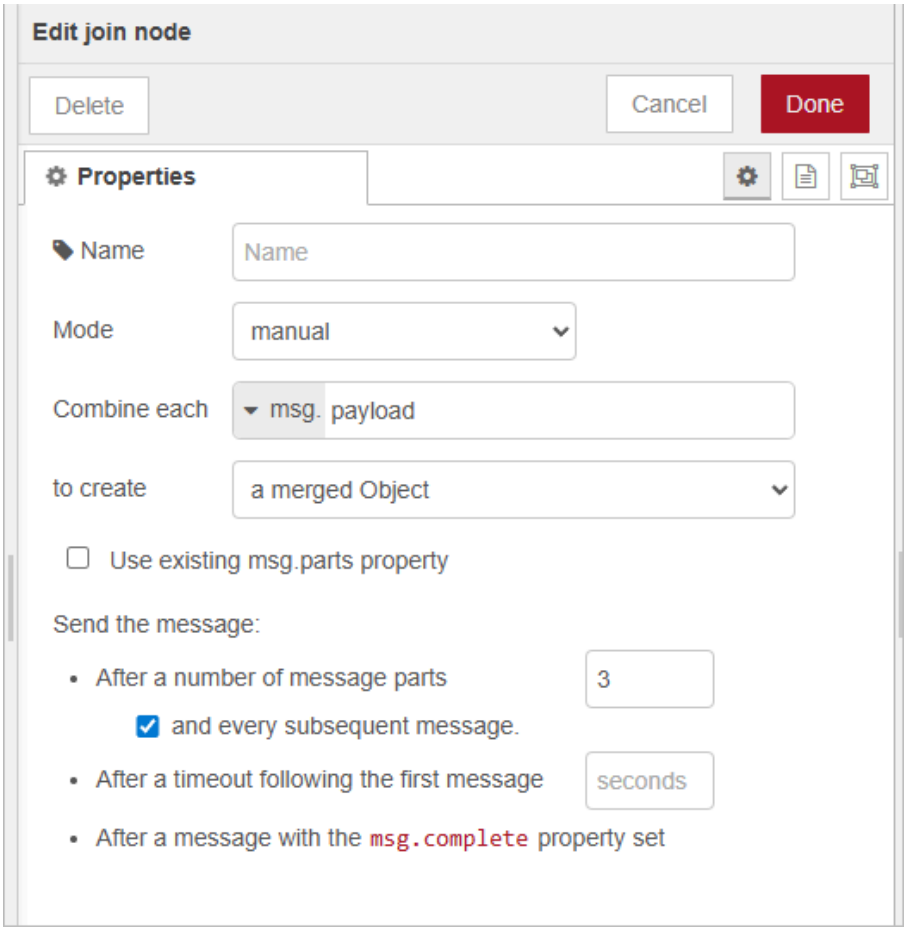
示例

平台已连接3个数据源，需将这些数据源的数据合并后用于后续处理。

```
{
  "workshop": {
    "furnace1": {
      "temp": 85
    },
    "furnace2": {
      "humidity": 64
    },
    "furnace3": {
      "pressure": 80
    }
  }
}
```

需使用的节点

本示例中，我们使用 `join` 节点来整合之前接收的多条消息。

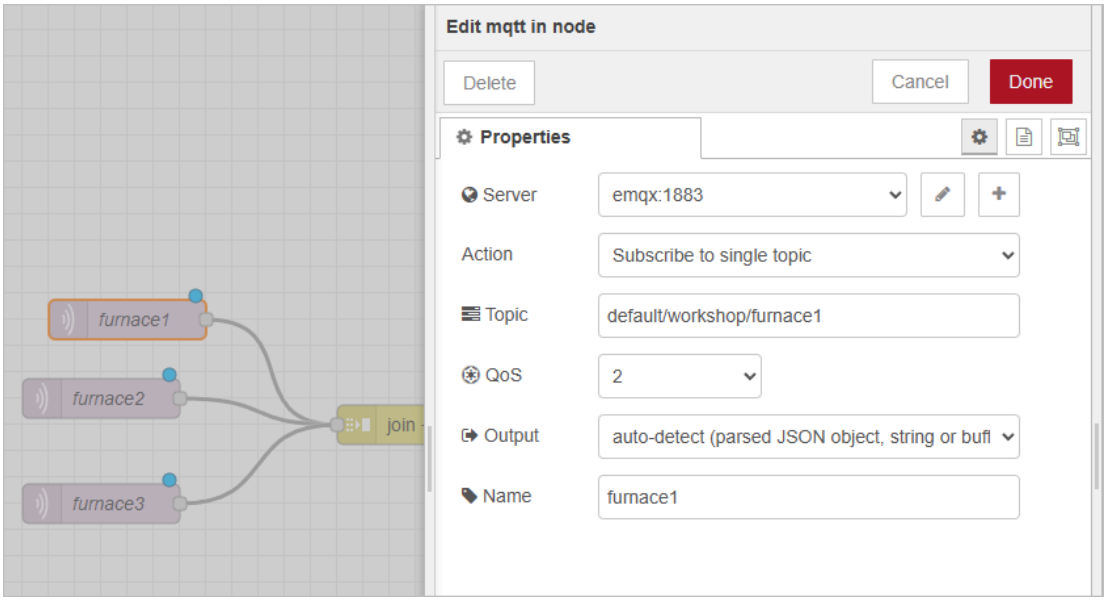


参数（Parameter）	说明（Description）
模式（Mode）	合并模式。 • **手动（manual）**：自行配置消息的合并方式。 • **自动（auto）**：基于 `msg.parts` 自动重组由 **拆分（split）** 节点拆分的消息序列。 • **归约序列（reduce sequence）**：对消息序列执行 JavaScript 归约函数，生成单个结果。
合并对象（Combine each）	想要合并的消息部分。 常用为 `msg.payload`，也可选择其他（如 `msg.data.value`）。
生成结果类型（to create）	合并后的结果类型： • **合并对象（a merged Object）**：将所有对象合并为单个对象（后接收的同名字段会覆盖先接收的）。 • **键值对象（a key/value Object）**：以 **键属性（key property）**（默认 `msg.topic`）为键，**合并对象（Combine each）** 对应的内容为值。 • **数组（an Array）**：按接收顺序将每个值追加到数组中。 • **字符串/二进制缓存（a String / a Buffer）**：拼接文本或二进制数据（字符串可设置分隔符；二进制缓存直接以二进制形式拼接）。
使用已有的 `msg.parts` 属性（Use existing `msg.parts` property）	勾选后，节点会使用前置节点传入的 `msg.parts` 属性（通常由 **拆分（split）** 节点生成）来确定消息分组规则，以及何时输出合并后的结果。 会用到 `msg.parts.count`、`msg.parts.index` 等字段。

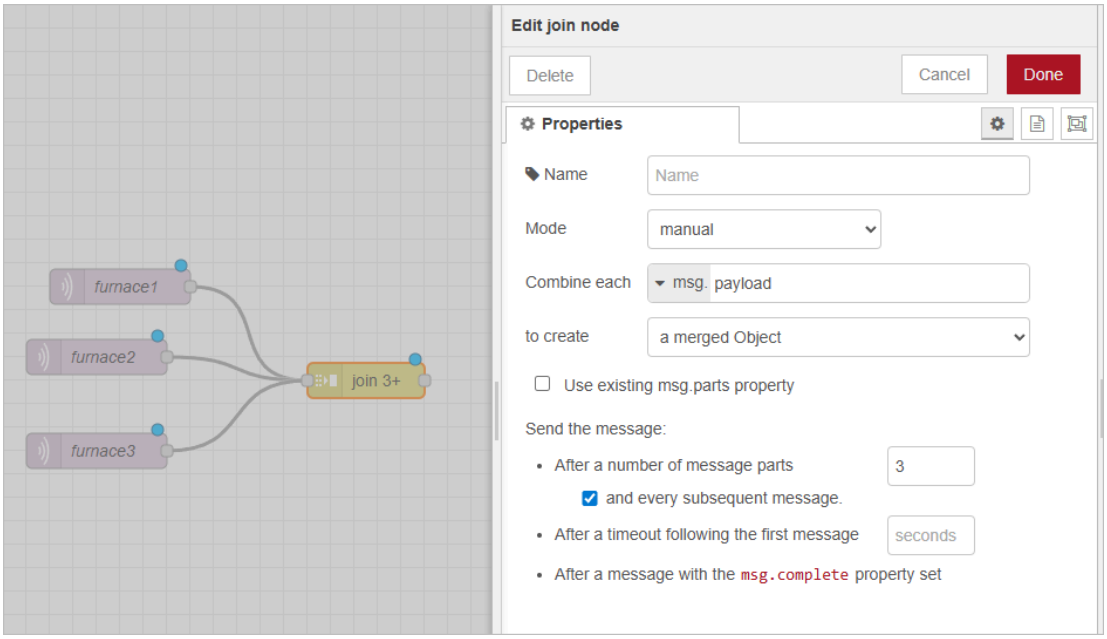
参数（Parameter）	说明（Description）
发送消息 → 接收指定数量的消息片段后（After a number of message parts）	接收到设定数量的消息片段后，输出合并后的结果。
发送消息 → 以及后续每条新消息到达时（and every subsequent message）	勾选后，首次输出合并结果后，每有新消息到达，节点都会输出更新后的合并结果。
发送消息 → 第一条消息到达后超时触发（After a timeout following the first message）	可选配置。第一条消息到达时启动计时器，超时（单位：秒）后， 节点输出当前已收集到的所有内容的合并结果。
发送消息 → 收到含 `msg.complete` 属性的消息时（After a message with the `msg.complete` property set）	当收到携带 `msg.complete = true` 的消息时，立即输出合并后的结果。
（高级）字符串/二进制缓存分隔符（String/Buffer separator）	若结果类型为 **字符串（String）**，可设置可选的分隔符； 若为 **二进制缓存（Buffer）**，则直接拼接数据，无分隔符。

如何合并多数据源?

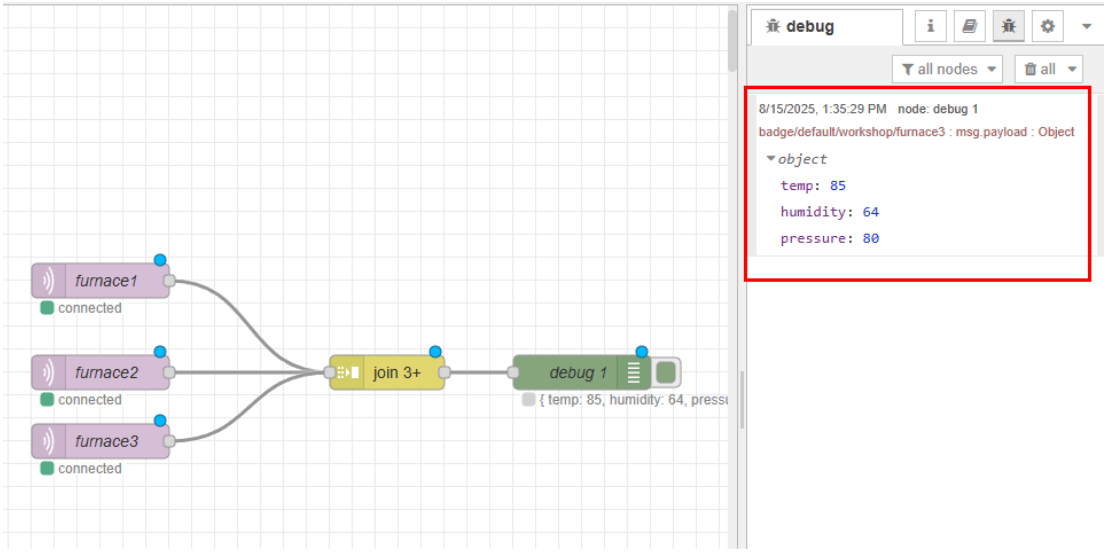
1. 在 **事件流程** 中拖拽 3 个 `mqtt in` 节点，分别订阅对应的主题。



2. 将 3 个节点都连接到同一个 `join` 节点，并配置为合并生成一个对象。



3. 连接 `debug` 节点，触发流程后查看合并结果。



数据过滤

示例

熔炉（furnace） 数据源包含 3 个字段，我们需要根据温度阈值对消息进行路由：

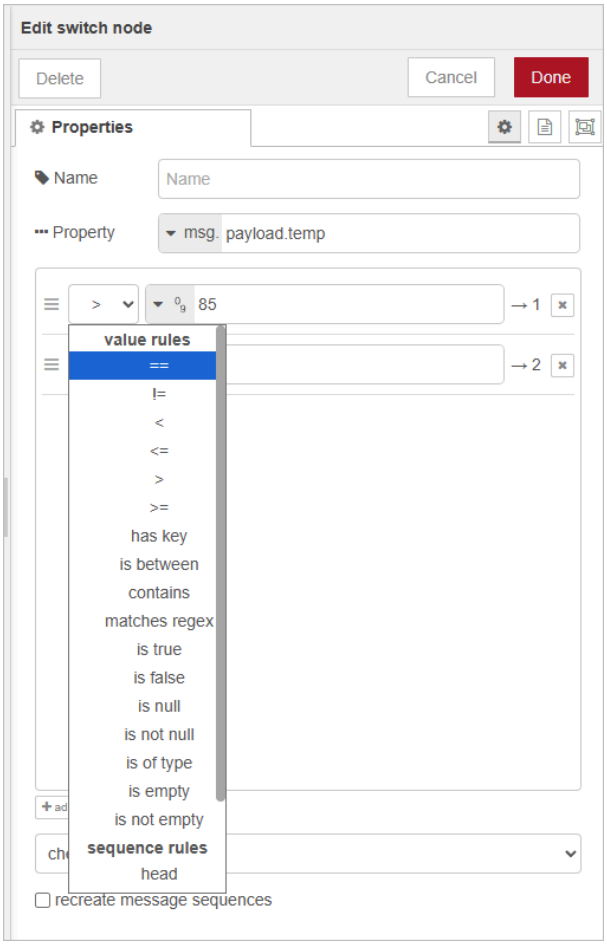
- 若 **温度（temp）> 85** → 路由至 **输出 1（Output 1）**
- 若 **温度（temp）≤ 85** → 路由至 **输出 2（Output 2）**

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
```

```
    }
}
```

需使用的节点

本示例中，我们使用 `switch` 节点根据条件对消息进行路由。

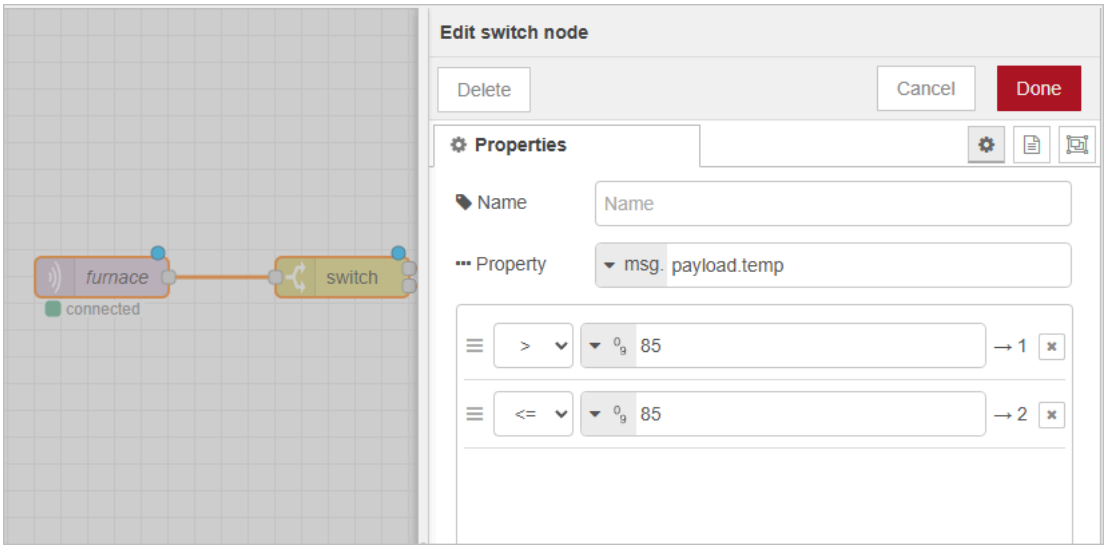


参数（Parameter）	说明（Description）
属性（Property）	需评估的属性路径。常用 `msg.payload` 或更深层的属性， 本示例中为 `msg.payload.temp`。
规则（Rules）	一组评估规则，每条规则对应一个输出端口。
类型（Type）	比较的数据类型：数字（number）、字符串（string）、布尔值（boolean）、JSONata 等。 **信息** 确保数值比较不会被当作字符串比较处理。

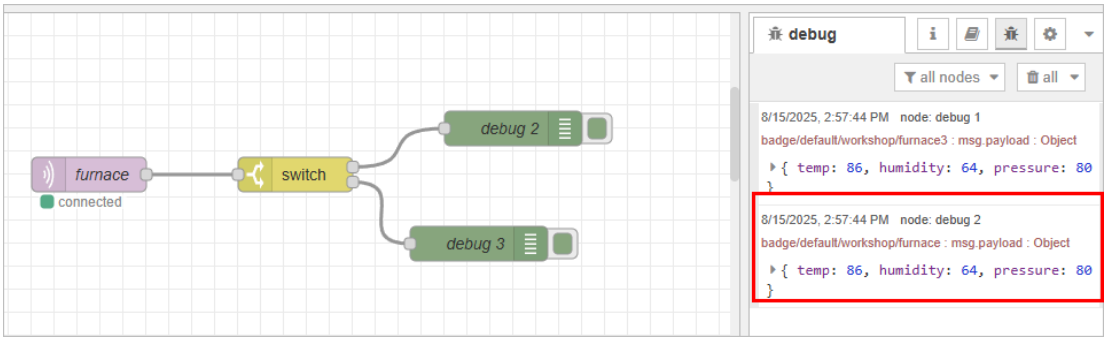
参数（Parameter）	说明（Description）
值（Value）	比较的目标值来源：常量值（如 85）、`msg.` 字段（如 `msg.threshold`）、`flow.` / `global.` 上下文变量，或 `JSONata` 表达式。
检查所有规则（checking all rules）	按顺序评估所有规则。若消息匹配多条规则，会发送至多个输出端口。
找到第一个匹配后停止（Stop after first match）	找到第一个匹配规则后停止评估，消息仅发送至一个输出端口。
重建消息序列（recreate message sequences）	启用后，保留并重建消息序列元数据（msg.parts）——通常由拆分（split）节点生成—— 确保消息序列在分支后仍能保持完整。

如何筛选数据？

1. 在 **事件流程** 中添加一个 `mqtt in` 节点，获取平台中的数据。
2. 拖拽一个 `switch` 节点并配置其属性。



3. 在 `switch` 节点的每个输出端口分别连接一个 `debug` 节点。
4. 触发流程并查看结果。



更改数据

示例

我们接收到一条消息，需要重命名字段、计算新值并移除无用字段，以便后续流程更简洁地处理数据。

输入消息

```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

目标转换要求

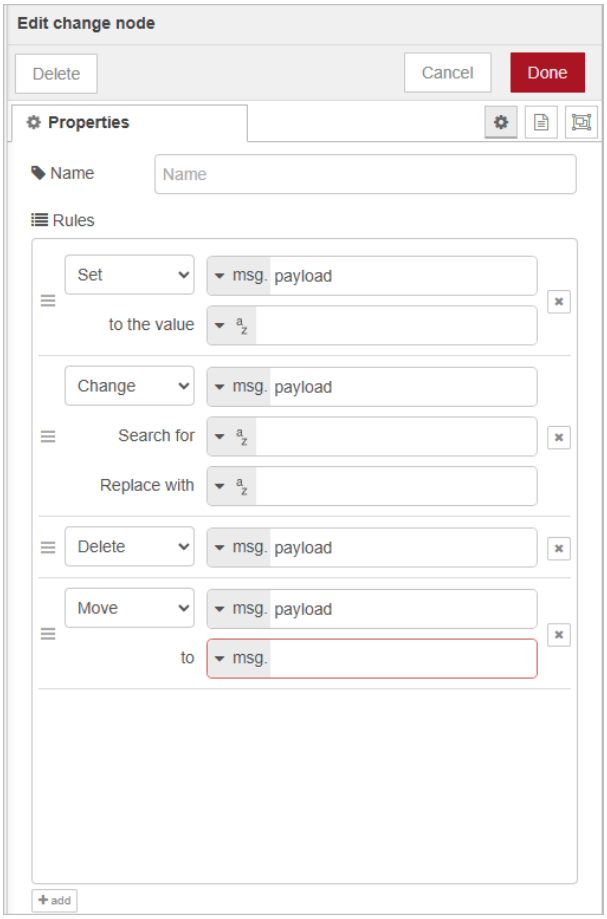
- 将 furnace.temp_c 移动至 furnace.temperature.celsius
- 新增 furnace.temperature.fahrenheit 字段（由摄氏度转换为华氏度）
- 移除 pressure 字段
- 将 site 字段值从 A1 修改为 B1

输出消息（示例）

```
{
    "furnace":{
        "humidity":64,
        "temperature":{
            "celcius":30.5,
            "fahrenheit":86.9
            }
        },
    "site":"B1"
}
```

需使用的节点

此场景下，我们使用 `change` 节点对消息属性执行 **设置（Set）**、**替换（Change）**、**删除（Delete）** 和 **移动（Move）** 操作。

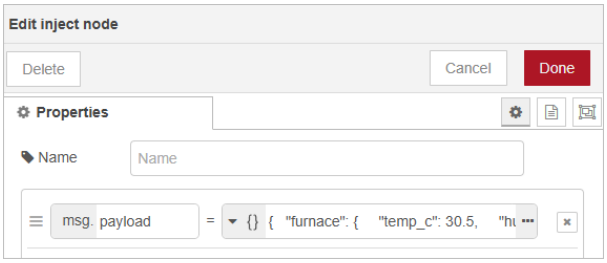


参数	说明
规则（Rules）	每条规则按从上到下的顺序执行。 可添加多条规则完成复杂的数据转换。
规则类型（Rule Type）	• **设置（Set）**：设置或新建属性值。 • **替换（Change）**：查找并替换值。 • **删除（Delete）**：移除指定属性。 • **移动（Move）**：将属性从一个路径移动至另一个路径。
属性/源/目标（Property/Source/Target）	定义要操作的属性路径，例如 `msg.payload` 或 `msg.payload.furnace.temp_c`。 也可引用 `flow.` 或 `global.` 上下文变量。
目标值（Target Value）	• **设置（Set）**：要赋值的目标值。 • **替换（Change）**：要查找并替换的目标值。

如何使用修改节点?

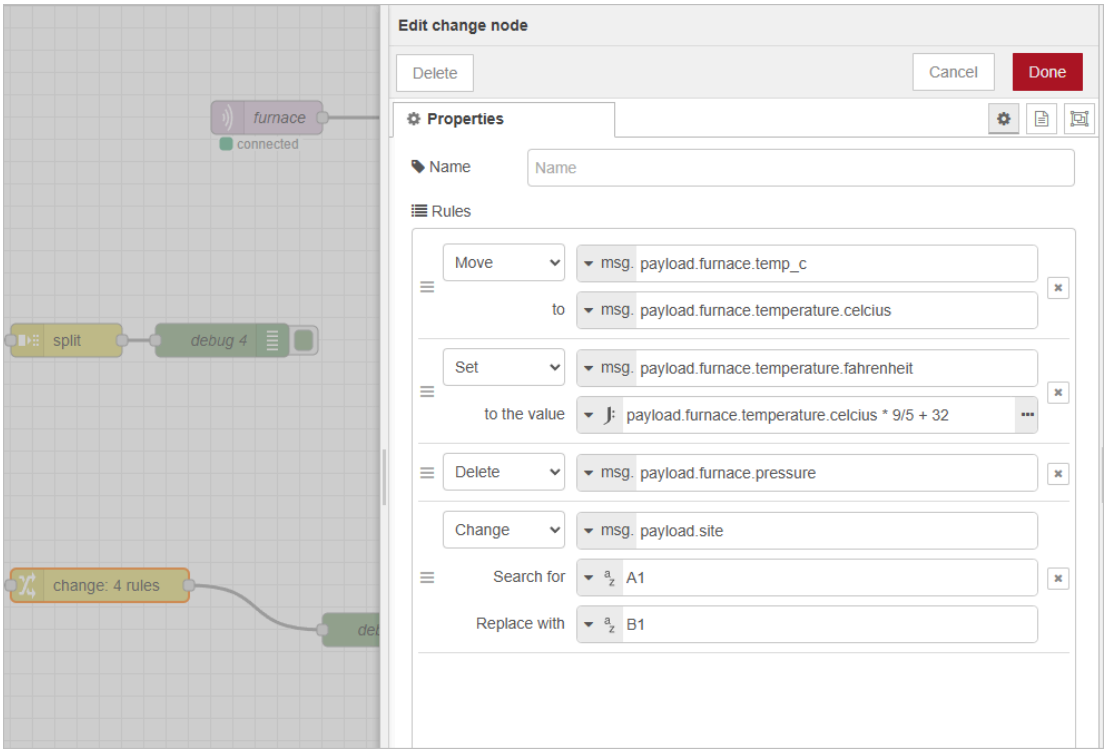
1. 在 **事件流程** 中拖拽一个 `inject` 节点，添加以下JSON作为输入内容。

```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

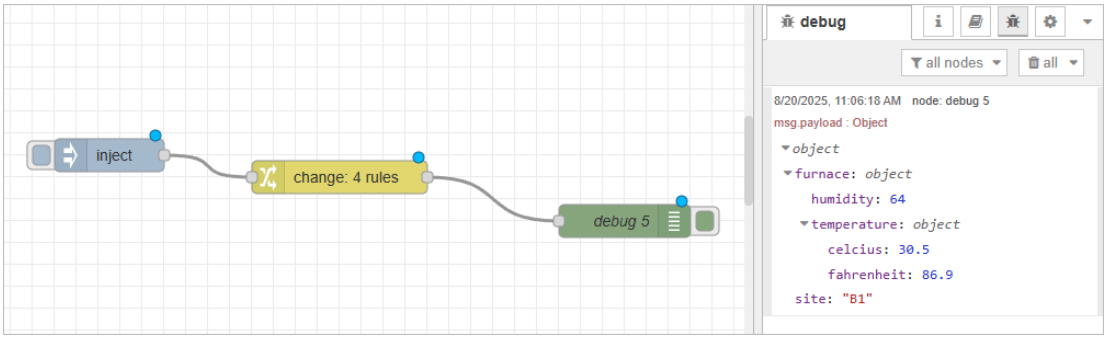


2. 将其与 `change` 节点连接。
3. 打开 `change` 节点配置面板，添加以下规则：

- Move: msg.payload.furnace.temp_c → msg.payload.furnace.temperature.celcius
- Set: msg.payload.furnace.temperature.fahrenheit → payload.furnace.temperature.celcius * 9/5 + 32
- Delete: msg.payload.pressure
- Change: 将 msg.payload.site 的值从 A1 改为 B1



4. 将 `change` 节点与 `debug` 节点连接，查看转换后的输出结果.



拆分数据

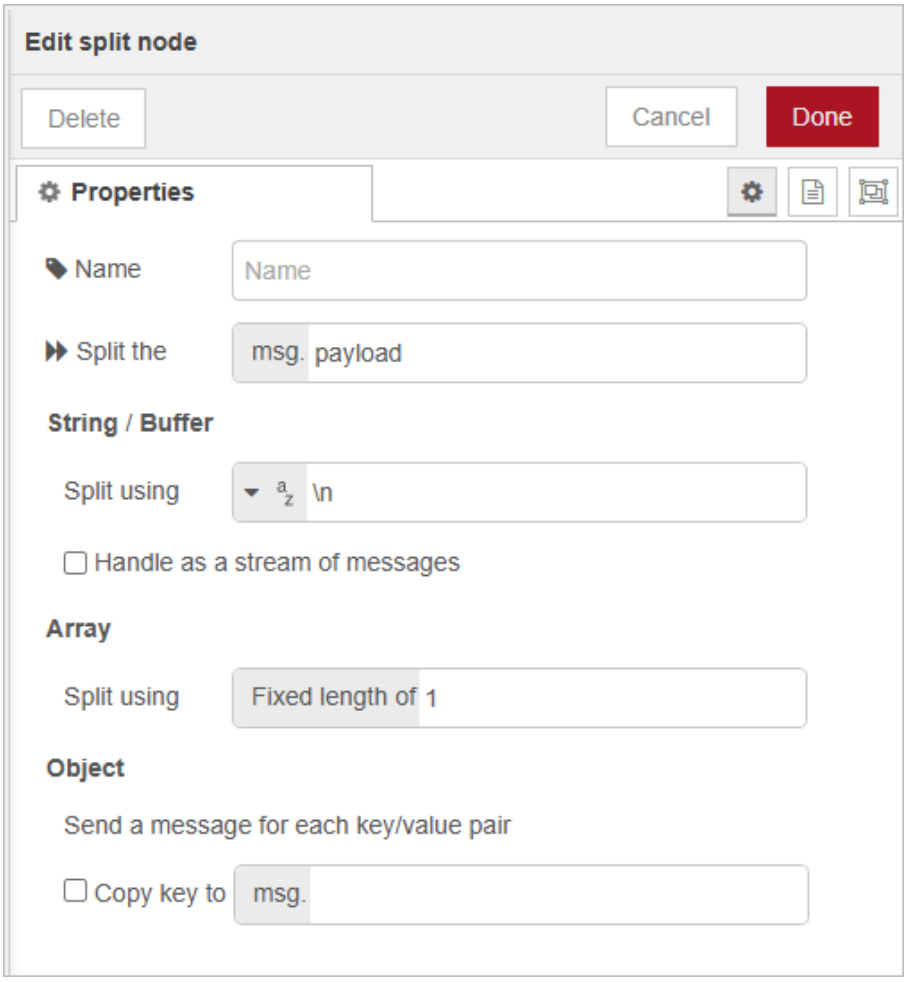
示例

我们有一条包含多组熔炉读数的消息，需要将其拆分为多条独立的消息，以便进行后续处理。

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
```

需使用的节点

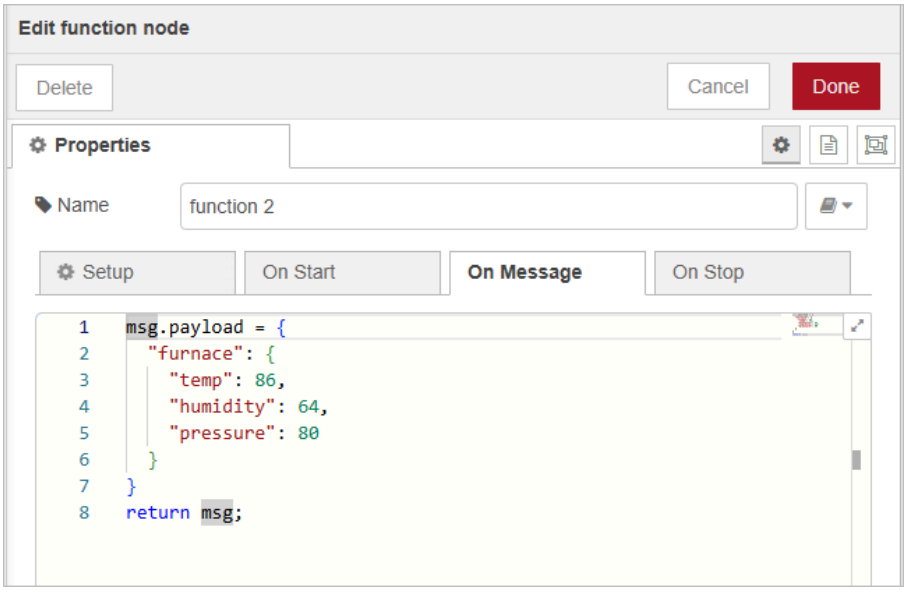
本示例中，我们使用 `split` 节点将一条消息拆分为多条更小的消息。



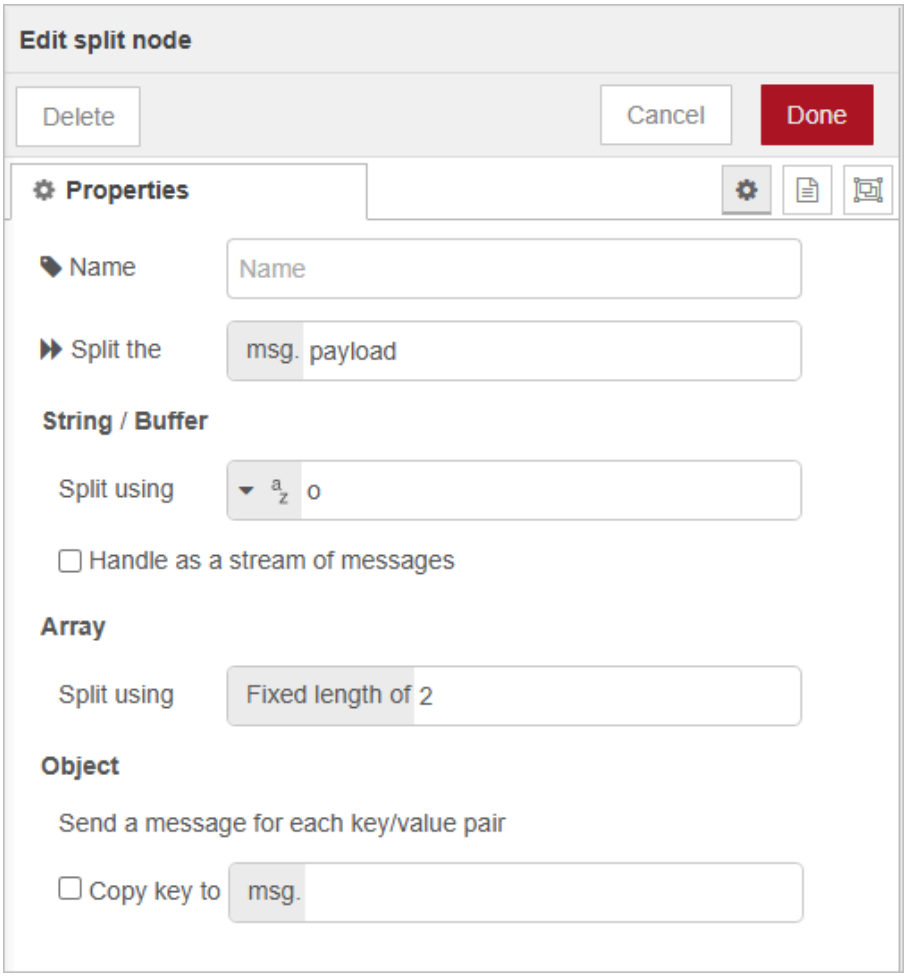
参数（Parameter）	说明（Description）
名称（Name）	节点的可选标签，用于在流程中标识该节点。
拆分对象（Split the）	要拆分的消息属性，通常为 `msg.payload`，也可以是其他属性（如 `msg.payload.data`）。
字符串/二进制缓存 - 拆分方式（String / Buffer - Split using）	针对字符串或二进制缓存类型的负载，指定拆分方式： • **字符串（string）**：使用特定字符串或正则表达式模式拆分消息。 • **二进制缓存（buffer）**：按设定的块大小拆分消息。 • **固定长度（Fixed length of）**：按设定的长度拆分字符串。 • 留空则不拆分字符串。
作为消息流处理（Handle as a stream of messages）	勾选后，节点会将传入数据视为连续流，数据到达时立即发送拆分后的消息，而非等待完整负载接收完毕。
数组 - 拆分方式（Array - Split using）	**固定长度 N（Fixed length of N）**：将数组拆分为多条消息，每条消息包含 N 个元素。
对象 - 为每个键值对发送一条消息（Object - Send a message for each key/value pair）	每个键值对的值将作为拆分后消息的负载（payload）返回。
将键复制到 `msg.payload`（Copy key to `msg.payload`）	勾选后，原始键名会作为拆分后消息的 `msg.payload` 返回。

如何拆分消息?

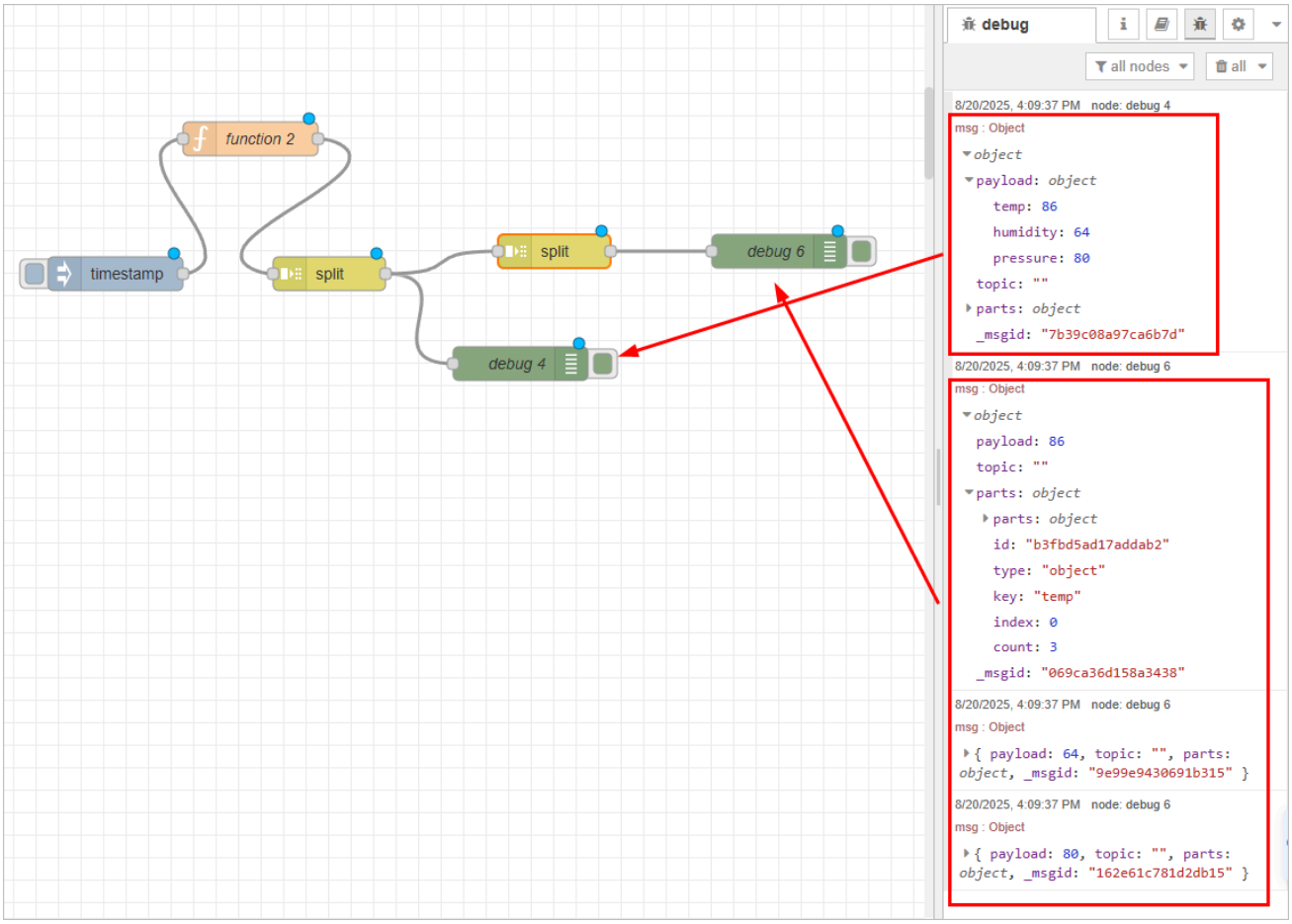
1. 在 **事件流程** 中拖拽一个 `inject` 节点，再通过一个 `function` 节点传递目标对象数据。



2. 按顺序将其连接到 2 个 `split` 节点，并为两个节点配置一样的「对象（Object）」属性.



3. 将 `split` 节点与 `debug` 节点连接，查看每条拆分后的输出消息。

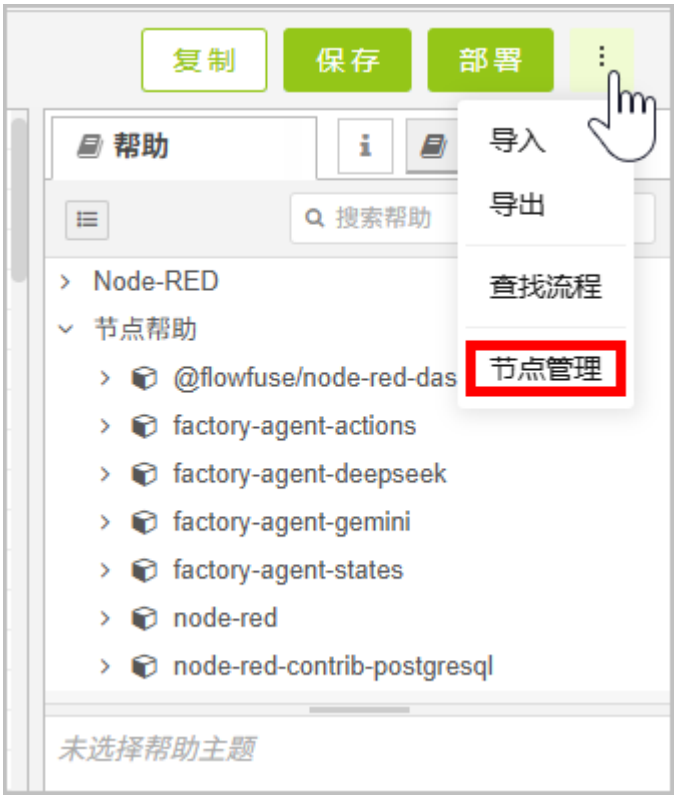


数据可视化

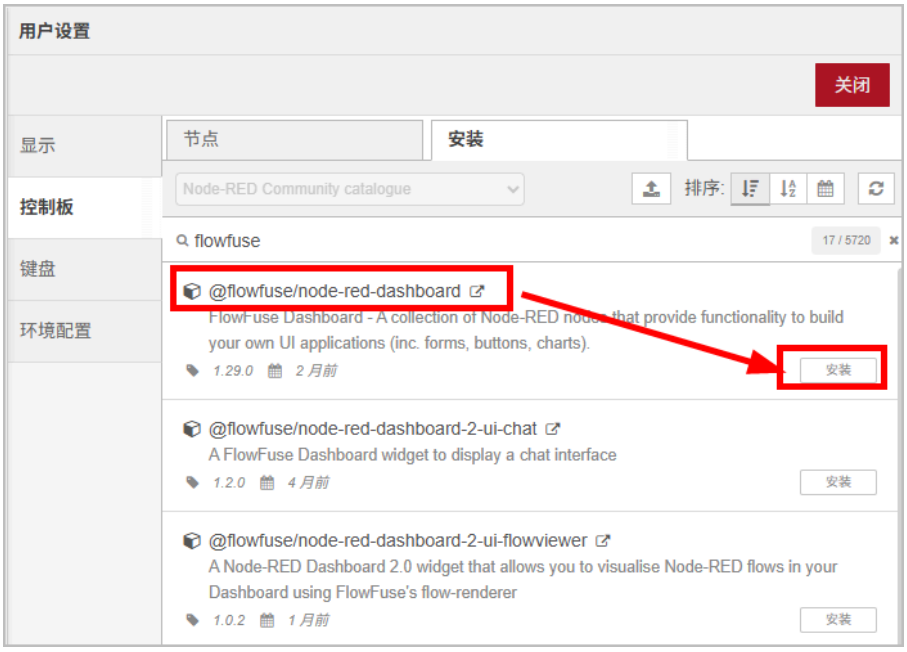
如何用事件流程构建数据看板

采集至平台并经过处理的数据，可在平台中以自定义样式的图表、表格、文本等格式进行可视化展示。

1. 登录平台，选择 **流程 > 事件流程**。
2. 单击 **新建流程**，将**流程类型**设为**事件流程**，添加一条流。
3. 单击流名称开始编辑。
4. 单击右上角 ⋮ 图标，然后单击 **节点管理**。



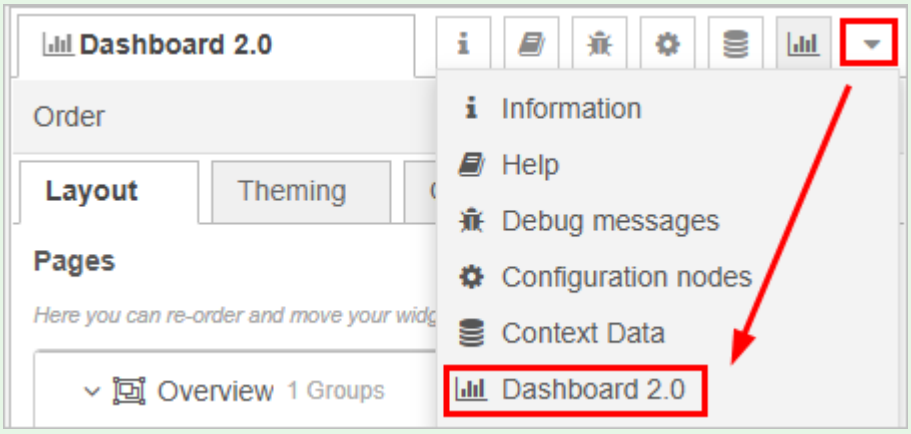
5. 在 **安装** 页面搜索并安装 `@flowfuse/node-red-dashboard` 节点。

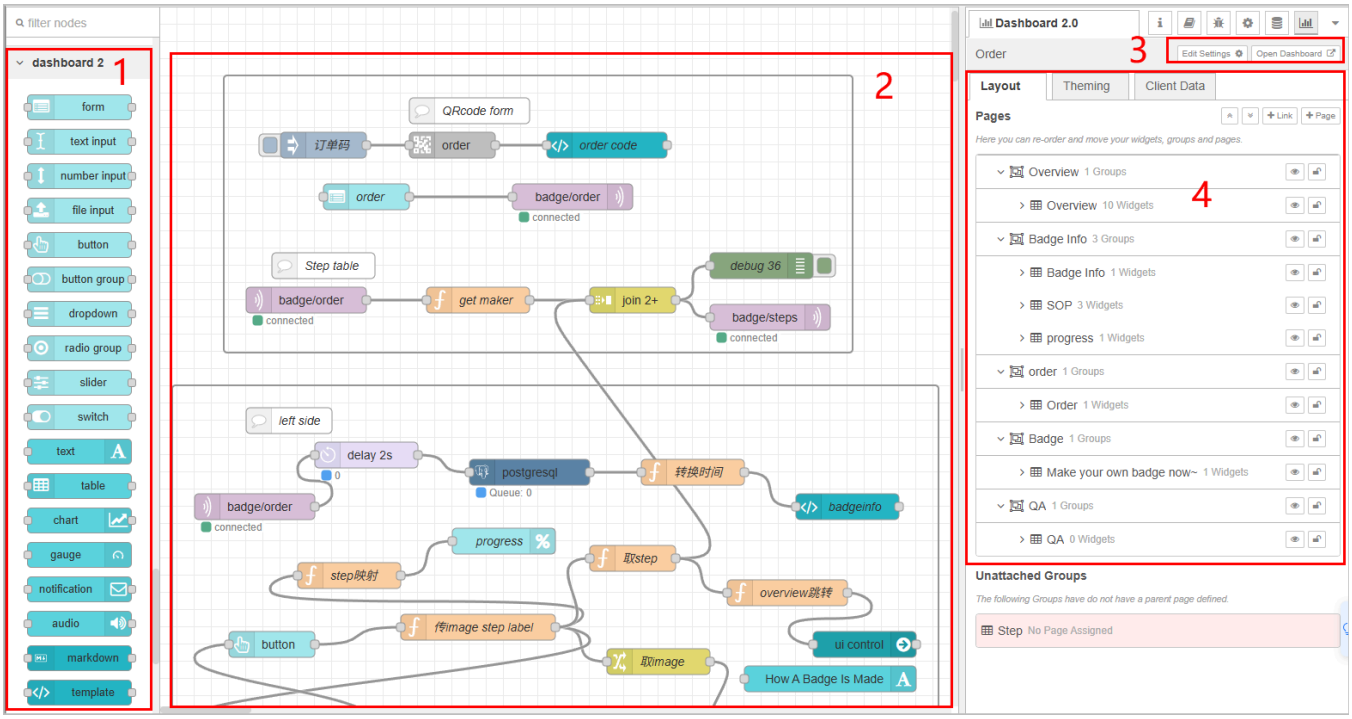


6. 使用 flowfuse 的节点构建数据看板：

提示

单击右上角 图标，然后单击 **Dashboard2.0** 可调整看板布局。





序号	项目	说明
1	节点库	Dashboard 2.0 相关节点，包括仪表盘、表单、进度条等。
2	画布	拖拽节点构建数据看板的操作区域。
3	操作栏	编辑 `ui-base` 节点及预览数据看板。
4	布局面板	按层级布局显示所有已添加的节点。

> **信息**
>
> 各节点详细使用方法请参考 Flowfuse 节点文档。

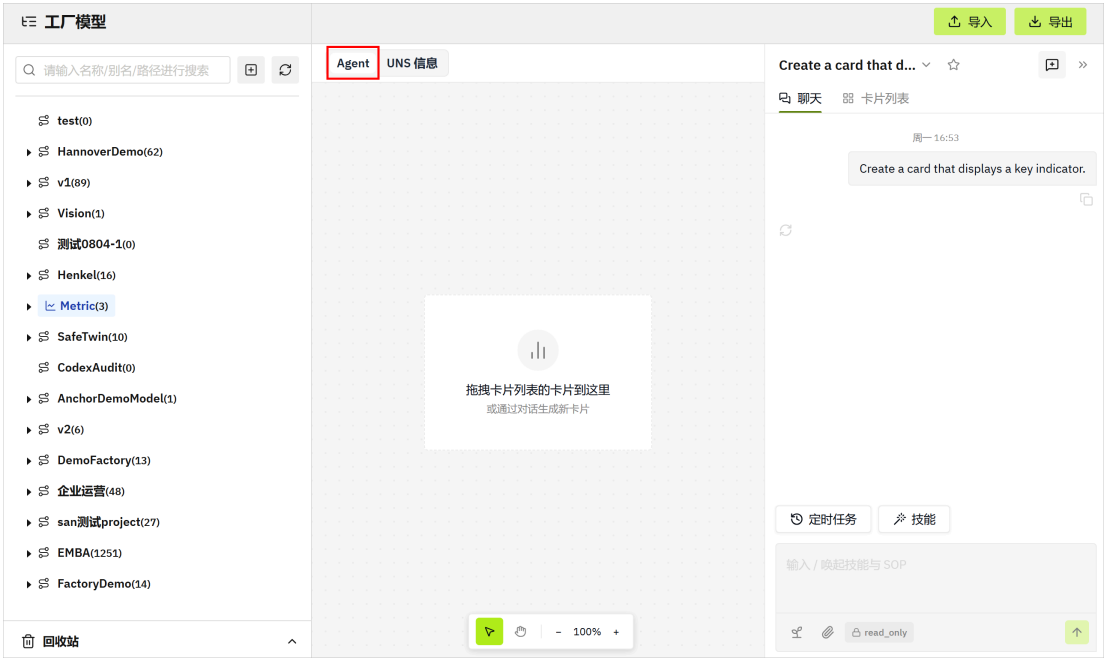
使用 UNS Agent

Tier0 在 UNS 模块中内置了 AI Agent，你可以直接使用自然语言与 UNS 模型进行交互。

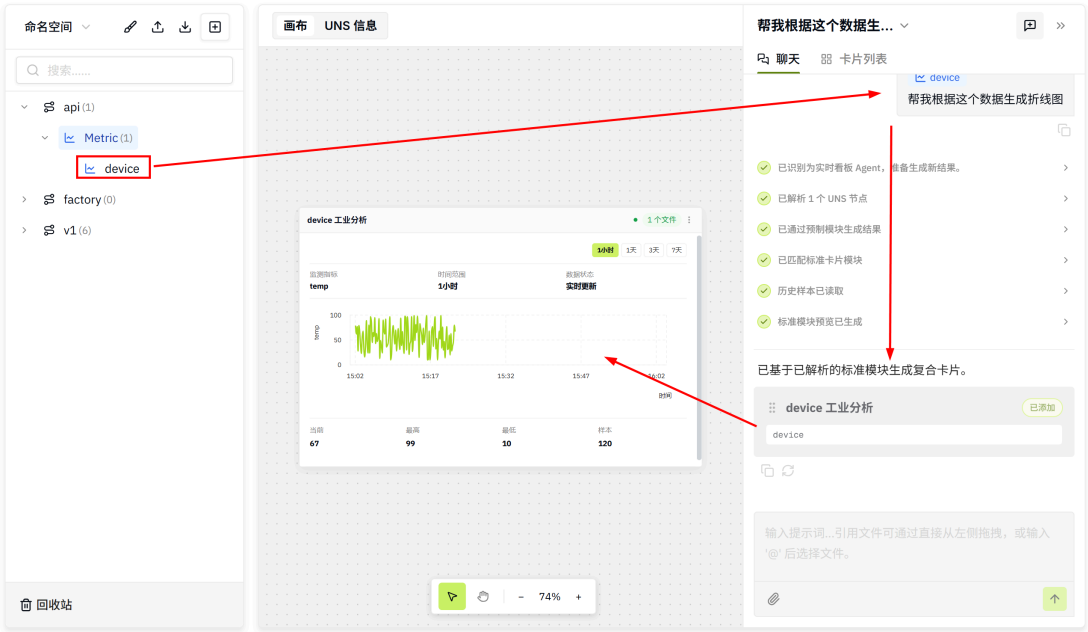
> **警告**
>
> 在与 Agent 对话前，你需要在 > **设置** > **AI 设置** 中设置智能体的 API Key，否则 Agent 将无法响应你的指令。

如何使用 UNS Agent

1. 登录 Tier0，在 **数据建模** 内，选择 **Agent** 页签。



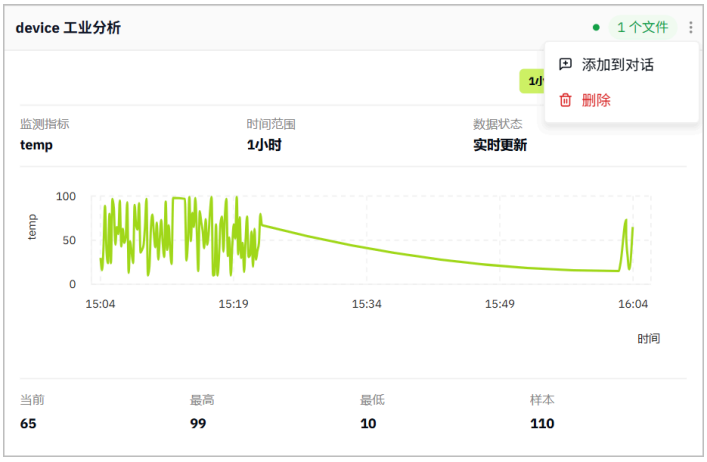
2. 在右侧聊天窗口中，将左侧的文件拖拽到输入框中或输入@选择文件完成数据引用，然后输入指令。
3. 将根据指令生成的卡片拖拽到画布中，即可查看可视化数据。



提示

- 单击对话消息左下角的 ，可以重新发送上一条指令。
- 单击顶部的 **卡片列表**，可以查看当前会话中生成的所有卡片。
- 单击右上角的 ，可以开始新的会话。

4. （可选）单击卡片右上角的 ，可以将卡片添加到聊天中或删除卡片。



AGENT 可以做什么?

UNS Agent 可以根据你的指令：

- 生成数据模型。
- 生成用于连接数据的**数据连接**流程，以及用于处理数据的**事件流程**。

高阶使用

同步数据

数据同步

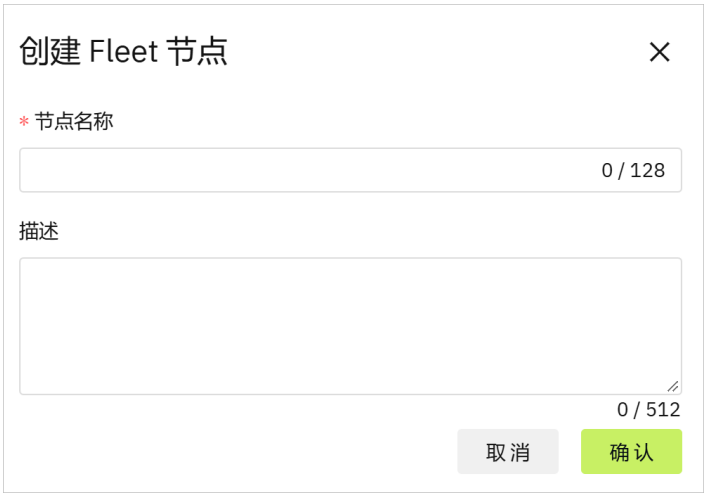
Tier0 企业版根据许可证类型分为两种版本。

- **中心节点**：许可证类型为 **Cluster** 时，对应的 Tier0 为中心节点。中心节点可以同步边缘节点的数据，并分配用户管理边缘节点。
- **边缘节点**：许可证类型为 **Base** 或 **Plus** 时，对应的 Tier0 为边缘节点。边缘节点可以向中心节点同步数据。

将边缘节点连接到中心节点

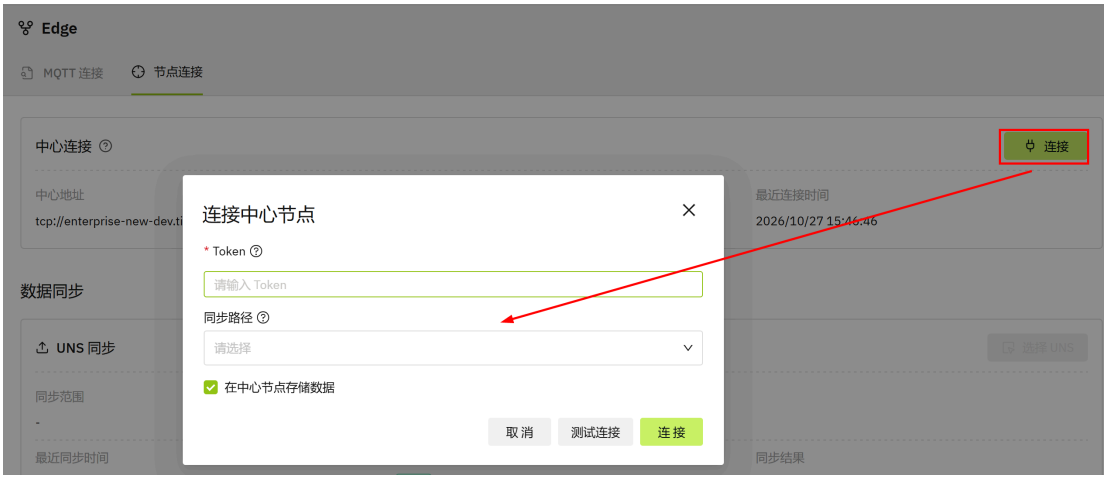
> **信息**
>
> 边缘节点既可以连接 Tier0 Enterprise（中心节点），也可以连接 Tier0 Cloud。

1. 在中心节点中，选择 **边缘** > **集群管理**。
2. 点击 **创建集群节点**，输入名称，然后点击 **确定**。



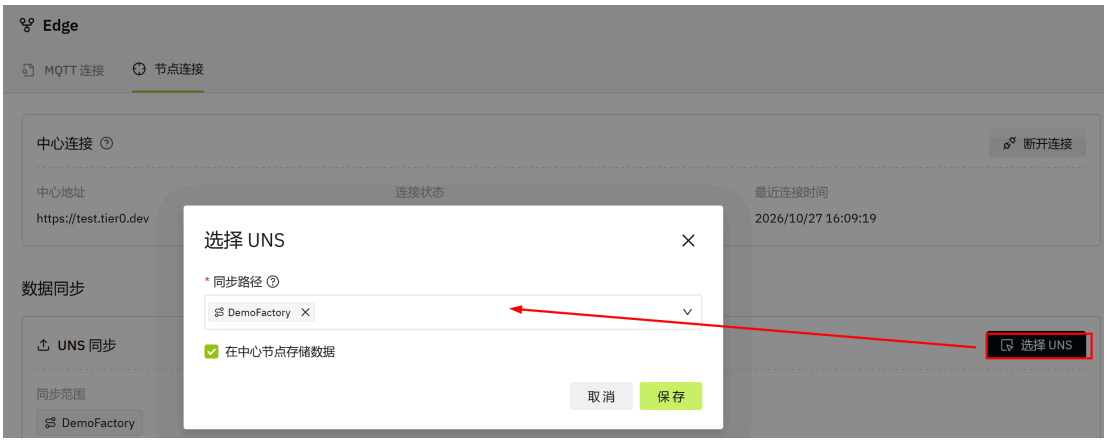
3. 复制访问令牌，然后在边缘节点中打开 **边缘** > **集群连接** 页面。
4. 点击 **连接**，输入访问令牌，然后再次点击 **连接**。

> **备注**
>
> 连接后，中心节点会显示边缘节点的资源、容器、UNS 和运行情况等信息。



从边缘节点向中心节点同步数据

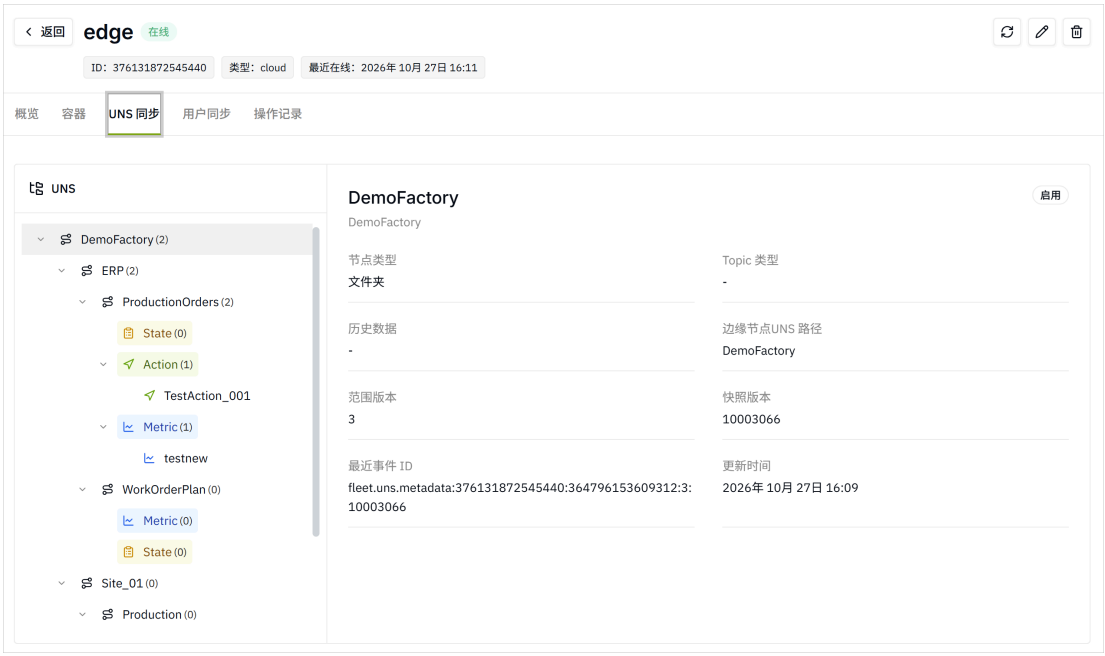
1. 在边缘节点的 **集群连接** 页面，点击 **选择 UNS**。



2. 选择要同步到中心节点的 UNS 路径，然后点击 **保存**。

> **信息**
>
> 选择 **在中心节点存储数据** 后，所选 UNS 路径下的所有数据都会同步到中心节点；否则只同步 UNS 结构。

3. 返回中心节点，检查已同步的 UNS。



分配用户管理边缘节点

1. 在中心节点的 **集群管理** 页面，选择已连接的边缘节点。
2. 在 **用户同步** 下，点击 **用户同步**，选择要分配来管理该边缘节点的用户。



3. 点击 **确定**，然后前往边缘节点。
4. 在 **集群连接** 下，点击 **同步用户**，同步已分配的用户。
5. 同步完成后，已分配的用户即可登录并管理边缘节点。

从 Tier0 获取数据

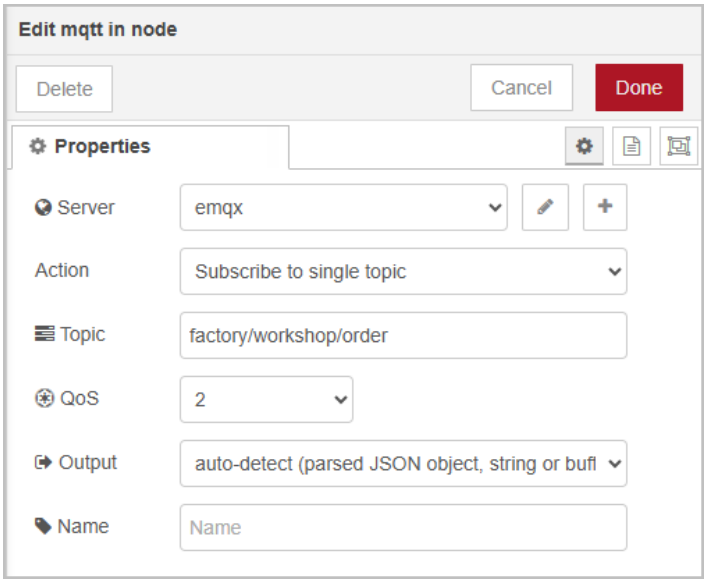
数据可通过多种协议采集至平台，也可通过多种方式传递给第三方。

通过MQTT订阅获取数据

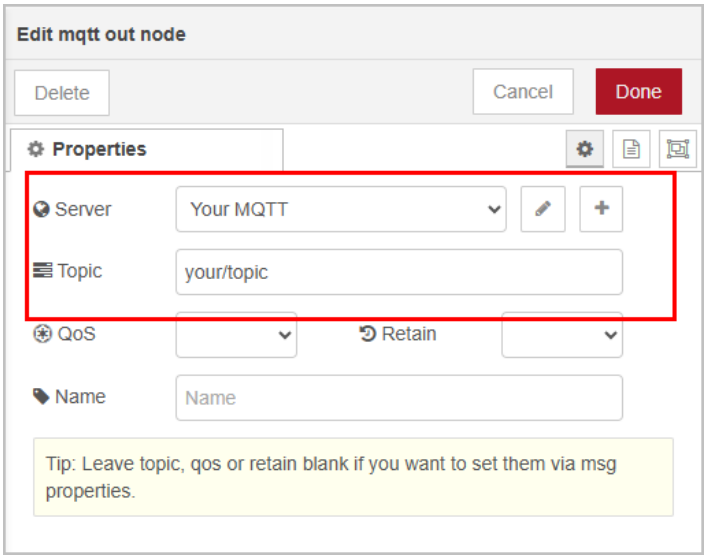
> **信息**
>
> 该方式适用于具备MQTT协议使用经验的第三方。

1. 登录平台，选择 **流程** > **事件流程**。
2. 单击 **新建流程**，将**流程类型**设为**事件流程**，输入名称后单击 **保存**。
3. 单击刚创建的事件流程，拖拽一个 `mqtt in` 节点并配置参数：

- 平台MQTT代理地址：`emqx:1883`
- 主题：需获取数据的文件路径（例如 `factory/workshop/order`）。



4. 对获取到的数据进行处理，或直接使用 `mqtt out` 节点将数据传输至第三方MQTT。

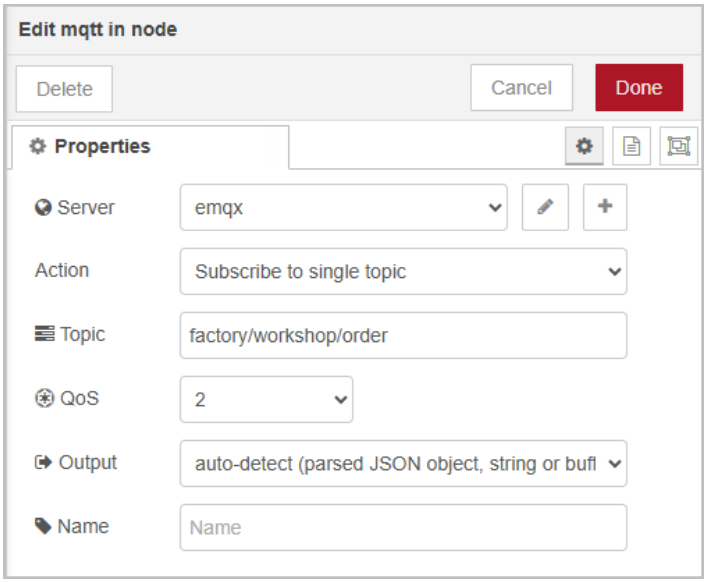


5. 部署流程，当订阅主题数据发生变化时，数据将自动推送至第三方。

通过API获取数据

> **信息**
>
> 该方式适用于不支持MQTT协议、需要事件驱动型数据的第三方。

1. 登录平台，选择 **流程 > 事件流程**。
2. 单击 **新建流程**，将**流程类型**设为**事件流程**，输入名称后单击 **保存**。
3. 单击刚创建的事件流程，拖拽一个 `mqtt in` 节点并配置参数：
   - 平台MQTT代理地址：`emqx:1883`
   - 主题：需获取数据的文件路径（例如 `factory/workshop/order`）。



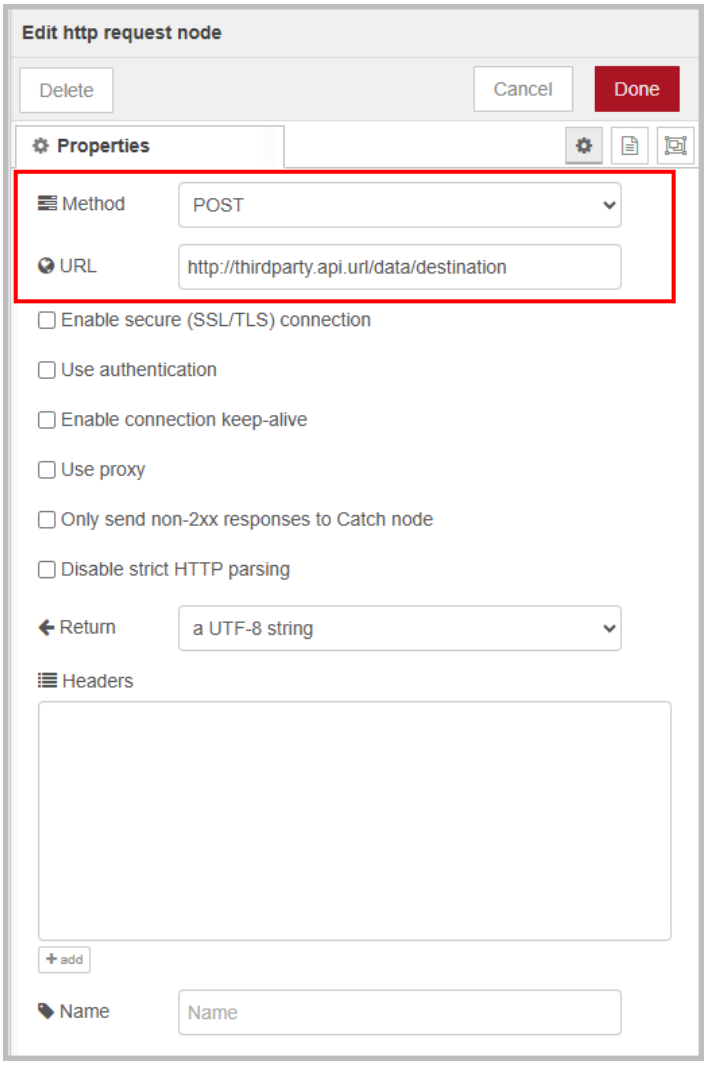
4. 拖拽一个 `function` 节点至画布，编写脚本将获取到的数据按第三方接收API的要求格式化。

```
if (typeof msg.payload === "string") {
    msg.payload = JSON.parse(msg.payload);
}

msg.headers = {
    "Content-Type": "application/json"
};

return msg;
```

5. 使用 `HTTP request` 节点通过API将数据发送至第三方。

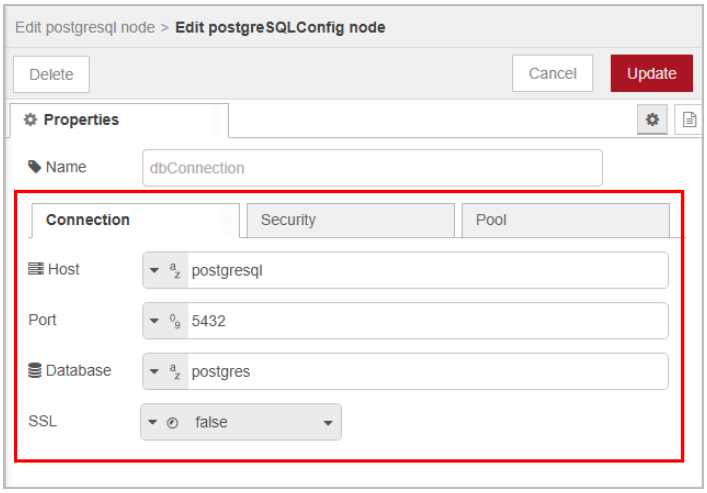


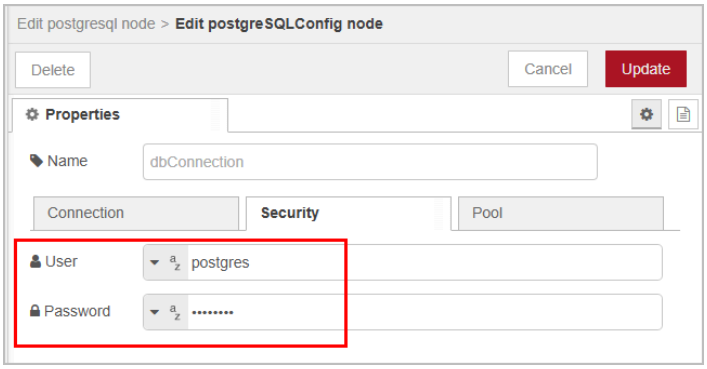
6. 部署流程，当订阅主题数据发生变化时，数据将自动推送至第三方。

直接从平台数据库获取数据

> **信息**
>
> 该方式适用于需要查询历史数据的场景。

1. 登录平台，选择 **流程 > 事件流程**。
2. 单击 **新建流程**，将**流程类型**设为**事件流程**，输入名称后单击 **保存**。
3. 单击刚创建的事件流程，拖拽一个 `inject` 节点。
4. 拖拽一个 `PostgreSQL` 节点至画布，连接平台PostgreSQL数据库：
    - **Connection**
        - 主机：`postgresql`
        - 端口：`5432`
        - 数据库：`postgres`
        - SSL：`false`
    - **Security**
        - 用户名：`postgres`
        - 密码：`postgres`





提示

若未安装 `PostgreSQL` 节点，可在节点管理中搜索 `node-red-contrib-postgresql` 进行安装。

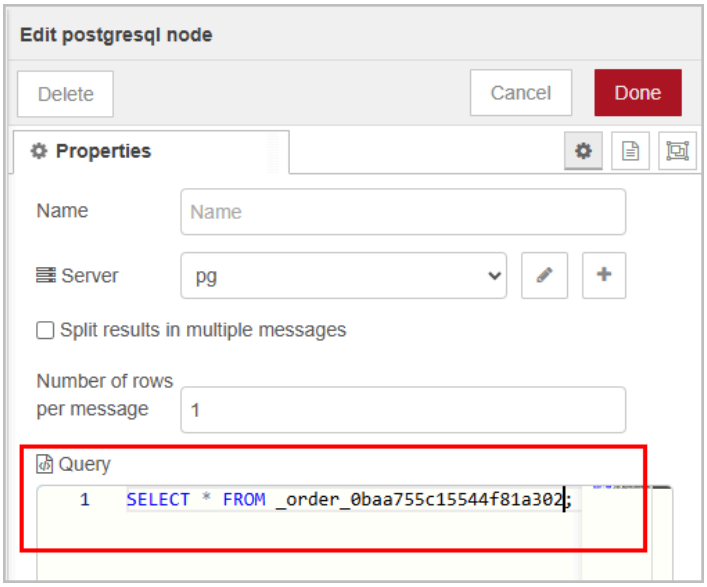
5. 编写SQL语句查询平台数据库中的数据：

信息

表名可从 **数据建模** 下对应文件的**明细**中获取。




```
SELECT * FROM 表名;
```



6. 添加 `HTTP request` 或 `mqtt out` 节点，将查询到的数据传输至第三方。

使用数据

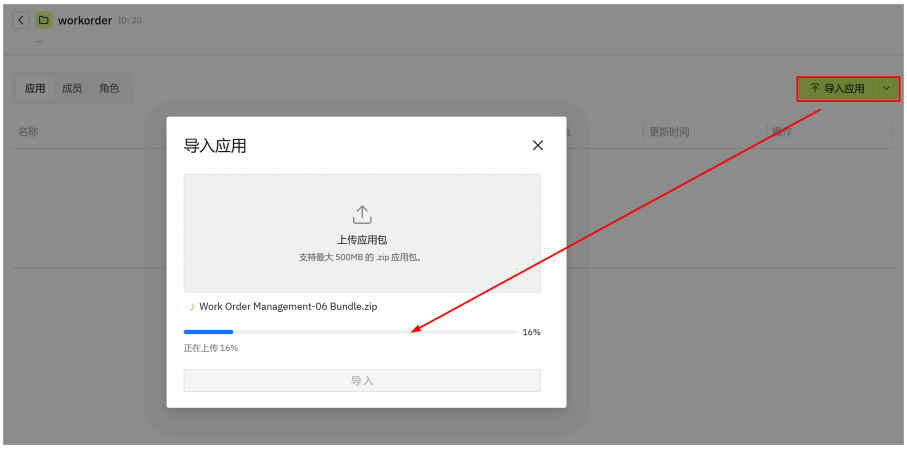
工业应用

Tier0 云服务版提供 App Builder，可通过无代码方式构建工业应用，并将应用导出到本地 Tier0 企业版。

导入应用

> **先获取应用包**
>
> 在 Tier0 云服务版中，导出已构建应用的安装包。详情参见导出应用。

1. 在 Tier0 企业版中，进入 **App** > **项目**，单击 **创建项目**。
2. 输入项目名称，单击 **保存**。
3. 进入项目，单击 **导入应用**，选择从 Tier0 云服务版导出的应用包。

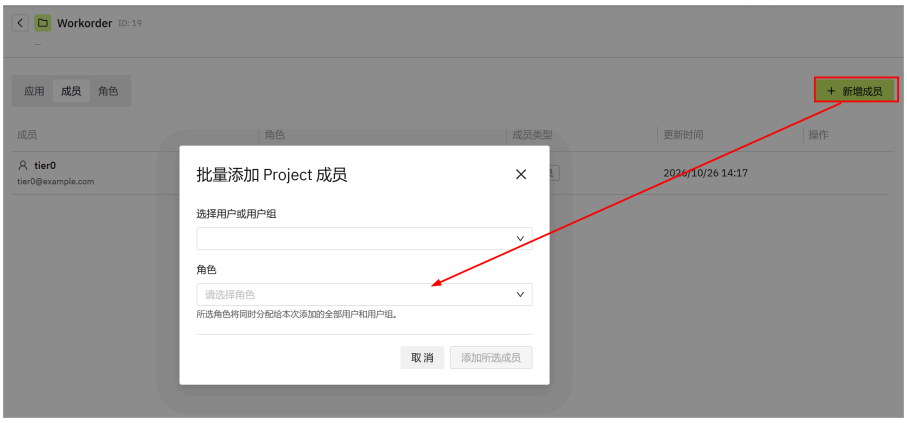


4. 核对应用信息，单击 **导入**。

配置应用

1. 在项目中，进入 **成员** 页签。
2. 添加项目成员并分配角色，使其在授权范围内访问应用。

> **角色**
>
> - 角色在 Tier0 云服务版的应用开发阶段创建。详情参见角色。
> - 项目中的 **角色** 页签会列出这些角色。



访问应用

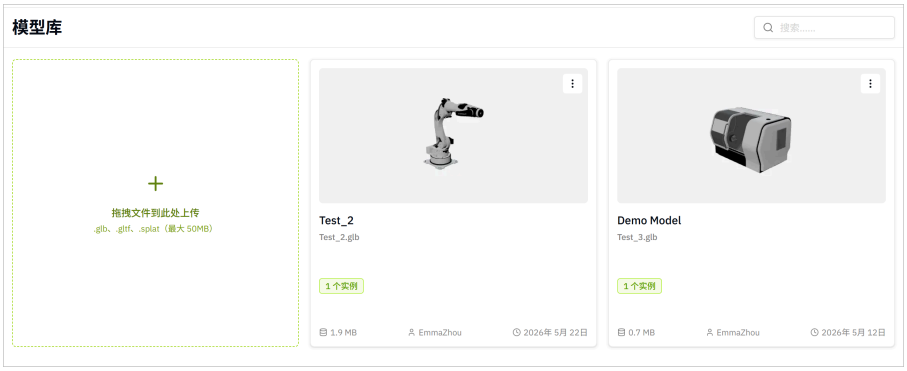
1. 进入 **应用** > **应用中心**，选择项目。
2. 单击要访问的应用。

配置 3D 模型

平台提供数字孪生功能，帮助用户以可视化方式构建数据模型。

上传数字孪生模型

1. 登录 Tier0，选择 **Anchor** > **模型库**。



2. 单击 +，选择模型文件，并将其上传到平台。

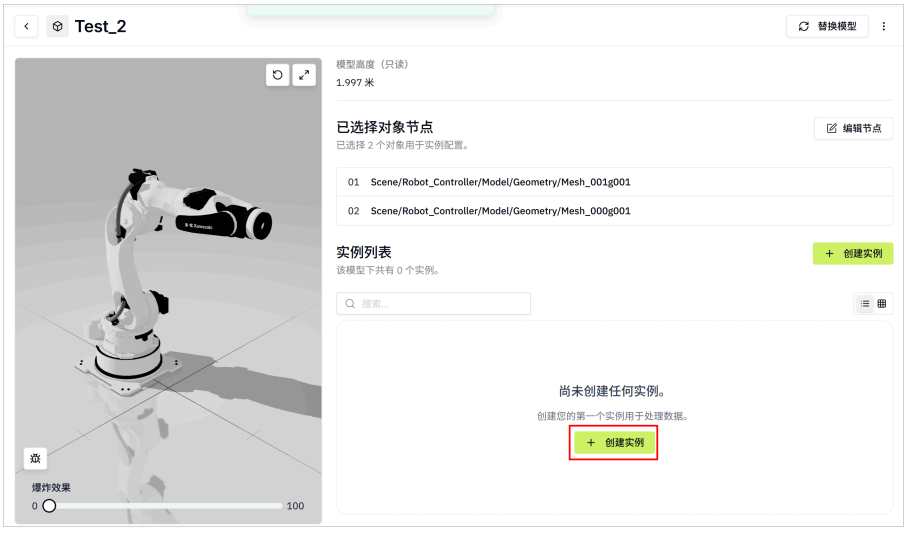
将 UNS 数据映射到数字孪生模型

> **信息**
>
> 3D 模型支持两种数据映射方式。
>
> - **静态映射**：将 UNS 数据作为静态标签显示在场景中。常用于高斯模型（无可动部件但需要显示数据）或结构化模型的静态属性，例如空调温度。
> - **运动映射**：将数据映射到模型的可动部件，部件会根据数据变化而运动。

1. 上传模型后，单击模型，开始建立模型与 UNS 数据的映射。
2. 选择需要映射的模型节点，然后单击 **确认**。



3. 单击 **创建实例**，选择用于控制模型动作的 UNS 数据模型，然后单击 **下一步**。



4. 输入 **实例名称**，设置模型的 **真实高度**，然后单击 **创建**。



运动映射

将 UNS 数据绑定到模型的可动部件。

1. 在 **模型映射** 页签中，单击 ，并选择要映射的模型位置坐标轴。



2. 将数据字段映射到所选坐标轴。



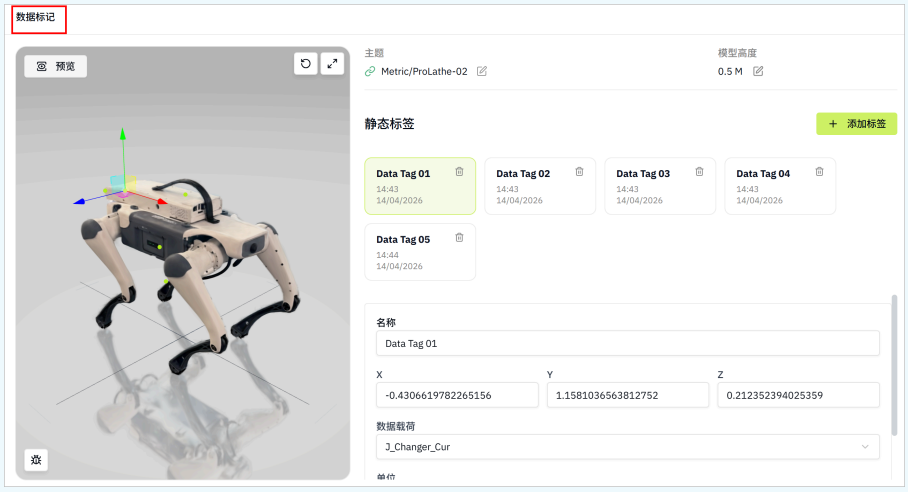
3. 对 **旋转绑定** 执行相同操作，用于确定模型部件如何根据数据变化旋转。

静态映射

将 UNS 数据以静态标签的形式绑定到模型。

信息

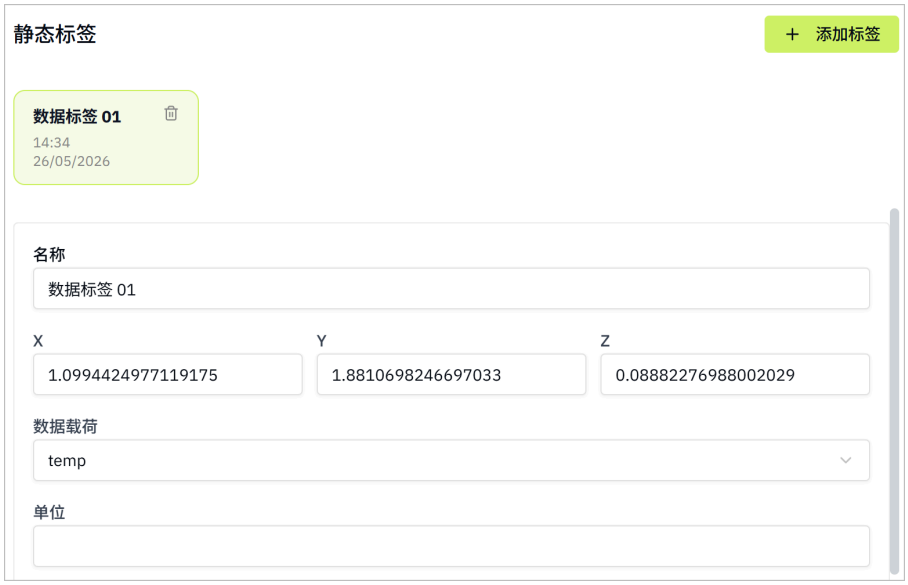
高斯模型仅支持静态映射，因为它们没有可动部件。



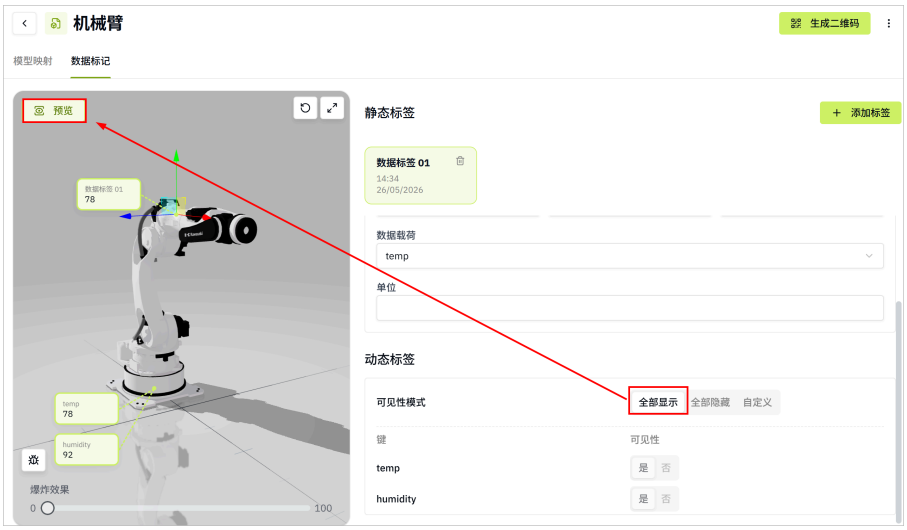
1. 单击 **数据标记**，然后单击 **添加标签**。



2. 单击模型，选择标签表示的位置点，并配置标签属性。



3. 设置运动标签的可见模式，然后单击 **预览**。



查看数字孪生模型

1. 将 UNS 数据映射到模型后，单击右上角的 **生成二维码**。
2. 选择 **分享模型**，然后单击 **生成**。



3. 扫描二维码，即可在移动设备上查看数字孪生模型。



> **信息**
>
> - 当模型添加到场景中后，可以分享场景，以包含位置、灯光和其他设置的方式查看模型。
> - 也可以下载二维码，或复制二维码链接供后续使用。

映射过程中的快捷操作

> **信息**
>
> 节点映射会自动保存。

在模型窗口中

- 单击右上角的 **替换模型**，可以在保留解析后的映射设置的同时替换模型。
- 拖动模型窗口底部的 **爆炸视图** 滑块，查看模型部件。
- 单击 ，将模型显示为原始状态。

- 单击 ，全屏查看模型。

在映射页签中

- 单击右上角的 **编辑节点**，修改已选择的节点。
- 单击 ，断开模型节点与实例的连接。
- 单击 ，为 UNS 文件添加数据，便于使用新数据快速映射。

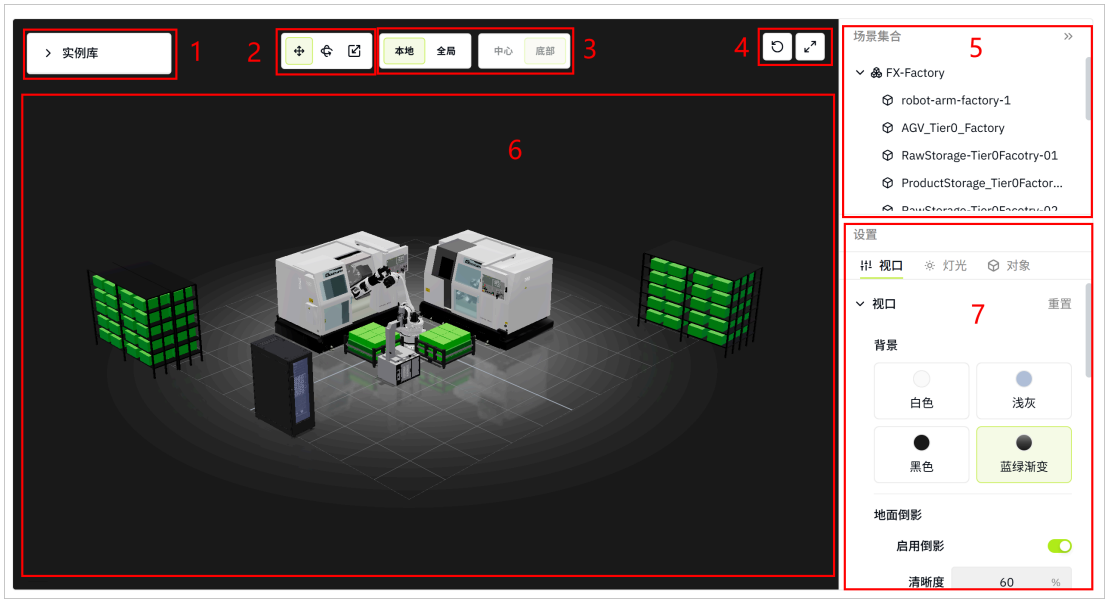


构建 3D 场景

Tier0 支持使用已映射实时 UNS 数据的 3D 模型构建场景，用于实时展示。

其他参数说明

- 场景说明。



编号	模块	项目	说明
1	实例库		列出 **Model Library** 中所有已配置的模型。
2	模型位置	/ /	沿 X、Y、Z 轴移动、旋转或缩放模型。

3	坐标轴	本地/全局	决定模型坐标基于模型自身，还是基于整个屏幕。
		中心/底部	设置坐标位置位于模型中心或底部。
4	通用操作	/	将场景恢复到原始视角/全屏查看场景。
5	场景集合		列出场景中的所有模型。
6	画布		显示场景画布。
7	设置	视口	设置场景视口，包括背景、相机参数和网格布局。
		灯光	在不同预设类型下设置场景中的光照条件。
		对象	显示场景中所选模型的属性。你可以通过手动修改数值来编辑其位置。

- **视口**



参数	说明
背景	场景画布的背景样式。
地面倒影	控制模型下方是否显示反射效果，用于模拟反光地面。
启用倒影	开启或关闭地面反射效果。

参数	说明
清晰度	调整反射的清晰度或锐利程度。数值越高，反射越清晰。
倒影深度	调整反射的垂直范围或深度。数值越高，反射通常向下延伸得越远。

- **相机**



参数	说明
焦距	设置相机焦距。数值越小，视野越宽。
镜头单位	指定镜头设置使用的单位。在该示例中，焦距以毫米为单位。
X/Y 偏移	在不旋转相机的情况下，水平/垂直移动相机视图。正值或负值会将画面向左、向右、向上或向下偏移。
裁剪模式	指定相机裁剪范围的控制方式。在 **Auto** 模式下，系统会根据场景自动计算可见范围。在 **Manual** 模式下，可以使用 **Clip Start** 和 **End** 设置近端和远端裁剪距离。
裁剪起点	定义距离相机最近的可见距离。比该距离更近的对象会被裁剪。
结束	定义距离相机最远的可见距离。超过该距离的对象会被裁剪。

- **网格**



参数	说明
显示网格	是否在场景中显示网格。
显示坐标轴	是否在场景中显示坐标轴。
网格范围	定义网格区域整体可见大小。
最小网格尺寸	定义每个网格单元的最小尺寸。
无限网格	设置网格是否无限延伸，而不是限制在固定区域内。

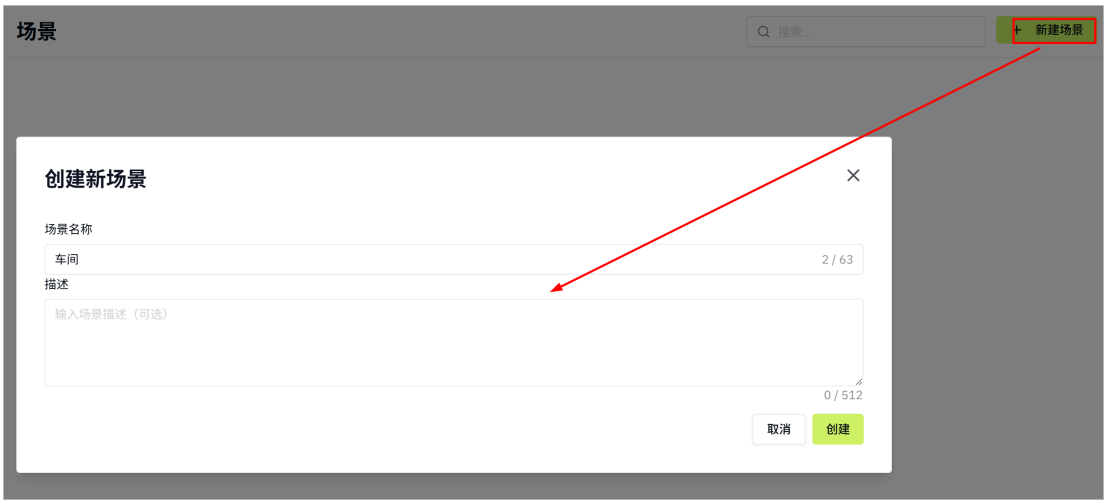
- **灯光**



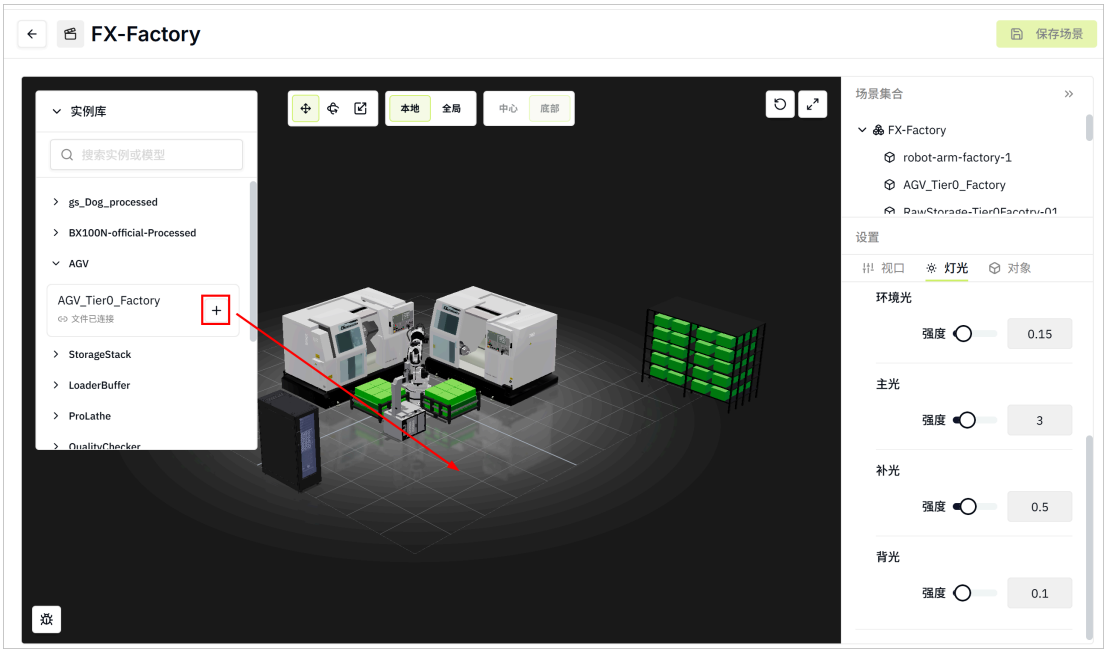
参数	说明
光照预设	选择预设光照方案。可选项包括 **Balanced**、**Soft** 和 **Contrast**。
IBL	启用基于图像的光照。该功能使用环境图像为场景提供更真实的环境光和反射效果。强度值用于控制 IBL 效果的强弱。
环境光	控制环境光强度。环境光为整个场景提供整体的非定向照明。
主光	控制主光源强度。主光源用于塑造对象的主要高光和阴影。
补光	控制补光强度。补光用于柔化主光源产生的阴影，并降低明暗对比。
背光	控制背光强度。背光从对象后方照亮对象，帮助对象与背景分离。

如何构建数字孪生场景

1. 登录 Tier0，选择 **Anchor** > **场景**。
2. 单击右上角的 **新建场景**，输入场景信息，然后单击 **创建**。



3. 打开已创建的场景，从左侧 **实例库** 中选择实例。



4. 配置添加到场景中的模型。

    i. 设置模型坐标。
    ii. 基于所选坐标调整模型的位置、旋转和缩放。
    iii. 在右侧根据需要编辑视口、相机和网格。
    iv. 编辑光照设置。
    v. 可选：通过设置 **对象** > **变换** 下的数值来编辑模型位置。
5. 单击右上角的 **保存场景**。

分析数据

Tier0 采用 Marimo Notebook，支持使用 Python 进行高级数据分析。

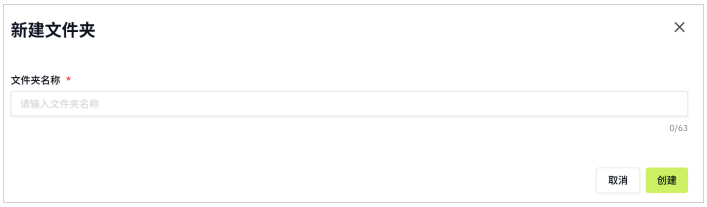
创建 Notebook

1. 登录 Tier0，然后选择 **Notebook**。

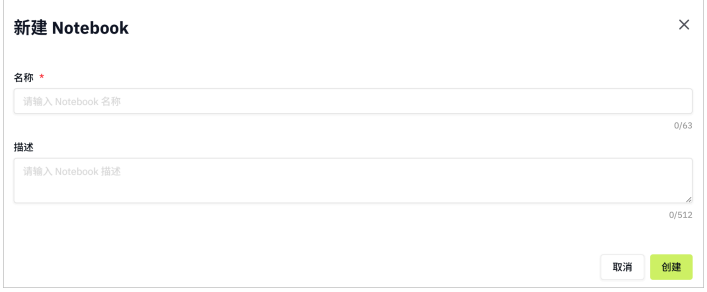


编号	项目	说明
1	分类	按状态显示 Notebook。
2	Notebook 树图	以文件夹-文件层级结构列出 Notebook，支持添加 Notebook、刷新列表和搜索 Notebook。
3	Notebook 列表	详细显示所选文件夹下的所有 Notebook。
4	导入	导入 Notebook。

2. 选择  > **新建文件夹**。

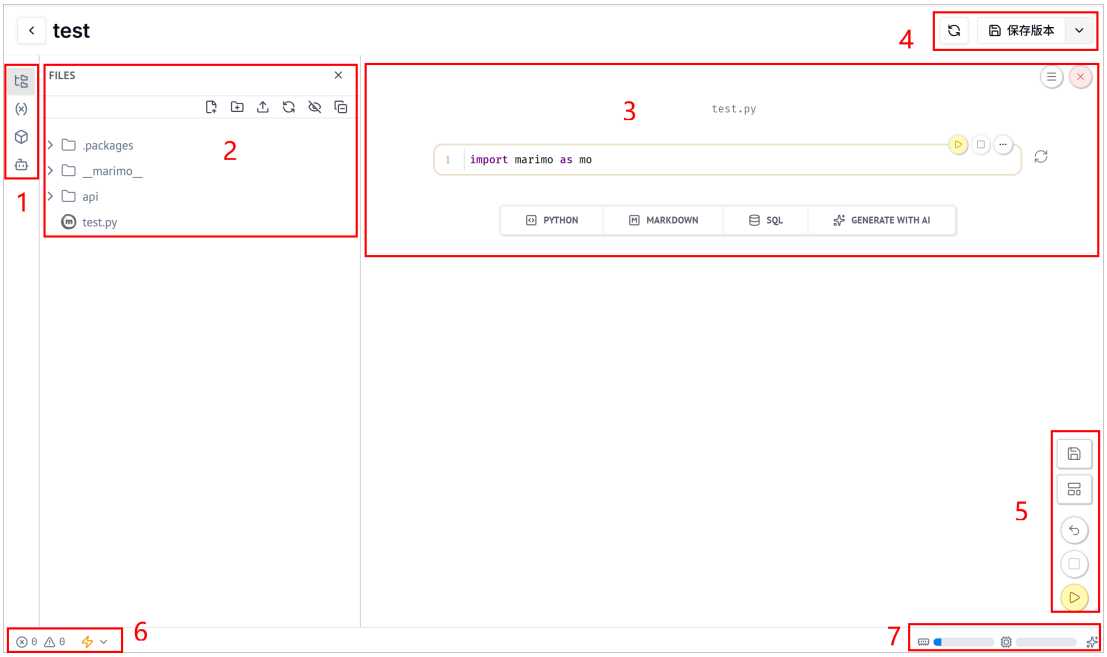


3. 输入文件夹名称，然后单击 **创建**。
4. 单击已创建的文件夹，然后选择  > **新建 Notebook**。



5. 输入 Notebook 信息，然后单击 **创建**。

6. 单击右侧已创建的 Notebook，开始编辑。

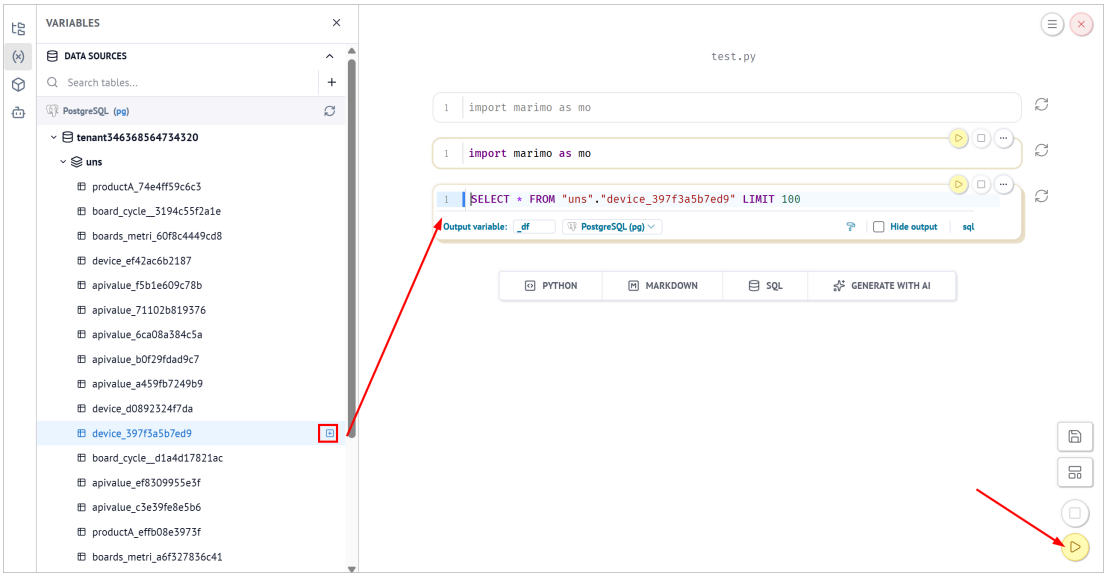


编号	项目	说明
1	模块	显示 Notebook 模块，包括数据源和变量、依赖包以及 AI 机器人。
2	文件	显示 Notebook 层级结构。
3	画布	显示单元格及对应结果。你可以选择创建不同类型的单元格、关闭 Notebook，以及修改 Notebook 设置。
4	工具栏	刷新单元格，并将 Notebook 保存为不同版本。
5	Notebook 操作	保存 Notebook。切换当前查看样式，包括单元格与结果或仅结果。停止或运行所有单元格。
6	开发者模式	进入开发者面板，并设置运行时活动。
7	状态	显示内存、CPU 和 AI 状态。

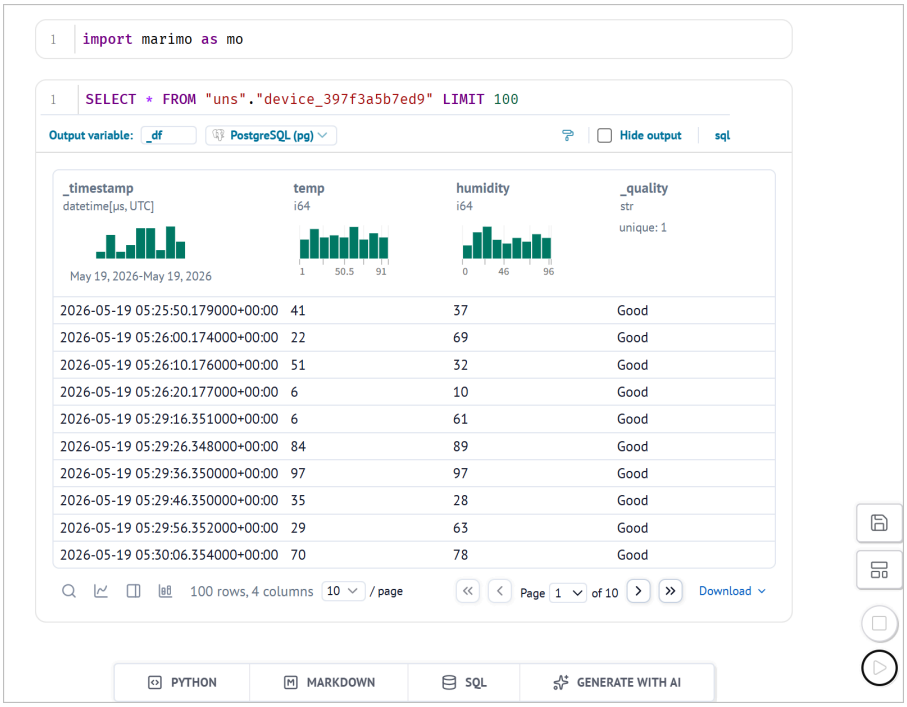
使用 Notebook 执行 SQL 查询

> **开始前须知**
>
> UNS 存储到数据库中的数据会作为 Notebook 中已有的数据源。

1. 登录 Tier0，然后选择 **Notebook**。
2. 创建并进入一个 Notebook，单击左侧的 **VARIABLES**。
3. 在左侧找到需要的表，单击表名旁边的 ⊞，自动生成 SQL 语句。



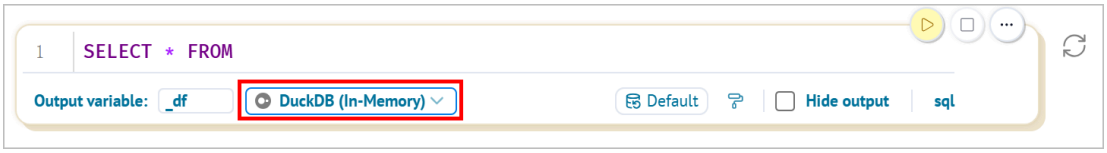
4. 单击右下角的 ，运行所有单元格并查看查询结果。



提示

- 你可以单击左侧 **DATA SOURCES** 下的 ，按需添加数据源。
- 使用 Notebook 时可能缺少部分依赖包。此时，单击左侧的 **Manage packages** 进行安装。

5. （可选）单击 **SQL** 添加新的单元格，然后选择已连接的数据库执行 SQL 语句。



使用 Notebook 分析数据

示例背景

- 数据源是从工厂采集并发送到 Tier0 UNS 的设备温度和湿度。
- 数据分析内容包括设置温湿度风险阈值、根据温湿度的组合数据识别风险，并生成表格和图表，便于查看。

操作步骤

1. 登录 Tier0，然后选择 **Notebook**。
2. 创建一个 Notebook，并在其中添加以下单元格。

- 单元格 1：导入库。

```
import marimo as mo
import pandas as pd
import altair as alt
```

- 单元格 2：从数据库查询数据。

```
SELECT * FROM "uns"."device_15d9d69fd5eb" LIMIT 100
```

- 单元格 3：清洗数据。

```
# 将SQL查询结果转成 pandas DataFrame
if hasattr(device_df, "to_pandas"):
    df = device_df.to_pandas()
elif hasattr(device_df, "to_dicts"):
    df = pd.DataFrame(device_df.to_dicts())
else:
    df = pd.DataFrame(device_df)

# 清洗并准备数据
df["_timestamp"] = pd.to_datetime(df["_timestamp"])
df = df.sort_values("_timestamp").reset_index(drop=True)

df["temp"] = pd.to_numeric(df["temp"], errors="coerce")
df["humidity"] = pd.to_numeric(df["humidity"], errors="coerce")

df = df.dropna(subset=["temp", "humidity"])

df.head()
```

- 单元格 4：设置风险阈值并生成风险状态字段。

```
temp_warning = mo.ui.slider(
    start=0,
    stop=100,
    value=70,
    label="Temperature warning threshold",
)
```

```
temp_critical = mo.ui.slider(
    start=0,
    stop=100,
    value=85,
    label="Temperature critical threshold",
)

humidity_warning = mo.ui.slider(
    start=0,
    stop=100,
    value=70,
    label="Humidity warning threshold",
)

humidity_critical = mo.ui.slider(
    start=0,
    stop=100,
    value=85,
    label="Humidity critical threshold",
)

mo.vstack([
    mo.md("## Threshold Settings"),
    mo.hstack([temp_warning, temp_critical]),
    mo.hstack([humidity_warning, humidity_critical]),
])
```

- 单元格 5：根据单元格 4 中设置的阈值，生成包含风险状态和异常状态的表格。

```
analyzed_df = df.copy()

def classify_risk(row):
    temp = row["temp"]
    humidity = row["humidity"]

    temp_high = temp >= temp_warning.value
    temp_critical_high = temp >= temp_critical.value

    humidity_high = humidity >= humidity_warning.value
    humidity_critical_high = humidity >= humidity_critical.value

    if temp_critical_high and humidity_critical_high:
        return "Critical: High temp + high humidity"
    elif temp_critical_high:
        return "Critical: High temperature"
    elif humidity_critical_high:
        return "Critical: High humidity"
    elif temp_high and humidity_high:
        return "Warning: Combined risk"
    elif temp_high:
        return "Warning: Temperature risk"
    elif humidity_high:
        return "Warning: Humidity risk"
    else:
        return "Normal"

analyzed_df["risk_status"] = analyzed_df.apply(classify_risk, axis=1)

analyzed_df["is_anomaly"] = analyzed_df["risk_status"] != "Normal"

analyzed_df.head()
```

- 单元格 6：生成异常汇总统计信息。

```
total_count = len(analyzed_df)
anomaly_count = analyzed_df["is_anomaly"].sum()
normal_count = total_count - anomaly_count
anomaly_rate = anomaly_count / total_count * 100 if total_count > 0 else 0

critical_count = analyzed_df["risk_status"].str.startswith("Critical").sum()
warning_count = analyzed_df["risk_status"].str.startswith("Warning").sum()

mo.vstack([
    mo.md("## Joint Anomaly Detection Summary"),
    mo.hstack([
        mo.stat(label="Total Records", value=f"{total_count}"),
        mo.stat(label="Normal Records", value=f"{normal_count}"),
        mo.stat(label="Warning Records", value=f"{warning_count}"),
        mo.stat(label="Critical Records", value=f"{critical_count}"),
        mo.stat(label="Anomaly Rate", value=f"{anomaly_rate:.1f}%"),
    ])
])
```

- 单元格 7：生成柱状图，显示不同风险状态的分布。

```
risk_summary = (
    analyzed_df
    .groupby("risk_status")
    .size()
    .reset_index(name="count")
    .sort_values("count", ascending=False)
)

risk_bar_chart = alt.Chart(risk_summary).mark_bar().encode(
    x=alt.X("count:Q", title="Record Count"),
    y=alt.Y("risk_status:N", sort="-x", title="Risk Status"),
    tooltip=["risk_status", "count"]
).properties(
    title="Risk Status Distribution",
    width=700,
    height=300
)

risk_bar_chart
```

- 单元格 8：生成散点图，显示温度和湿度之间的关系，并按风险状态区分点的颜色。

```
scatter_chart = alt.Chart(analyzed_df).mark_circle(size=80, opacity=0.75).encode(
    x=alt.X("temp:Q", title="Temperature"),
    y=alt.Y("humidity:Q", title="Humidity"),
    color=alt.Color("risk_status:N", title="Risk Status"),
    tooltip=[
        alt.Tooltip("_timestamp:T", title="Timestamp"),
        alt.Tooltip("temp:Q", title="Temperature"),
        alt.Tooltip("humidity:Q", title="Humidity"),
        alt.Tooltip("risk_status:N", title="Risk Status"),
    ],
).properties(
    title="Temperature and Humidity Joint Anomaly Detection",
    width=700,
    height=420,
```

```
)

temp_warning_line = alt.Chart(pd.DataFrame({"x": [temp_warning.value]})).mark_rule(
    strokeDash=[6, 4]
).encode(
    x="x:Q"
)

humidity_warning_line = alt.Chart(pd.DataFrame({"y": [humidity_warning.value]})).mark_rule(
    strokeDash=[6, 4]
).encode(
    y="y:Q"
)

scatter_chart + temp_warning_line + humidity_warning_line
```

- 单元格 9：生成表格，显示异常记录详情。

```
anomaly_df = analyzed_df[analyzed_df["is_anomaly"]].copy()

anomaly_df = anomaly_df[
    ["_timestamp", "temp", "humidity", "risk_status"]
].sort_values("_timestamp")

mo.vstack([
    mo.md("## Anomaly Details"),
    mo.md(f"Detected **{len(anomaly_df)}** abnormal records based on the current thresholds."),
    mo.ui.table(anomaly_df, page_size=10),
])
```

- 单元格 10：生成分析结论摘要。

```
most_common_risk = (
    risk_summary.iloc[0]["risk_status"]
    if len(risk_summary) > 0
    else "N/A"
)

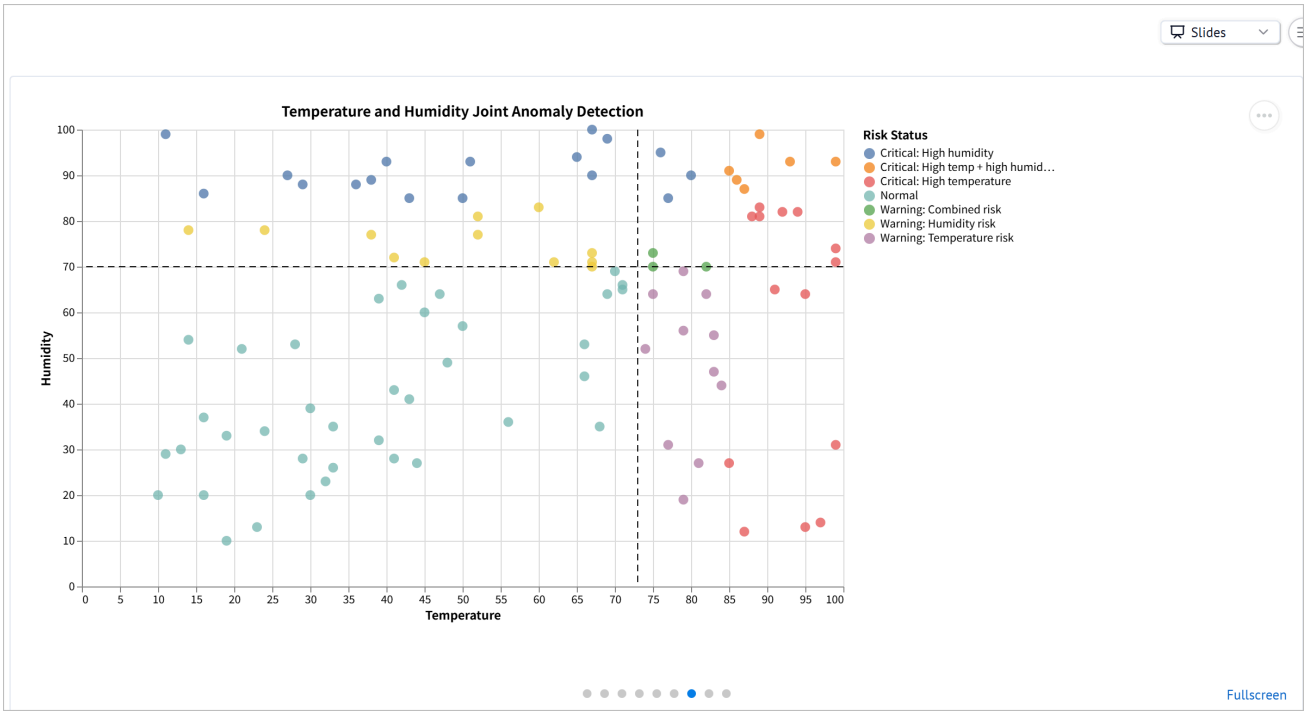
mo.md(f"""
## Analysis Conclusion

Based on the current threshold settings:

- Total records analyzed: **{total_count}**
- Abnormal records detected: **{anomaly_count}**
- Overall anomaly rate: **{anomaly_rate:.1f}%**
- Warning records: **{warning_count}**
- Critical records: **{critical_count}**
- Most common status: **{most_common_risk}**

This analysis uses both temperature and humidity to identify equipment environment risks.
Compared with single-threshold monitoring, joint anomaly detection can better identify combined environmental risks,
such as high temperature and high humidity occurring at the same time.
""")
```

3. 单击右下角的  ，运行所有单元格并查看结果。



提示

在右上角的 **Vertical**、**Grid** 和 **Slides** 中切换查看样式。

视觉分析

你可以将摄像头视频流接入 Tier0，对实时画面进行视觉分析。

连接摄像头

1. 在 Tier0 企业版中，进入 **分析** > **视觉**。
2. 在 **摄像头** 页签中，单击 **添加摄像头**。



3. 输入摄像头信息和视频流地址，测试连接，然后单击 **确定**。

编辑算法

1. 在 **算法** 页签中查看算法列表，找到需要使用的算法。
2. 单击 **详情** 查看算法信息；如需更新模型或添加可识别对象，单击 **编辑**。



3. 单击 **确定**。

分析摄像头画面

1. 在 **分析任务** 页签中，单击 **创建任务**。

> **提示**
>
> 也可以在 **算法** 页签中，单击所选算法的 **使用算法** 来创建任务。

2. 在 **识别配置** 页签中，输入任务名称，选择摄像头和算法。



3. 切换到 **检测区域** 页签，单击图像右下角的加号，再单击图像绘制检测区域。
4. 单击 **完成**，结束区域绘制。



5. 在 **运行与输出** 页签中，配置分析任务的运行计划和输出设置。

> **提示**
>
> 将鼠标悬停在参数上可查看其说明。



6. 单击 **创建**，可在 **分析结果** 页签查看结果。

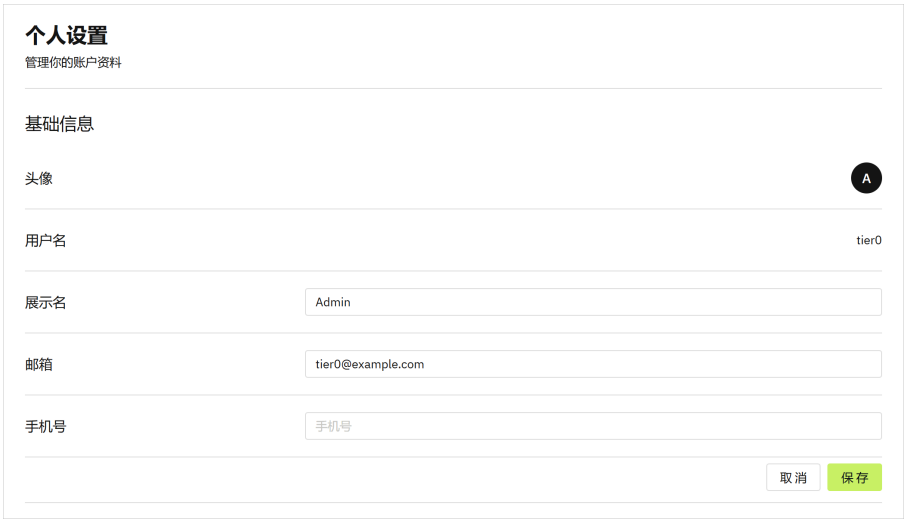
系统设置

用户

在此处，你可以修改个人资料、系统语言和密码。

修改个人资料

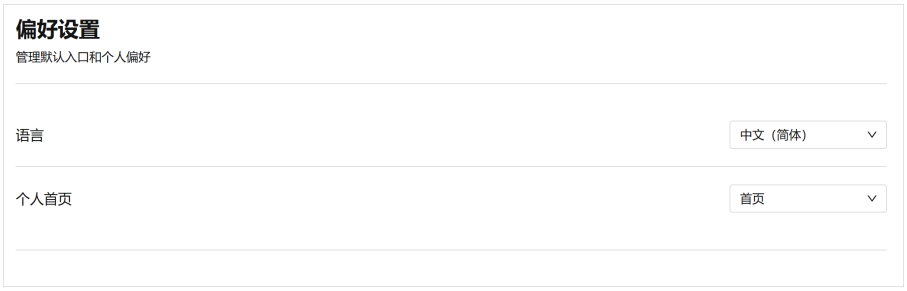
1. 在 Tier0 中，点击右上角的用户图标，然后点击 **个人资料**。
2. 点击 **编辑**，修改显示名称、邮箱地址和电话号码等个人资料。



3. 点击 **保存**。

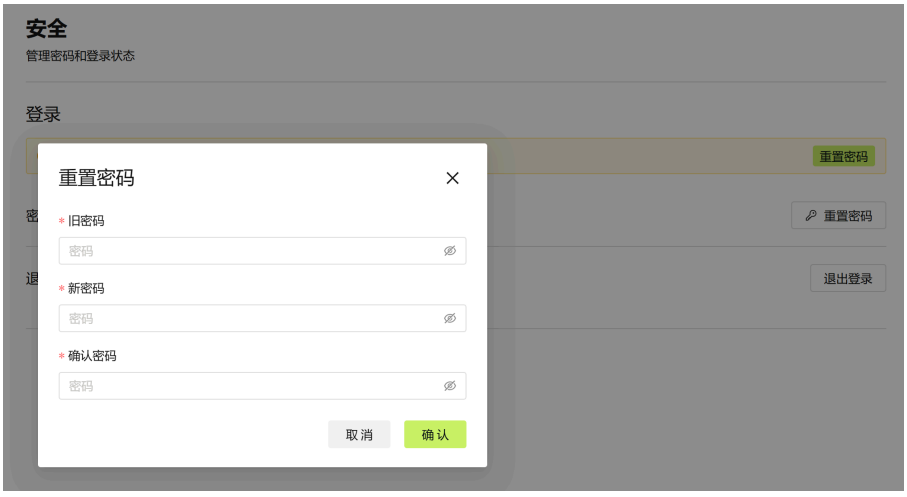
修改偏好设置

1. 在 Tier0 中，点击右上角的用户图标，然后点击 **偏好设置**。
2. 将系统语言切换为中文或英文。
3. 根据需要更改 Tier0 首页。



修改密码

1. 在 Tier0 中，点击右上角的用户图标，然后点击 **安全设置**。
2. 点击 **重置密码**，输入所需密码，然后点击 **确定**。



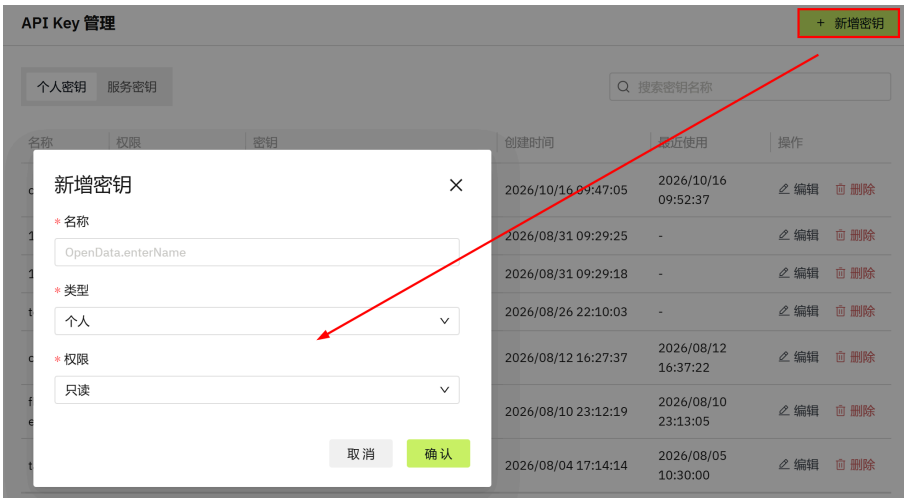
3. （可选）点击 **退出登录**，直接退出当前账号。

服务

设置 API 密钥

> **信息**
>
> 通过 API 和 CLI 访问平台时，需要使用 API 密钥。

1. 在 Tier0 中，点击右上角的用户图标，然后点击 **API 密钥**。
2. 点击 **新建密钥**，设置密钥类型和权限。
    - **服务密钥**：由管理员创建，拥有全部权限。
    - **个人密钥**：由操作员创建，仅有**只读**权限。



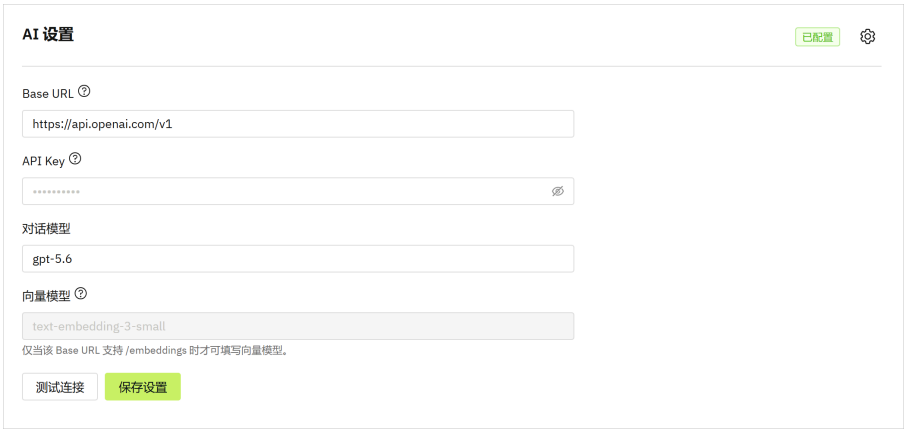
3. 点击 **确定**。

> **提示**
>
> 参考页面底部的指南，了解如何使用 API 密钥通过 API 和 CLI 访问平台。

设置 AI API 密钥

> **警告**
>
> 使用 UNS 智能体前，需要完成此设置。

1. 在 Tier0 中，点击右上角的用户图标，然后点击 **AI 设置**。
2. 填写 AI 模型所需的信息。



> **如需切换服务提供商**
>
> 点击右上角的 ⚙，切换模型服务提供商。

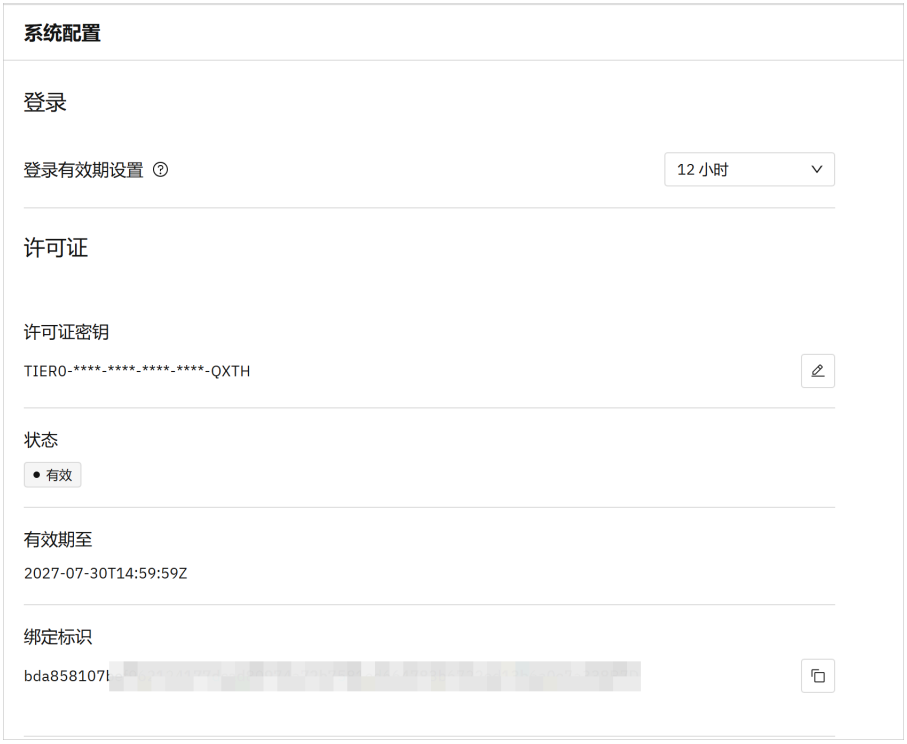
3. 点击 **保存设置**。

平台

系统配置

你可以在此页面完成以下操作：

- **登录**：修改账号保持登录状态的时长。
- **许可证**：
  - 编辑从供应商处获取的许可证密钥，更改 Tier0 许可证类型。
  - 查看许可证状态、到期日期和绑定 ID。

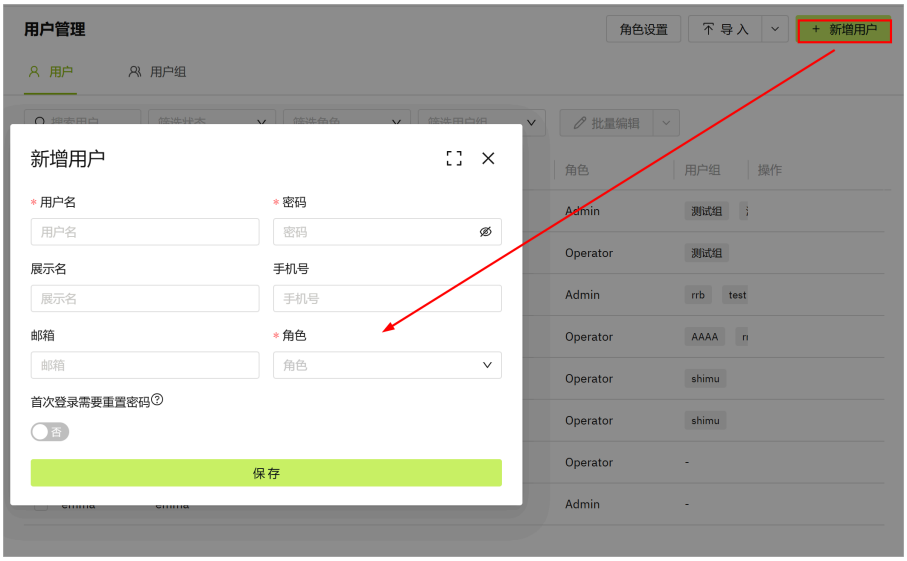


管理用户

在 Tier0 中，点击右上角的用户图标，然后点击 **用户管理**。

新增用户

1. 在 **用户** 页签右上角，点击 **新建用户**。

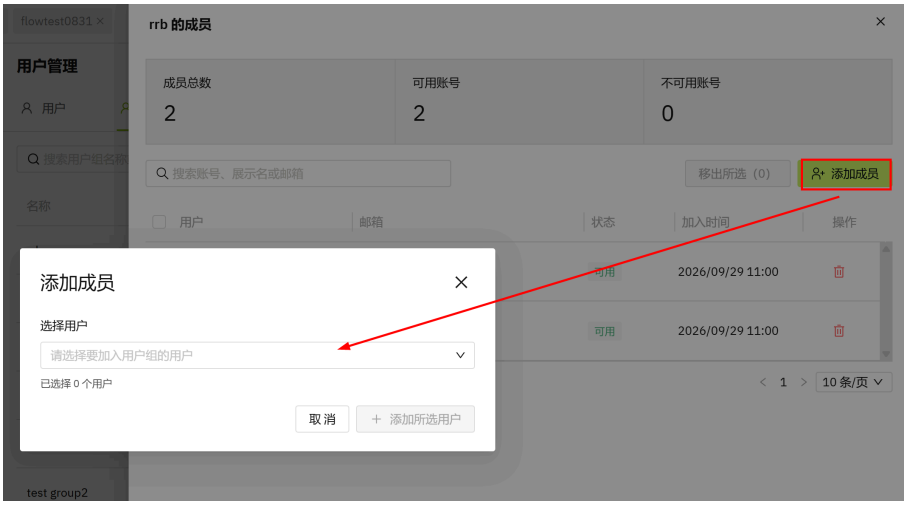


2. 填写用户信息并选择角色，然后点击 **保存**。

- 你可以禁用、编辑、删除用户，以及重置用户密码。
- 默认用户无法禁用或删除。

新增用户组

1. 在 **用户组** 页签右上角，点击 **新建用户组**。
2. 填写信息，然后点击 **保存**。
3. 点击该用户组，然后点击 **添加成员**，将用户加入组中。



4. 点击 **添加选中的用户**。

添加 OAuth 客户端

此功能目前不可用。

查看日志

- 在 **审计日志** 中查看详细的操作日志。
- 在 **登录日志** 中查看登录记录。

设置通知

你可以创建通知，并发送给不同的接收对象。

1. 在 Tier0 中，点击右上角的用户图标，然后点击 **系统消息管理**。
2. 点击 **创建通知**，输入消息标题和内容，并设置发送信息。



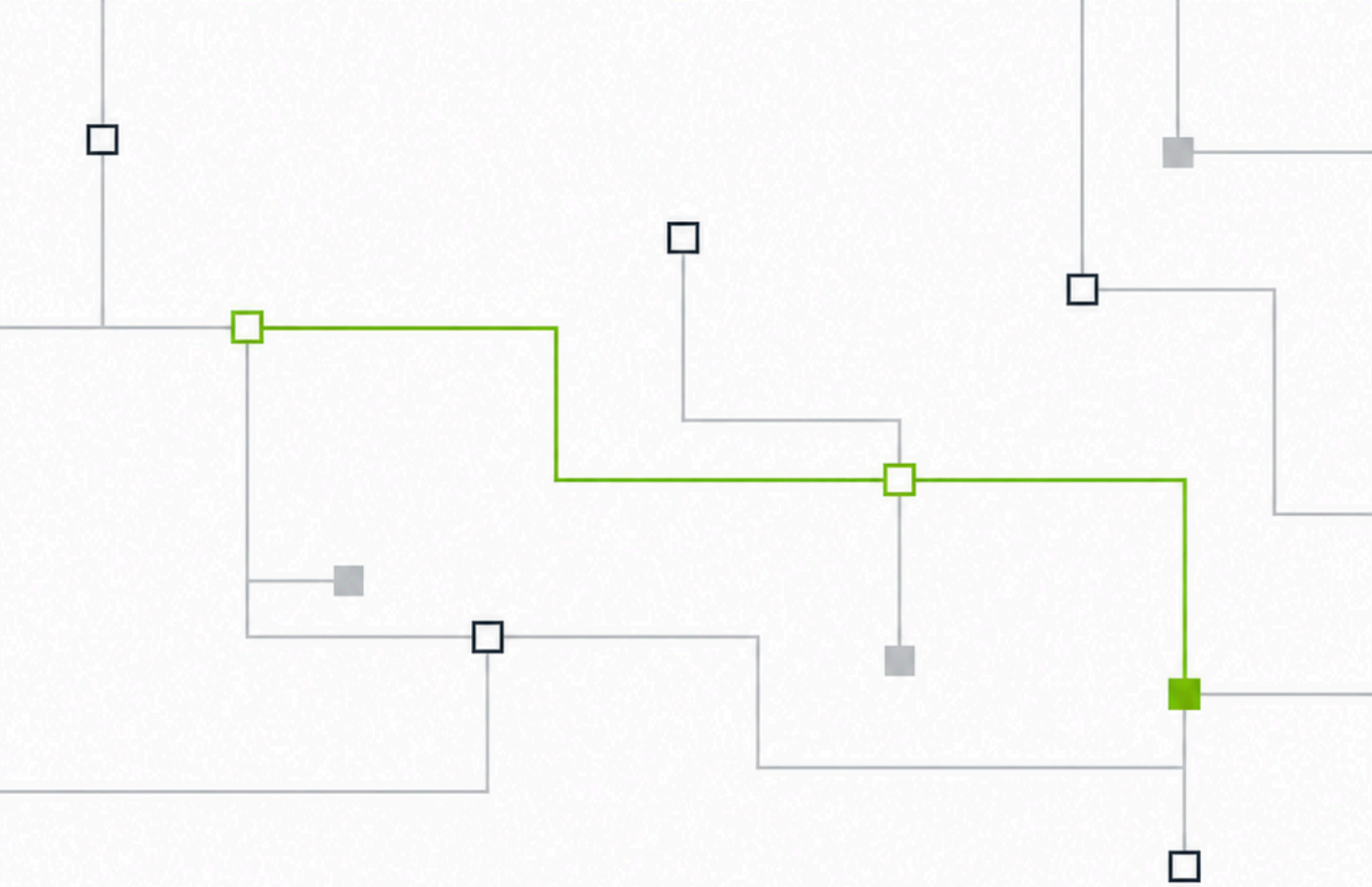
3. 发送通知。

术语表

核心术语及定义

术语	定义
Tier0	一款基于统一命名空间（UNS）方法论构建的开源工业数据集成平台，用于工业数据的管理、处理与优化。
UNS（统一命名空间）	一种数据管理方式，集中整合各类设备与系统的数据，提供结构化的统一框架，避免数据孤岛产生。
Node-RED	一款开源自动化工具，采用流式编程，帮助用户快速连接设备、API 与在线服务。
MQTT	一种轻量级消息传输协议，适用于小型传感器与移动设备，专为高延迟或不稳定网络优化，是物联网（IoT）通信的常用协议。
JSON	JavaScript 对象表示法（JavaScript Object Notation），一种轻量级数据交换格式，易于人类读写，且便于机器解析和生成。

术语	定义
TimescaleDB	一款基于 PostgreSQL 构建的开源时序数据库，专为处理大规模时序数据设计，尤其适用于监控系统与物联网应用。
PostgreSQL	一款高级开源关系型数据库管理系统，采用 SQL 语言查询，具备高性能与高可靠性。
API（应用程序编程接口）	一组协议与工具集，用于不同软件应用之间的通信与交互。
Topic	Tier0 系统中的数据唯一标识，用于对多源数据进行分类整理，方便快速访问与管理。
REST API（REST 接口）	一种 Web 服务接口，通过 HTTP 协议通信，支持 GET、POST、PUT、DELETE 等标准方法访问资源。
Time-Series Data（时序数据）	按固定时间间隔采集记录的数据，例如设备运行时间、温度及其他性能指标数据。
JSON Payload（JSON 消息）	以 JSON 对象格式传输的数据，包含传感器读数、设备状态等需处理或存储的相关信息。
Authentication（身份验证）	验证用户或系统身份的过程，确保只有授权实体才能访问相关资源。
Event-Driven（事件驱动）	一种编程范式，程序的执行流程由事件（如用户操作、传感器数据、其他系统消息等）触发。
Digital Twin（数字孪生）	物理对象、过程或系统的数字化表示，利用数据反映其状态和行为，可用于监测、分析或仿真。
AI Agent（人工智能智能体）	利用人工智能理解目标、作出决策并执行操作的软件系统，通常可以调用工具或与其他系统交互。
Marimo Notebook（Marimo 笔记本）	使用 marimo 构建的交互式 Python 笔记本；单元格会根据依赖项的变化自动更新结果。
API Key（API 密钥）	访问 API 时用于标识和验证应用程序或调用方身份的凭据。
OAuth	一种授权框架，允许应用程序在无需获取用户密码的情况下，获得对用户资源的有限访问权限。
Visual Algorithm（视觉算法）	分析图像或视频数据，以检测、识别、测量或分类对象和模式的算法。
JSONB	PostgreSQL 中的一种数据类型，以二进制格式存储 JSON 数据，相比普通 JSON 字段，具备更高效的索引、更快的查询速度和更优的性能。



TIER 0

tier0.app